

Unity 6000.5 · Entities 6.5

Unity DOTS 入门

Entities、Netcode、ECS 工作流、job、调试、优化与迁移。



Youngmin Cho (SillyToolValley)

简体中文版 · 2026

Unity DOTS 入门

涵盖 Entities 6.5、Netcode for Entities、ECS 工作流、优化与迁移的实战指南。

本简体中文版于 2026-05-21 从 Unity DOTS Introduction 仓库生成。

作者：Youngmin Cho (SillyToolValley) / UCI (Unity Certified Instructor)

Unity、Entities、Netcode for Entities 及相关包名称是 Unity Technologies 的商标或产品名称。本书是社区学习总结资料，并非 Unity 官方文档。

允许公开阅读和引用。未经书面许可，不得进行自动抓取、数据集收录、AI 训练以及 AI 答案采集。

Unity DOTS 入门

一本涵盖 Entities 6.5、Netcode for Entities、ECS workflow、优化与迁移的实战手册。

本书把 Unity DOTS 指南仓库整理成一条连贯的阅读主线，内容始终贴近实战：环境搭建、SubScene 与 Baker workflow、ECS 组件与系统模式、Job 调度、结构性变更的安全性、EntityCommandBuffer 用法、Netcode for Entities、优化、调试、迁移，以及术语表。

本书面向 Unity 6000.5+、Entities 6.5.0、Netcode for Entities 6.5.0，以及随编辑器一同附带的 DOTS 核心包。

如何阅读本书

- DOTS 新手建议先读完第一部分，再依次读第二部分的第 4 至 15 章，然后进入 Netcode 部分。
- 用过 Entities 1.x 的读者可以略读核心章节，直接翻到第四部分看迁移说明。
- 性能优化建议放在主要的 ECS workflow 章节之后再看——熟悉了数据模型，再理解 Chunk 布局和性能分析会顺畅得多。

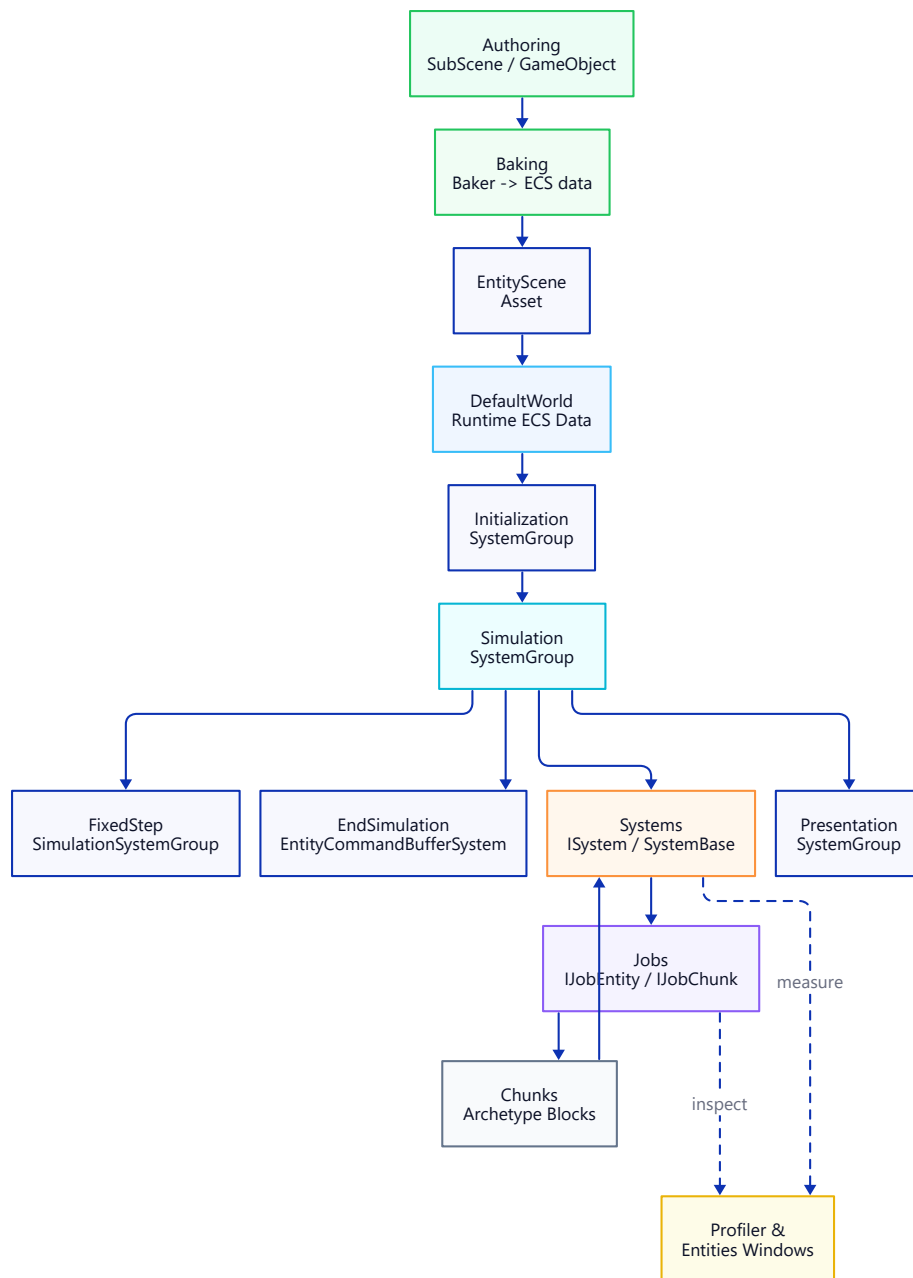


Figure: Unity DOTS Architecture Flow.

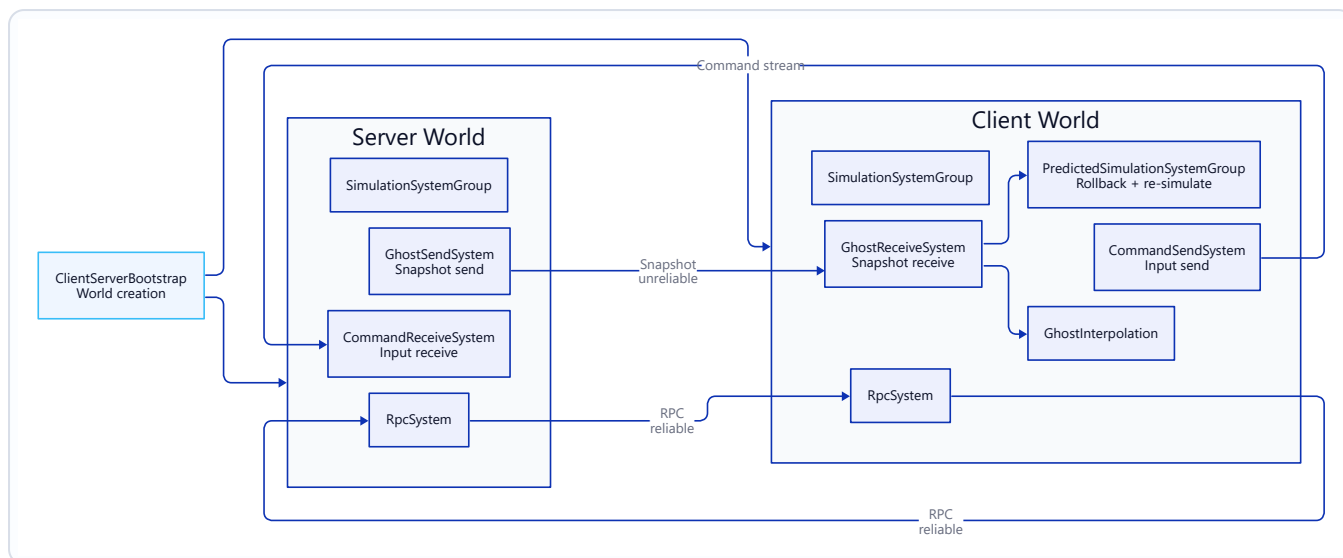


Figure: Netcode for Entities Architecture Flow.

目录

章节	主题
第一部分. 入门	
第 1 章	环境搭建 — Unity 6000.5 + Entities 6.5.0
第 2 章	核心包详解
第 3 章	Hello DOTS — 第一个 Entity
第二部分. DOTS workflow	
第 4 章	Baker 模式 & SubScene
第 5 章	Spawner 示例
第 6 章	ECS 核心概念
第 7 章	Entity 引用 — Entity · EntityPrefabReference · UnityObjectRef
第 8 章	Component 类型
第 9 章	可启用 Component
第 10 章	Singleton Component
第 11 章	System — ISystem 与 SystemBase
第 12 章	System Group 与更新顺序
第 13 章	JobSystem & Burst
第 14 章	IJobEntity · SystemAPI.Query

章节	主题
第 15 章	IJobEntity 与 IJobChunk 对比
第 16 章	结构性变更与安全性
第 17 章	EntityCommandBuffer · 延迟 Entity
第 18 章	ParallelWriter · 确定性回放
第 19 章	Netcode 客户端-服务器 World 与 Bootstrap
第 20 章	Netcode 网络连接与审批
第 21 章	Netcode Ghost 快照与同步
第 22 章	Netcode 预测与回滚
第 23 章	Netcode 命令流与输入
第 24 章	Netcode RPC
第 25 章	Netcode Ghost 优化 · 优先级 · 相关性
第 26 章	Netcode 物理集成与延迟补偿
第 27 章	Netcode Profiler 与调试
第三部分. 优化与调试	
第 28 章	Chunk 布局与 TypeManager
第 29 章	Systems · Entity Inspector · Query 窗口
第 30 章	Profiler · 瓶颈分析
第 31 章	托管对象引用审查
第四部分. 迁移	
第 32 章	Entities 1.x → 6.5 概览
第 33 章	Package Manager → 核心包
第 34 章	托管对象引用 → UnityObjectRef
第 35 章	foreach → IJobEntity 迁移
第 36 章	IAAspect 废弃 — 迁离 Aspect
附录. 变更日志	
附录 A	Entities 1.4 → 6.5 关键变更
附录 B	Netcode for Entities 1.4 → 6.5 关键变更
参考. 术语表	
术语表	术语表

第一部分. 入门

第 1 章. 环境搭建 — Unity 6000.5 + Entities 6.5.0

1. 概述

从 **Unity 6000.4+** 起, Entities 成为 **Core Package**, 随编辑器一同发布, 其版本与编辑器版本绑定, 因此你不必再手动选择 Entities 的版本。不过它**不会被**自动添加到新项目中: 你仍需在每个项目里通过 Package Manager (Unity Registry) 添加一次。本章将带你安装编辑器、创建项目、添加 Entities, 并确认它已正常启用。

本书环境参考: Unity 6000.5.0b2 · Entities 6.5.0 · Collections / Mathematics / Entities Graphics (均为 Core Package) 。

2. 安装 Unity 6000.5+

- 1. 打开 **Unity Hub**。若还没装, 先到 <https://unity.com/download> 安装 Unity Hub。
- 2. 在 Hub 中进入 **Installs → Install Editor**。
- 3. 挑一个 **6000.5.x** 版本 (b 系列测试版或更新的版本均可) 。Entities 6.5 只在 6000.5+ 上提供。
- 4. 模块保持默认即可, ECS 本身不需要额外模块。

在安装编辑器时**不需要**勾选任何叫"DOTS"的模块——从 6000.4 起, Entities 包随编辑器一同发布。你仍需稍后把它添加到每个项目中 (见后续步骤) 。

3. 创建新项目

依次点击 **Hub → Projects → New project**:

字段	值
Editor Version	6000.5.x
Template	Universal 3D (或 3D (Built-in) 、 HDRP ——均可使用)
Project Name	任意名称

点击 **Create project**，编辑器会打开一个空场景。

在 6000.4+ 上，过去那个专用的"DOTS 模板"已经不再需要，任何模板都能承载 Entities 工作流。

4. 添加 Entities 包并验证其已激活

打开 **Window** → **Package Manager**，把范围切换到 **Unity Registry**，选中 **Entities**，然后点击 **Install**（也可以用 + → **Add package by name...** 并输入 `com.unity.entities`）。由于 Entities 是 Core Package，你无需选择版本——编辑器会安装与之捆绑的那个版本（在 Unity 6000.5 上为 Entities 6.5）。安装 Entities 时还会一并引入 **Collections**、**Mathematics** 和 **Entities Graphics** 作为依赖。

接着把范围切换到 **In Project**，确认 **Entities**、**Collections**、**Mathematics** 和 **Entities Graphics** 都列在其中。若想从文件层面确认，可以打开 `Packages/manifest.json`：现在里面会有一个 `com.unity.entities` 条目（它所引入的依赖会被自动解析，可能不会各自单独出现条目）。

接着打开几个 ECS 窗口：

- **Window** → **Entities** → **Hierarchy**——运行时显示 Entity 列表。
- **Window** → **Entities** → **Systems**——显示已注册的 System。
- **Window** → **Entities** → **Archetypes**——显示 Chunk 布局。

这三个菜单都在，说明 Entities 已经配置妥当。

5. 冒烟测试——创建 SubScene

1. 在 Hierarchy 中右键 → **New Sub Scene** → **Empty Scene...**
2. 命名为 `MainSubScene.unity` 并保存。
3. 在新建的 SubScene 里放一个基本体（**3D Object** → **Cube**）。
4. 保存场景。一旦 SubScene 关闭（图标变黄）或进入构建，Cube 就会被 **Bake** 成 Entity。
5. 按下 **Play**，打开 **Window** → **Entities** → **Hierarchy**，能看到一个 Cube Entity，冒烟测试就通过了。

SubScene 是 Authoring 的入口：其中的 GameObject 会在保存时和构建时被 Bake 成 Entity。后续教程 03_Hello DOTS – First Entity.md 会在此基础上继续展开。

6. 故障排查

症状	原因 / 解决方法
Window → Entities 菜单缺失	Entities 尚未安装（通过 Package Manager → Unity Registry → Entities 添加），或者编辑器版本低于 6000.4（升级编辑器即可）。
Baker<T> / IComponentData 未找到	程序集定义文件没有包含 <code>Unity.Entities</code> 。把它加进引用，或者临时删掉 <code>asmdef</code> 做个快速验证。
SubScene 一直停留在 GameObject 状态，从不 Bake	只有 SubScene 关闭 （图标变黄）后，Bake 出的 Entity 才会出现。用 Hierarchy 中名称旁的复选框开关 SubScene。
Entities Hierarchy 窗口在 Play 模式下不显示任何内容	确认 SubScene 已包含在当前场景中，并把 Window → Entities → Hierarchy 的"World"下拉菜单设为 <code>DefaultGameObjectInjectionWorld</code> 。
构建错误： <code>com.unity.entities</code> <code>not found</code>	该包尚未安装，或者 <code>manifest.json</code> 中固定的是此编辑器上并不存在的 1.x 版本。从 Package Manager 安装 Entities，让该条目解析到编辑器的 Core 版本。详见 Migration/02_Package Manager → Core Package.md 。

第 2 章. 核心包详解

1. 概述

随着 **Entities 6.4.0** 发布（与 Unity Editor 6000.4 一同推出），几个主要的 DOTS 包成为 **Core Package**——它们现在随编辑器一同发布（从编辑器安装包中安装，而非从远程注册表下载），其版本与编辑器绑定。包变更日志的原话是：

"Package is now a core package embedded in Unity."

受影响的包包括 `com.unity.entities`、`com.unity.collections`、`com.unity.mathematics`、`com.unity.entities.graphics`，以及 Netcode for Entities（在其对应的 6.5 版本中跟进）。它改变了这些包的安装、版本管理和追踪方式——但这只是**分发方式**的变化，并非 API 变化。

本节说明：

- 哪些包现在算 Core Package，哪些不算。
- 它们的版本号由什么决定。
- `manifest.json` 和 `packages-lock.json` 会有哪些变化。
- 为什么要做这次调整。

2. 什么是 Core Package?

Core Package 指**随编辑器一同发布**的包。你仍要像添加其他包一样，通过 Package Manager 把它添加到项目中，但它从编辑器安装包中安装，而非从远程注册表下载；你无法脱离编辑器单独调整它的版本，它的发布节奏也与编辑器一致。（这与 **Built-in 包**不同——后者指像 Physics 那样、你只能启用或禁用的引擎模块。）

特性	Package Manager 包	Core Package
安装步骤	通过 UPM 添加	通过 Package Manager 添加 (从编辑器安装包中安装)
版本由谁决定	你自己，在 <code>manifest.json</code> 中指定	Unity，按编辑器版本决定
发布节奏	UPM 独立发布	随编辑器发布

特性	Package Manager 包	Core Package
出现在 manifest.json 中	是	是——安装时会写入
出现在 packages-lock.json 中	是	是，标为 builtin
在 Package Manager 中显示	在 "In Project" 下	在 "Unity Registry" 下安装，随后出现在 "In Project" 下

3. Unity 6000.5+ 上的 DOTS 包映射

包	在 Unity 6000.5+ 上	用途
Entities	Core Package (6.5)	ECS 运行时：Entity、Component、System、World
Collections	Core Package	原生容器类型（NativeList<T>、NativeHashMap<K,V> 等）
Mathematics	Core Package	Burst 优化的 float3、quaternion、int4 等
Entities Graphics	Core Package	通过 SRP 渲染基于 Entity 的图形
Netcode for Entities	Core Package (6.5)	面向 Entity 的客户端-服务器联网
Unity Physics	Package Manager	面向 Entity 的无状态确定性物理
Character Controller	Package Manager	基于 ECS 的角色控制器
Burst Compiler	引擎内置功能	把 Job 和 System 编译成紧凑高效的原生代码
Job System	引擎内置功能	多线程 Job 调度

注意：Job System 和 Burst 从来就不是 UPM 包，它们本就是引擎的一部分。6000.4 的这次变化只针对构建在它们之上的 *DOTS* 包。

4. 过渡后的版本号

Core Package 的版本号与编辑器对齐：

编辑器	Core Entities
Unity 6000.4	Entities 6.4
Unity 6000.5	Entities 6.5

对于暂时还无法迁移到 6000.4+ 的项目，旧的 1.x 系列依然可用。两条版本线相互独立；要升级到 6.x，就升级编辑器。

Entities 的过渡说明见 [Changelog/Entities 1.4 → 6.5 Key Changes.md](#)，Netcode 的专项变更见 [Changelog/Netcode for Entities 1.4 → 6.5 Key Changes.md](#)。

5. 对项目文件的影响

manifest.json

当你添加 Entities 时，这里会出现一个 `com.unity.entities` 条目。它所引入的包（`com.unity.collections`、`com.unity.mathematics`、`com.unity.entities.graphics`）会作为依赖被解析，通常无需各自单独的条目。升级 1.x 项目时，替换掉旧的固定版本，让 Package Manager 把它们解析到编辑器的 Core 版本，详见 [Migration/02_Package Manager → Core Package.md](#)。

packages-lock.json

Core Package 会以 `"source": "builtin"` 的形式出现，而不是一个版本 URL。

程序集定义 (.asmdef)

.asmdef 文件照旧引用程序集名称（如 `Unity.Entities`、`Unity.Collections`），这里什么都不用改——变的只是包的分发层。

6. 此次变更的原因

官方给出的目的是：把 DOTS 更新绑定到编辑器的发布周期，而不再走独立的包发布计划，这样 ECS 团队就能更快、更小步地交付功能。同时，引擎也得以在内部依赖 ECS——而当这些包还是可选项时，这种依赖关系处理起来颇为别扭。

7. 故障排查

症状	原因 / 解决方法
Package Manager 只提供旧的 1.x 版本的 Entities	你用的是 6000.4 之前的编辑器。要么升级编辑器，要么继续留在 1.x 系列。

症状	原因 / 解决方法
修改 com.unity.entities 版本后, manifest.json 报解析错误	删除 packages-lock.json, 让编辑器重新生成一份。
某个示例或 Asset Store 包依赖 com.unity.entities@1.x	在 6000.5+ 上, 1.x API 大多已被取代; 可查阅 Migration 文件夹找对应的替代方案。
两个 Editor 安装显示的 Entities 版本不一样	这是正常的, 版本由各自的编辑器安装决定。

第 3 章. Hello DOTS — 第一个 Entity

1. 本章成果

一个绕 Y 轴旋转的 Cube，由 ECS System 驱动。一气呵成的练习中，你会用到所有核心概念：

SubScene、**Authoring** → **Baker**、**IComponentData**、**ISystem**，以及 **Entities Hierarchy / Systems** 窗口。最后，再把主线程 System 换成 **IJobEntity**，看看并行是怎么引入的。

前置条件：[01_Environment Setup.md](01_Environment Setup (Unity 6000.5 + Entities 6.5.0).md)——一个 SubScene 和 Entities 菜单都已确认可用的 Unity 6000.5 项目。

2. 创建 SubScene

1. 打开你的空场景。
2. 在 **Hierarchy** 中右键 → **New Sub Scene** → **Empty Scene...**
3. 保存到 Assets/Scenes/ 下，命名为 HelloDots.unity。
4. 在新建的 SubScene 里右键 → **3D Object** → **Cube**，重命名为 RotatingCube。
5. 保存场景 (Ctrl/⌘ + S)。点击 Hierarchy 中 SubScene 名称旁的复选框关闭它——图标变黄，表示其内容正在被 Bake 成 Entity。

此后，每当你重新打开或关闭 SubScene，对 RotatingCube 组件的改动都会自动触发重新 Bake。

3. 定义 Component——RotationSpeed

新建 Assets/Scripts/RotationSpeed.cs：

```
using Unity.Entities;

public struct RotationSpeed : IComponentData
{
    public float RadiansPerSecond;
}
```


说明：

- 在**非托管** struct 上实现 IComponentData 是首选方案——数据直接存放在 Chunk 中，且对 Burst 友好。
- 字段要保持 blittable（可位块传输）：基础类型、Unity.Mathematics 类型，或其他非托管结构体。

4. 定义 Authoring + Baker

新建 Assets/Scripts/RotationSpeedAuthoring.cs：

```
using Unity.Entities;
using Unity.Mathematics;
using UnityEngine;

public class RotationSpeedAuthoring : MonoBehaviour
{
    public float DegreesPerSecond = 90f;

    class Baker : Baker<RotationSpeedAuthoring>
    {
        public override void Bake(RotationSpeedAuthoring authoring)
        {
            // Dynamic - this entity will have its transform written each frame.
            var entity = GetEntity(TransformUsageFlags.Dynamic);

            AddComponent(entity, new RotationSpeed
            {
                RadiansPerSecond = math.radians(authoring.DegreesPerSecond)
            });
        }
    }
}
```

将 Authoring 附加到 Cube 上

1. 在 SubScene 中选中 RotatingCube。
2. **Inspector** → **Add Component** → **Rotation Speed Authoring**。
3. DegreesPerSecond 保持为 90。

SubScene 一关闭（图标变黄），Baker 就立即运行，把 Cube 转成一个带 RotationSpeed Component 的 Entity。

验证 Bake 结果

- **Window** → **Entities** → **Hierarchy**——里面应该能看到 RotatingCube。

- 点选它，右侧 **Inspector** 会列出它的 Component——找到 `RotationSpeed { RadiansPerSecond: ~1.5708 }`（即 $\pi/2$ ）。

5. 编写 System——RotationSystem

新建 `Assets/Scripts/RotationSystem.cs`：

```
using Unity.Burst;
using Unity.Entities;
using Unity.Mathematics;
using Unity.Transforms;

[BurstCompile]
public partial struct RotationSystem : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        float dt = SystemAPI.Time.DeltaTime;

        foreach (var (transform, speed) in
            SystemAPI.Query<RefRW<LocalTransform>, RefRO<RotationSpeed>>())
        {
            transform.ValueRW = transform.ValueRO.RotateY(speed.ValueRO.RadiansPerSecond * dt);
        }
    }
}
```

说明：

- `partial struct + ISystem` 就是**非托管 System** 的写法。它被 Burst 编译，运行时没有 GC 开销。
- `SystemAPI.Query<RefRW<A>, RefRO<B>>()` 是新版的迭代 API，取代了已废弃的 `Entities.ForEach`。
- `LocalTransform.RotateY(radians)` 返回一个旋转后的副本，再用 `transform.ValueRW = ...` 写回去。

按下 Play

Cube 转起来了。在 **Window** → **Entities** → **Systems** 里，`RotationSystem` 会出现在仿真组下，并显示每帧耗时。

6. 检视成果

进入 Play 模式后，几个常用窗口：

窗口	显示内容
Entities Hierarchy	Entity 的实时列表。点选任意 Entity，即可查看它所在的 Chunk 和 Component。
Systems	所有运行中的 System，按 SystemGroup 分组，并显示每个 System 的耗时。
Archetypes	按 Component 签名分组的 Chunk，便于发现碎片化问题。

不妨花点时间确认：

- 1. RotatingCube Entity 确实存在，并带有 LocalTransform 和 RotationSpeed。
- 2. RotationSystem 出现在 SimulationSystemGroup 下，并在累计帧耗时。

7. 切换为 IJobEntity（并行化）

把 RotationSystem.cs 换成基于 Job 的版本，看看并行是怎么接入的：

```
using Unity.Burst;
using Unity.Entities;
using Unity.Mathematics;
using Unity.Transforms;

[BurstCompile]
public partial struct RotateJob : IJobEntity
{
    public float DeltaTime;

    void Execute(ref LocalTransform transform, in RotationSpeed speed)
    {
        transform = transform.RotateY(speed.RadiansPerSecond * DeltaTime);
    }
}

[BurstCompile]
public partial struct RotationSystem : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        new RotateJob { DeltaTime = SystemAPI.Time.DeltaTime }
            .ScheduleParallel();
    }
}
```

说明：

- `IJobEntity.Execute(ref A, in B)` 经源代码生成后变成一个 Chunk 级别的 Job，参数就是你要操作的 Component。
- `.ScheduleParallel()` 把 Chunk 分摊到各工作线程。只有一个 Cube 时看不出加速，但这套代码形态不用改动就能扩展到上千个 Entity。
- System 的依赖会自动处理——在 `ISystem.OnUpdate` 里调用 `ScheduleParallel()` 时，`state.Dependency` 会自动合并。

8. 后续阅读

如果你想.....	请阅读
详细了解 Authoring 如何映射到 Entity	DOTS Workflows/01_Baker Pattern & SubScene.md
在运行时生成大量 Entity	DOTS Workflows/02_Spawner Example.md
理解 Entity / Component / Chunk	DOTS Workflows/03_ECS Core Concepts.md
在 ISystem 和 SystemBase 之间做取舍	DOTS Workflows/08_System - ISystem vs SystemBase.md

9. 故障排查

症状	原因 / 解决方法
Cube 没出现在 Entities Hierarchy 里	SubScene 还开着（图标是白色，不是黄色）。用 Hierarchy 里的复选框关掉它。
Cube 出现了，却没有 RotationSpeed Component	<code>RotationSpeedAuthoring</code> 没附加，或者附加后 SubScene 没重新 Bake。把 SubScene 复选框关掉再打开一次。
Cube 不旋转	<code>RotationSystem</code> 没出现在 Systems 窗口里。最常见的原因：struct 漏了 <code>partial</code> 关键字，或者脚本有编译错误。
编译错误：TransformUsageFlags does not exist	Authoring 文件漏了 <code>using Unity.Entities;</code> 。
编译错误：math.radians does not exist	漏了 <code>using Unity.Mathematics;</code> 。
<code>SystemAPI.Query<...></code> 未找到	漏了 <code>using Unity.Entities;</code> ，或者 struct 没标记 <code>partial</code> 。
Inspector 显示 Entity 上没有 LocalTransform	Baker 调用的是 <code>GetEntity(TransformUsageFlags.None)</code> ，而非 <code>Dynamic</code> 。改一下这个标志。

症状	原因 / 解决方法
<code>ScheduleParallel()</code> 抛出"LocalTransform 上存在竞争条件"	同一帧里另一个 System 也在写 LocalTransform，却没指定执行顺序。加上 [UpdateBefore]/[UpdateAfter]，或改用共享的 EntityCommandBuffer。

第二部分. DOTS workflow

第 4 章. Baker 模式 & SubScene

1. 概述

Entity 没法直接在 Hierarchy 里编排。你要做的是在 **SubScene** 里编排 **GameObject**，再由 **Baker** 在 SubScene 关闭或构建时把它们转换成 Entity。

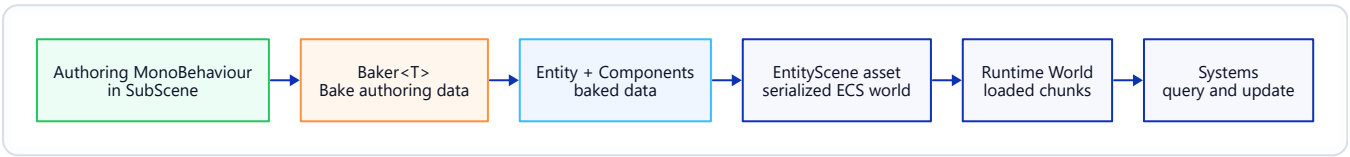


Figure: Baker and SubScene Pipeline.

2. SubScene 生命周期

SubScene 有三种可见状态：

Hierarchy 图标	状态	含义
 (白色复选框)	打开	其中的 GameObject 可编辑；它们仍是 GameObject，尚未 Baking。
 (黄色复选框)	关闭	Baker 已运行；其内容仅以 Entity 形式存在。
	Baking 中	增量 Bake 进行中（瞬间即过）。

创建 SubScene

- **Hierarchy** → **右键** → **New Sub Scene** → **Empty Scene...**——会保存一个 .unity 资产，并在父 GameObject 上添加 SubScene Component。
- 用 SubScene 名称旁的复选框来开关，关闭时就会触发 Bake。

增量 Baking

SubScene 打开时，你编辑某个 GameObject，只有处理过这个被改 Authoring 的 Baker 会重新运行。正因如此，别在 Bake() 里做耗时的活儿。

3. Baker<T> API

一个 Baker 把一个 Authoring MonoBehaviour 映射成一个或多个 Entity 及其 Component。

```
public class EnemyAuthoring : MonoBehaviour
{
    public float MaxHealth = 100f;
    public GameObject ProjectilePrefab;

    class Baker : Baker<EnemyAuthoring>
    {
        public override void Bake(EnemyAuthoring authoring)
        {
            // Primary entity for this GameObject.
            var self = GetEntity(TransformUsageFlags.Dynamic);

            AddComponent(self, new Health { Value = authoring.MaxHealth });

            // Bake a GameObject reference into an Entity reference.
            AddComponent(self, new ProjectilePrefabRef
            {
                Value = GetEntity(authoring.ProjectilePrefab, TransformUsageFlags.Dynamic)
            });
        }
    }
}
```

关键 API:

API	用途
<code>GetEntity(TransformUsageFlags)</code>	取当前 Authoring GameObject 对应的 Entity。
<code>GetEntity(Object, TransformUsageFlags)</code>	取另一个 GameObject/Prefab 对应的 Entity（会注册一个 Prefab Entity）。
<code>CreateAdditionalEntity(TransformUsageFlags)</code>	关联到同一 Authoring 的第二个 Entity（比如子部件）。
<code>AddComponent<T>(entity, value)</code>	附加 Component 数据。
<code>AddBuffer<T>(entity) / SetBuffer<T></code>	附加 DynamicBuffer<T>。
<code>AddSharedComponent<T></code>	附加 ISharedComponentData。
<code>DependsOn(asset)</code>	声明一项资产依赖，资产变更时 Baker 便会重新运行。

4. TransformUsageFlags

只有当你通过 TransformUsageFlags 明确提出请求，Baking 才会添加 LocalTransform / LocalToWorld / Parent。多余的 Transform Component 会白白撑大 Chunk，所以要选满足需求的最小标志。

标志	添加内容	典型用途
None	无	纯数据 Entity（配置、Singleton、纯逻辑）。
ManualOverride	无；Transform 由你自己管理	自定义 Transform 布局。
Renderable	LocalToWorld	从不移动的静态网格。
Dynamic	LocalTransform、LocalToWorld (若有父级则附加 Parent)	会移动、或会被 System 写入的对象。
WorldSpace	LocalTransform、LocalToWorld； 禁止父子关系	不参与层级关系的 Entity。
NonUniformScale	附加 PostTransformMatrix	非均匀缩放（在 DOTS 中不常见）。

凡是会被 System 移动或旋转的对象，用 Dynamic 总没错。

5. Baking System（高级）

一个 Baker 只从单个 Authoring 生成 Component。而 **Baking System** 会在所有 Baker 跑完之后运行，能看到完整的 Baked World——适合做跨 Entity 的后处理，比如解析引用、计算派生数据。

```
[WorldSystemFilter(WorldSystemFilterFlags.BakingSystem)]
public partial struct EnemyWaveLinkSystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {
        // Runs during bake only. Query entities tagged earlier by Bakers
        // and wire them together.
    }
}
```

当 Baker 拿不到它需要的全部信息时（比如计数、交叉引用），就用这种方式。要保证它的确定性——Baking 每次运行都必须产出相同的结果。

6. 示例——为抛射物 Spawner 写 Authoring

Authoring:

```
using Unity.Entities;
using UnityEngine;

public class ProjectileSpawnerAuthoring : MonoBehaviour
```

```

{
    public GameObject Prefab;
    public float Interval = 0.5f;

    class Baker : Baker<ProjectileSpawnerAuthoring>
    {
        public override void Bake(ProjectileSpawnerAuthoring authoring)
        {
            var self = GetEntity(TransformUsageFlags.Dynamic);

            AddComponent(self, new ProjectileSpawner
            {
                Prefab = GetEntity(authoring.Prefab, TransformUsageFlags.Dynamic),
                Interval = authoring.Interval,
                TimeLeft = authoring.Interval
            });
        }
    }
}

```

Component:

```

using Unity.Entities;

public struct ProjectileSpawner : IComponentData
{
    public Entity Prefab;
    public float Interval;
    public float TimeLeft;
}

```

之后，运行时 System 就能在计时器触发时实例化 Prefab。详见 02_Spawner Example.md。

7. 故障排查

症状	原因 / 解决方法
Entity 出现了，却缺少某个 Component	Baker 没运行（Authoring 是 SubScene 关闭后才加上的）。切换一下 SubScene 来重新 Bake。
GetEntity(prefab, ...) 返回 Entity.Null	被引用的 GameObject 既不是 Prefab 资产，也不是 Baker 能访问到的场景对象。确保它是 Prefab，或者就在同一个 SubScene 里。
Baker 在 Editor 中每帧运行	Bake() 里有非确定性逻辑（比如 Random、Time.time）。Baker 必须是 Authoring 的纯函数。
Entity 有 LocalToWorld 但没有 LocalTransform	用的是 TransformUsageFlags.Renderable。Entity 若要移动，改用 Dynamic。
改了 Authoring 字段却不触发重新 Bake	Baker 没读这个字段，或者没对外部资产调用 DependsOn。

症状	原因 / 解决方法
Baking 很慢	Bake() 里做了重活。把跨 Entity 的逻辑挪到 Baking System, 或运行时的一次性初始化器里。

Chapter 5. Spawner 示例

1. 概述

Spawner 就是一个能创建出更多 Entity 的 Entity——可以在启动时创建，可以定时创建，也可以由事件触发创建。本节会从头到尾搭一个完整的 Spawner，涵盖：

- 在 SubScene 里为 Spawner 写 Authoring。
- 保存一个**预制体 Entity** 引用（由 GameObject Prefab Bake 而来）。
- 运行时通过 **EntityCommandBuffer** 实例化，让结构性变更安全地发生。

先读一遍 01_Baker Pattern & SubScene.md 会更容易上手。

2. 数据模型

```
using Unity.Entities;
using Unity.Mathematics;

public struct Spawner : IComponentData
{
    public Entity Prefab;           // baked prefab entity
    public float3 SpawnPoint;
    public float Interval;
    public float TimeLeft;
    public int RemainingCount; // stop when 0
}
```

这里有两个设计取舍值得点出来：

- Prefab 的类型是 **Entity**，不是 GameObject。Baker 会把 GameObject Prefab 转成预制体 Entity，运行时只见 Entity。
 - TimeLeft 和 RemainingCount 都放在 Spawner Entity 上，这样 System 本身保持无状态。
-

3. Authoring + Baker

```
using Unity.Entities;
using Unity.Mathematics;
using UnityEngine;
```

```

public class SpawnerAuthoring : MonoBehaviour
{
    public GameObject Prefab;
    public float      Interval = 0.5f;
    public int        Count    = 100;

    class Baker : Baker<SpawnerAuthoring>
    {
        public override void Bake(SpawnerAuthoring authoring)
        {
            var self = GetEntity(TransformUsageFlags.Dynamic);

            AddComponent(self, new Spawner
            {
                Prefab          = GetEntity(authoring.Prefab, TransformUsageFlags.Dynamic),
                SpawnPoint      = authoring.transform.position,
                Interval        = authoring.Interval,
                TimeLeft        = 0f,           // fires immediately on first update
                RemainingCount  = authoring.Count
            });
        }
    }
}

```

把 SpawnerAuthoring 挂到 SubScene 里的一个空 GameObject 上，指定 Prefab，设好数量。关闭 SubScene，Spawner 就成了一个 Entity。

4. 运行时 System

```

using Unity.Burst;
using Unity.Entities;
using Unity.Mathematics;
using Unity.Transforms;

[BurstCompile]
public partial struct SpawnerSystem : ISystem
{
    [BurstCompile]
    public void OnCreate(ref SystemState state)
    {
        state.RequireForUpdate<Spawner>();
    }

    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        float dt = SystemAPI.Time.DeltaTime;

        // Use the EndSimulation ECB so structural changes flush at a safe point.
        var ecb = SystemAPI
            .GetSingleton<EndSimulationEntityCommandBufferSystem.Singleton>()
            .CreateCommandBuffer(state.WorldUnmanaged);

        foreach (var (spawnerRW, spawnerEntity) in

```

```

        SystemAPI.Query<RefRW<Spawner>>().WithEntityAccess())
    {
        ref var spawner = ref spawnerRW.ValueRW;

        if (spawner.RemainingCount <= 0)
        {
            ecb.DestroyEntity(spawnerEntity);    // done – remove the spawner
            continue;
        }

        spawner.TimeLeft -= dt;
        if (spawner.TimeLeft > 0f) continue;

        spawner.TimeLeft = spawner.Interval;
        spawner.RemainingCount--;

        var instance = ecb.Instantiate(spawner.Prefab);
        ecb.SetComponent(instance, LocalTransform.FromPosition(spawner.SpawnPoint));
    }
}

```

要点:

- `RequireForUpdate<Spawner>()` 让 System 在没有 Spawner 时直接跳过。
- `EndSimulationEntityCommandBufferSystem` 的 Singleton 提供一个共享 ECB，在帧末统一刷新——比在循环中途直接用 `EntityManager` 改动更安全。
- `ecb.Instantiate(prefab)` 创建一个**延迟 Entity**；`ecb.SetComponent(instance, ...)` 则为其排上 Component 写入。instance 句柄只在这个 ECB 回放期间有效。详见 `14_EntityCommandBuffer · Deferred Entity.md`。

5. 扩大规模——并行 Spawner

如果要每帧从大量 Spawner 生成数千个 Entity，就改写成带并行 ECB Writer 的 `IJobEntity`：

```

[BurstCompile]
public partial struct SpawnJob : IJobEntity
{
    public float DeltaTime;
    public EntityCommandBuffer.ParallelWriter ECB;

    void Execute([ChunkIndexInQuery] int chunkIndex,
                Entity entity,
                ref Spawner spawner)
    {
        if (spawner.RemainingCount <= 0)
        {
            ECB.DestroyEntity(chunkIndex, entity);
            return;
        }
    }
}

```

```
    spawner.TimeLeft -= DeltaTime;
    if (spawner.TimeLeft > 0f) return;

    spawner.TimeLeft = spawner.Interval;
    spawner.RemainingCount--;

    var inst = ECB.Instantiate(chunkIndex, spawner.Prefab);
    ECB.SetComponent(chunkIndex, inst, LocalTransform.FromPosition(spawner.SpawnPoint));
}
}
```

System 部分则是：

```
var ecb = SystemAPI
    .GetSingleton<EndSimulationEntityCommandBufferSystem.Singleton>()
    .CreateCommandBuffer(state.WorldUnmanaged)
    .AsParallelWriter();

new SpawnJob { DeltaTime = SystemAPI.Time.DeltaTime, ECB = ecb }
    .ScheduleParallel();
```

chunkIndex 排序键正是并行回放能保持确定性的关键。详见 15_ParallelWriter · Deterministic Playback.md。

6. 故障排查

症状	原因 / 解决方法
什么都没生成	Spawner Entity 不存在。打开 SubScene，确认 SpawnerAuthoring 已挂上，且 SubScene 已关闭（图标为黄色）。
Prefab 都生成在世界原点	ECB 一直没设置 LocalTransform。加上 <code>ecb.SetComponent(instance, LocalTransform.FromPosition(...))</code> ，或者让 Prefab 本身就带有目标位置的 Transform。
Prefab 可见但不移动	Prefab Bake 时用的是 <code>TransformUsageFlags.Renderable</code> ，而不是 <code>Dynamic</code> 。
<code>SystemAPI.GetSingleton<...ECBS.Singleton>()</code> 抛出异常	ECB System 没注册——常见于被大幅精简的 World。改用 <code>state.EntityManager</code> 加手动 ECB，或在 Bootstrap 里注册 ECB System。
Spawner 永不停止	<code>RemainingCount</code> 是在缓存的 Spawner 副本上递减的，而不是通过 <code>spawnerRW.ValueRW</code> 。务必通过 <code>RefRW.ValueRW</code> 写回。
<code>ecb.SetComponent</code> 对延迟实例不起作用	<code>Instantiate</code> 和 <code>SetComponent</code> 用的是不同的 ECB。这两次调用必须用同一个 ECB 实例。

Chapter 6. ECS 核心概念

1. 五个核心名词

Entities ECS 归根结底就是五个核心名词：

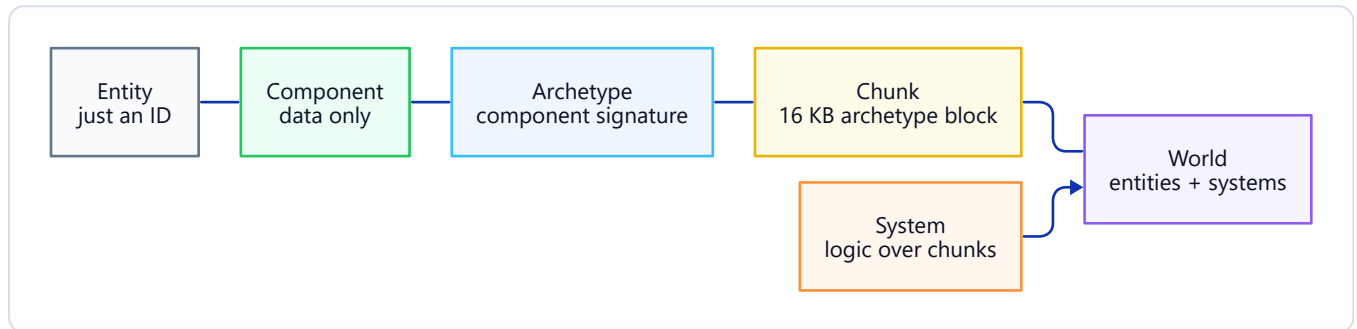


Figure: ECS Core Nouns.

把这张图吃透，读懂现有的 DOTS 代码就成功了一大半。

2. Entity

Entity 不过是一个 64 位句柄：`struct Entity { int Index; int Version; }`。它自身既无数据也无方法，只承担身份标识这一项职责——所有数据都放在 **Component** 里。

```
Entity e = state.EntityManager.CreateEntity(); // fresh, no components
state.EntityManager.AddComponent<Health>(e); // now it has data
state.EntityManager.DestroyEntity(e);         // handle becomes stale
```

Version 字段负责让旧引用失效：如果在同一个 **Index** 上销毁再重建 **Entity**，原来那个 **Entity** 值就对不上了。

3. Component

Component 是个纯数据容器，不带任何行为。逻辑代码都写在 **System** 里。

```
public struct Health : IComponentData
{
    public float Value;
```

```
public float Max;
}
```

Component 有好几种变体：非托管、托管、Buffer、共享、清理、Tag、Chunk、可启用。详见 05_Component Types.md。

4. Archetype

Archetype 指一组 Entity 所共有的那一套 Component 类型组合。

- 拥有 { LocalTransform, Health, Enemy } 的 Entity 属于 Archetype A。
- 拥有 { LocalTransform, Health, Enemy, Burning } 的 Entity 属于 Archetype B——多了 Burning, Component 组合就变了, 于是是另一个 Archetype。

在 Archetype 之间迁移（增删 Component）属于**结构性变更**，会把 Entity 实际搬到新的 Chunk 里。详见 13_Structural Change & Safety.md。

5. Chunk

同一 Archetype 的 Entity 会紧凑地塞进 **16 KB 的 Chunk**。在 Chunk 内部，每种 Component 各自构成一个连续数组——对缓存局部性极为有利。

事实	影响
Chunk 大小为 16 KB	每个 Component 都要占一段空间；Archetype 里的 Component 越多、越大，每个 Chunk 能装下的 Entity 就越少。
数据按 Component 列式 (SoA) 存储	IJobChunk 能一次只遍历一个 Component 数组，并借助 Burst 做向量化。
添加/移除 Component 会移动 Entity	这是结构性变更。Query 迭代器默认迭代期间 Archetype 不会变。
Chunk 跟踪 Component 变更版本	System 可以跳过自上次运行以来没改动过的 Chunk (EntityQuery.SetChangedVersionFilter)。

实时的 Chunk 布局可以在 **Window → Entities → Archetypes** 里查看。

6. System

System 是每帧运行一次的一段逻辑，通常处理一个 Entity Query。

```
[BurstCompile]
public partial struct HealthRegenSystem : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        float dt = SystemAPI.Time.DeltaTime;

        foreach (var health in SystemAPI.Query<RefRW<Health>>())
        {
            health.ValueRW.Value = math.min(health.ValueRW.Value + dt, health.ValueRW.Max);
        }
    }
}
```

System 会自动注册，并归属于某个 **SystemGroup**。详见 08_System – ISystem vs SystemBase.md 和 09_System Group & Update Order.md。

7. World

World 是一个容器，它持有：

- 一个 **EntityManager**（创建、销毁、查询 Entity）。
- 一组 **System** 及其 SystemGroup。
- Chunk 本身（通过 EntityManager）。

默认的运行时 World 是 DefaultGameObjectInjectionWorld。大多数游戏正式上线时只有一个 World；多 World 的场景主要出现在 Netcode（服务器 / 客户端）和 Editor 工具里。Netcode 的 World 拆分详见 16_Netcode Client-Server World & Bootstrap.md。

```
World world = World.DefaultGameObjectInjectionWorld;
EntityManager em = world.EntityManager;

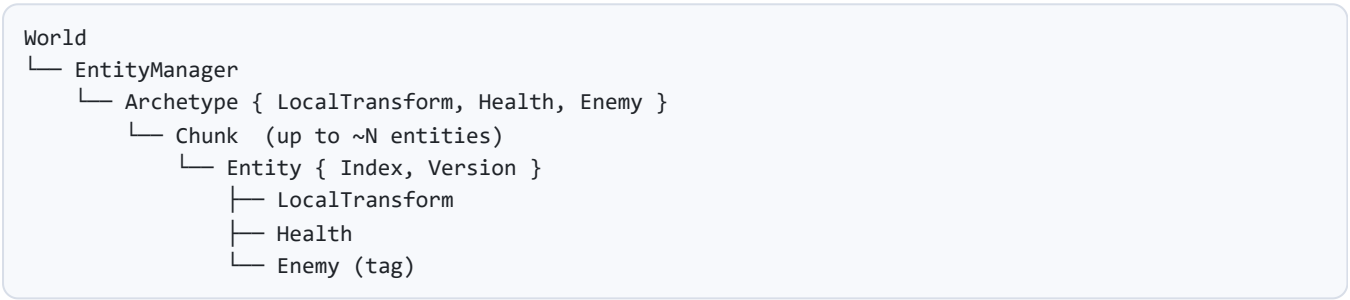
Entity e = em.CreateEntity(typeof(Health));
em.SetComponentData(e, new Health { Value = 100, Max = 100 });
```

8. EntityManager

EntityManager 是主线程上做结构性变更的核心 API：创建、销毁、增删 Component、实例化预制体 Entity、复制 World 等等。经由 EntityManager 的结构性变更是立即生效的，可能让任何在途的 Query 迭代器或 Job 引用失效。

若要在 Job 中或迭代过程中做延迟的结构性变更，就用 **EntityCommandBuffer**——详见 14_EntityCommandBuffer · Deferred Entity.md。

9. 整体结构



一个查询 { LocalTransform, Health } 的 System，会访问所有**同时含这两者**的 Archetype 下的 Chunk。这既包括上面那个 Archetype，也包括其他任何带 Enemy 及额外 Component 的 Archetype。

10. 故障排查

症状	原因 / 解决方法
Entity 查找拿到的是过期数据	这个 Entity 已经被销毁又重建，Version 对不上了。重新解析一次这个 Entity。
Query 匹配不到，可 Entity 明明带着那个 Component	这个 Component 是在另一个 World 里加的，或者它是可启用 Component，眼下处于禁用状态。
Archetype 数量激增	独特的 Component 组合太多了（常因高基数的逐 Entity Tag 而起）。改用共享 Component 或可启用标志来归并。
一个 Chunk 里只装着 1-2 个 Entity	Archetype 太大了——去掉 Entity 用不上的 Component，或者拆成多个 Entity/Archetype。
迭代时报 InvalidOperationException: ... invalidated ...	Query 迭代过程中发生了结构性变更。改用 ECB，或者把变更攒到循环结束后批量处理。

Chapter 7. Entity 引用 — Entity · EntityPrefabReference · UnityObjectRef

1. 概述

DOTS 代码会用到少量引用类型。它们都"指向某个东西", 因此很容易混淆, 但它们分别处在从 Authoring 到运行时流程的不同阶段。

类型	命名空间	引用对象	典型用途
Entity	Unity.Entities	某个 ECS World 内的一个 Entity。	运行时的关联关系, 以及 Component 字段。
EntityPrefabReference	Unity.Entities.Serialization	一份已 Bake、可稍后加载的 Entity 预制体资产。	流式加载和内容工作流。
UnityObjectRef<T>	Unity.Entities	以非托管包装器形式持有的 UnityEngine.Object 资产或对象。	在 ECS 数据中引用网格、材质、纹理、音频剪辑等已编排资产。

本节只讲 DOTS / Entities 的引用类型。引擎层面的 UnityEngine.Object 标识符 API 不在本书范围内。

2. Entity

Entity 是运行时的 ECS 句柄。它是个非托管值, 用来标识某个 World 内的一行 Component 数据。

```
using Unity.Entities;

public struct Target : IComponentData
{
    public Entity Value;
}
```

重要规则:

- Entity 值只在创建它的那个 World 里才有意义。
- 销毁 Entity 会让旧句柄失效。
- Entity 索引会被复用; 要解引用一个存下来的句柄时, 若它不是刚查出来的, 先用 EntityManager.Exists(entity) 检查一下。

- 别把原始 Entity 值存到磁盘，也别拿它当稳定 ID 在独立的 World 之间传递。

```
if (state.EntityManager.Exists(target.Value))
{
    var transform = state.EntityManager.GetComponentData<LocalTransform>(target.Value);
}
```

3. Baking 中的 Entity 引用

Baker 把 Authoring 对象转换成 ECS 数据时，用 `GetEntity` 把被引用的 Authoring 对象或 Prefab 转成一个 Entity 引用。

```
using Unity.Entities;
using UnityEngine;

public class SpawnerAuthoring : MonoBehaviour
{
    public GameObject Prefab;
}

public struct Spawner : IComponentData
{
    public Entity Prefab;
}

public class SpawnerBaker : Baker<SpawnerAuthoring>
{
    public override void Bake(SpawnerAuthoring authoring)
    {
        var entity = GetEntity(TransformUsageFlags.None);
        AddComponent(entity, new Spawner
        {
            Prefab = GetEntity(authoring.Prefab, TransformUsageFlags.Dynamic)
        });
    }
}
```

得到的是一个运行时 ECS 引用，而不是托管的 `GameObject` 引用。

4. EntityPrefabReference

有些 Prefab 内容应当走 Entities 内容管线来加载，而不是作为 Entity 引用直接嵌进当前场景——这种情况就用 `EntityPrefabReference`。

```
using Unity.Entities;
using Unity.Entities.Serialization;
using UnityEngine;
```

```

public class ProjectileLibraryAuthoring : MonoBehaviour
{
    public GameObject ProjectilePrefab;
}

public struct ProjectileLibrary : IComponentData
{
    public EntityPrefabReference ProjectilePrefab;
}

public class ProjectileLibraryBaker : Baker<ProjectileLibraryAuthoring>
{
    public override void Bake(ProjectileLibraryAuthoring authoring)
    {
        var entity = GetEntity(TransformUsageFlags.None);
        AddComponent(entity, new ProjectileLibrary
        {
            ProjectilePrefab = new EntityPrefabReference(authoring.ProjectilePrefab)
        });
    }
}

```

Prefab 属于内容管理或流式加载 workflow 时，用 EntityPrefabReference。Prefab 已经 Bake 进同一个 SubScene、随场景一起可用时，用普通的 Entity 预制体引用即可。

5. UnityObjectRef<T>

UnityObjectRef<T> 让非托管的 ECS 数据也能引用 UnityEngine.Object，而不必把 Component 变成托管 Component。

```

using Unity.Entities;
using UnityEngine;

public struct ProjectileVisual : IComponentData
{
    public UnityObjectRef<Mesh> Mesh;
    public UnityObjectRef<Material> Material;
}

```

从 Authoring 数据 Bake 出来：

```

using Unity.Entities;
using UnityEngine;

public class ProjectileVisualAuthoring : MonoBehaviour
{
    public Mesh Mesh;
    public Material Material;
}

public class ProjectileVisualBaker : Baker<ProjectileVisualAuthoring>

```

```

{
    public override void Bake(ProjectileVisualAuthoring authoring)
    {
        var entity = GetEntity(TransformUsageFlags.Dynamic);
        AddComponent(entity, new ProjectileVisual
        {
            Mesh = new UnityObjectRef<Mesh>(authoring.Mesh),
            Material = new UnityObjectRef<Material>(authoring.Material)
        });
    }
}

```

只在主线程或托管的表现层代码里解引用 `.Value`：

```

Mesh mesh = visual.Mesh.Value;
Material material = visual.Material.Value;

```

规则：

- 存下来的包装器是非托管的，可以放进 `IComponentData`。
- 目标对象本身仍是托管的 Unity 对象；`.Value` 不能在 Burst Job 里用。
- 解引用的那一刻，被引用对象必须仍然处于加载状态。

6. 选对引用类型

需求	使用
在同一 World 里，从一个 Entity 指向另一个	Entity
引用一个已 Bake 进同一 SubScene 的 Prefab	通过 <code>GetEntity(authoring.Prefab, ...)</code> 得到的 Entity 预制体引用
引用一个用于内容流式加载 / 延迟加载的 Prefab	EntityPrefabReference
在非托管 ECS 数据里引用网格、材质、纹理、音频剪辑或 ScriptableObject	UnityObjectRef<T>
让 Entity 跨会话持久保存	用游戏自己定义的稳定 ID，而不是原始 Entity

7. 故障排查

症状	原因 / 解决方法
存下来的 Entity 已经不存在了	这个 Entity 已被销毁，或者属于另一个 World。重新解析它，或检查 <code>EntityManager.Exists</code> 。

症状	原因 / 解决方法
Baking 之后预制体引用是 Entity.Null	Baker 没对这个 Prefab 调用 GetEntity，或者这个 Prefab 没被纳入已 Bake 的场景/内容 workflow。
UnityEngine.ObjectRef<T>.Value 返回 null	被引用的 Unity 对象没加载，或已被销毁。给解引用加上判空保护，并重新加载/解析资产。
Burst Job 无法使用网格/材质引用	UnityEngine.ObjectRef<T> 本身的存储是非托管的，但 .Value 会触及托管的 Unity 对象。把解引用挪到主线程的表现层代码里。
存档文件里存了原始 Entity 值	原始 Entity 句柄只在本 World 内有效，并不稳定。改存游戏自有的 ID 或内容键。

Chapter 8. Component 类型

1. 概述

ECS 里的"Component"是挂在 Entity 上的任意数据容器。Entities 提供了好几种形态，存储方式、生命周期和开销各不相同。选对类型，ECS 系统的设计就完成了一半。

快速参考表：

接口 / 形式	存储位置	修改时触发结构性变更？	典型用途
IComponentData (非托管 struct)	Chunk 内，SoA	否（仅改值时）	默认 Component
IComponentData (托管 class)	附属表	否	引用类型（网格、GameObject 引用）
IBufferElementData	Chunk 内 + 堆溢出	否	逐 Entity 的动态数组
ISharedComponentData	每 Chunk	是（将 Entity 移至新 Chunk）	将共享配置的 Entity 分组
ICleanupComponentData	Chunk 内	正常	Entity 销毁清理协议
ICleanupSharedComponentData	每 Chunk	正常	共享资源清理协议
Tag (空 IComponentData)	Chunk 内零尺寸	否	过滤标记，无数据
Chunk Component	每 Chunk	是	每个 Chunk 计算一次的元数据
可启用 (IEnableableComponent)	Chunk 内 + 位掩码	否	逐 Entity 开关某项行为

本节余下篇幅会把每一行逐一展开。

2. 非托管 IComponentData

默认类型。以紧凑的 SoA 形式直接存在 Chunk 里，对 Burst 友好。

```
public struct Velocity : IComponentData
{
    public float3 Value;
}
```

规则：

- 必须是 struct，且不能有引用字段（不能用 class、string、List<T>）。
- 可用的字段类型：基本类型、Unity.Mathematics、其他非托管 struct、Entity、FixedString*、BlobAssetReference<T>、UnityObjectRef<T>。
- 在 SystemAPI.Query 里用 RefRO<T> 读、RefRW<T> 写。

3. 托管 IComponentData

存在一张按 Entity 索引的附属表里。能用引用类型，代价是会产生 GC 分配，且这个 Component 不兼容 Burst。

```
public class EnemyAI : IComponentData
{
    public AIModel Brain;        // a managed class
    public List<Entity> Allies;
}
```

确实需要托管引用时才用它（比如网格、纹理、编排好的 ScriptableObject）。如果只是想对 UnityEngine.Object 做个弱引用，优先在非托管 Component 里用 UnityObjectRef<T>。

4. IBufferElementData——动态 Buffer

DynamicBuffer<T> 是逐 Entity 的可变长数组。

```
public struct Waypoint : IBufferElementData
{
    public float3 Position;
}

var buffer = state.EntityManager.AddBuffer<Waypoint>(entity);
buffer.Add(new Waypoint { Position = new float3(1, 0, 0) });
```

存储方式：前 InternalBufferCapacity 个元素存在 Chunk 内，超出部分溢出到堆上。容量用 [InternalBufferCapacity(N)] 调整。

5. ISharedComponentData——分组

共享 Component 携带一个**用来给 Chunk 分组的值**。两个 Entity 若 ISharedComponentData 值相同，就会落进同一个 Chunk；值不同则进入不同的 Chunk。

```
public struct RenderMeshGroup : ISharedComponentData
{
    public int MaterialID;
}
```

影响：

- 改动这个值会把 Entity 搬到另一个 Chunk → 属于**结构性变更**，开销不小。
- 适合那种不常变、但按组各异的数据（材质集、队伍 ID、Bake 区域）。
- 如果类型里含引用，值会存进一张托管表（ISharedComponentData 可以是托管的）。

6. 清理 Component——ICleanupComponentData

给 Entity 打上标记，让它“被销毁”时只留下清理 Component 继续存活，直到你的 System 处理完它。

```
public struct ConnectionCleanup : ICleanupComponentData
{
    public int ConnectionHandle;
}

// DestroyEntity(e) removes all normal components but leaves ConnectionCleanup
// behind so you can close the socket in a dedicated system, then destroy the
// cleanup component explicitly.
```

与之配套的 ICleanupSharedComponentData 把同一套协议用在共享资源上。

使用场景：释放原生资源、从外部系统注销、跨帧分阶段销毁。

7. Tag Component

Tag 是一个空的 IComponentData：

```
public struct Enemy : IComponentData {}
```

它在 Chunk 里占零字节，又能像普通 Component 一样查询，很适合做“属于 X 类型”这种标记。但 Tag 用得太多（成百上千个）会让 Archetype 碎片化——高基数的状态应改用可启用 Component 或共享 Component。

8. Chunk Component

Chunk Component 是**每个 Chunk 一个值**，而不是每个 Entity 一个值。

```
state.EntityManager.AddChunkComponentData<RegionBounds>(query, new RegionBounds { ... });
```

典型用途：那些对 Chunk 内所有 Entity 都适用的预计算数据（边界、区域 ID、分区键）。修改 Chunk Component 会引发结构性变更。

9. 可启用 Component (IEnableableComponent)

深入内容见专题 06_Enableable Component.md。简单说：它是一种能逐 Entity 启用/禁用、又不会触发结构性变更的 Component，靠的是每个 Chunk 一份的位掩码。

```
public struct Stunned : IComponentData, IEnableableComponent
{
    public float Duration;
}

state.EntityManager.SetComponentEnabled<Stunned>(entity, false); // no structural change
```

可启用 Component 处于禁用状态的 Entity，Query 会自动跳过——这是把 Entity 临时排除在 System 之外的常用手段，开销也很低。

10. 决策指南

我想.....	使用
逐 Entity 存放一些数值数据	非托管 IComponentData
逐 Entity 持有一个托管引用	托管 IComponentData 或 UnityObjectRef<T>
逐 Entity 存一个变长列表	IBufferElementData
把 Entity 按组归类以便批处理	ISharedComponentData
每帧都要开关某个 Component	可启用 Component
只是给类型打个标记	Tag (空 IComponentData)
为整个 Chunk 预先算好数据	Chunk Component
在 Entity 销毁时执行一段代码	清理 Component

11. 故障排查

症状	原因 / 解决方法
InvalidOperationException: The type T must be unmanaged	在要求非托管类型的地方用了托管类型。换 Component, 或改用对应的 API (如 ComponentLookup<T> 对 ManagedAPI.GetComponent<T>)。
设置共享 Component 值时画面卡顿	每次改动都是一次结构性变更。用 ECB 批量处理, 或者确保这个值很少改动。
DynamicBuffer<T> 溢出到堆上	[InternalBufferCapacity(N)] 设得不够大。调大它, 或把缓冲区大小控制在范围内。
DestroyEntity 之后 Entity 又冒出来了	它身上还挂着清理 Component。清理工作做完后, 记得显式移除它。
Archetype 太多	Tag 膨胀了。把那些逐 Entity 各不相同的"状态"改成可启用 Component。
Tag Component 没起到过滤 Query 的作用	Tag Component 没有数据, 所以不能在 SystemAPI.Query<...> 元组里请求 RefRW<Tag> / RefRO<Tag>。改用 .WithAll<Tag>() / .WithNone<Tag>()。

Chapter 9. 可启用 Component

1. 可启用 Component 存在的原因

增删 Component 属于**结构性变更**：Entity 会被搬到另一个 Chunk，Query 必须重新解析，引用也可能失效。如果每帧都对每个 Entity 这么干（比如反复开关"Stunned"），光这一项开销就能拖垮整帧。

可启用 Component 让你能在**不触发**结构性变更的前提下开关某项行为。Component 始终留在 Chunk 里；每个 Chunk 的掩码中有一个位，决定了在 Query 看来某个 Entity 是否"拥有"它。

2. 接口

同时实现 IComponentData 和 IEnableableComponent：

```
using Unity.Entities;

public struct Stunned : IComponentData, IEnableableComponent
{
    public float Duration;
}
```

IBufferElementData 也可以做成可启用的。

3. 开关切换

在 System 里：

```
// Turn Stunned off without removing the component
state.EntityManager.SetComponentEnabled<Stunned>(entity, false);

// From an ECB (deferred, safe mid-iteration)
ecb.SetComponentEnabled<Stunned>(entity, true);

// From a job with a ComponentLookup
var stunnedLookup = SystemAPI.GetComponentLookup<Stunned>();
stunnedLookup.SetComponentEnabled(entity, false);
```

读取当前状态：

```
bool isStunned = state.EntityManager.IsComponentEnabled<Stunned>(entity);
```

这三种方式操作的都是位掩码，都不会触发结构性变更。

4. Query 行为

SystemAPI.Query<...> 以及基于 EntityQuery 的迭代，都会**跳过**可启用 Component 当前处于禁用状态的 Entity——默认情况下，你只能看到那些列出的 Component 全部启用的 Entity。

```
foreach (var stunned in SystemAPI.Query<RefRW<Stunned>>())
{
    // only fires for entities where Stunned is enabled
}
```

若想连已禁用的 Entity 也一起看到，可以覆盖这个默认行为：

```
foreach (var (stunnedRW, entity) in SystemAPI
    .Query<RefRW<Stunned>>()
    .WithEntityAccess()
    .WithOptions(EntityQueryOptions.IgnoreComponentEnabledState))
{
    // fires for ALL entities with the component, enabled or not
}
```

5. 示例——会过期的眩晕效果

同一个 IJobEntity 不能对同一个 Component 既收 ref Stunned（也就是 RefRW<T>）又收 EnabledRefRW<Stunned>——Entities 明确禁止在一个 Job 里用两种形式包装同一个 Component。有两种可行的写法：

5.1 读写值，用 ECB 切换启用位

```
[BurstCompile]
public partial struct StunTickJob : IJobEntity
{
    public float DeltaTime;
    public EntityCommandBuffer.ParallelWriter ECB;

    void Execute([ChunkIndexInQuery] int chunkIndex,
        Entity entity,
        ref Stunned stunned)
    {
        stunned.Duration -= DeltaTime;
        if (stunned.Duration <= 0f)
            ECB.SetComponentEnabled<Stunned>(chunkIndex, entity, false);
    }
}
```


回放在下一个 ECB 边界进行（比如 EndSimulationECBS）——详见 14_EntityCommandBuffer · Deferred Entity.md。这不涉及结构性变更，只是在一个确定的时间点翻转位掩码。

5.2 用不上 Component 值时直接切换启用位

```
[BurstCompile]
public partial struct StunDisableJob : IJobEntity
{
    void Execute(EnabledRefRW<Stunned> stunnedEnabled)
    {
        stunnedEnabled.ValueRW = false;    // no structural change
    }
}
```

这个 Job 里对 Stunned 的唯一引用就是 EnabledRefRW<T>，所以能编译通过。当一个 System 纯粹就是个开关时，用这种写法。

想在一趟遍历里既读值又切换同一 Entity 的启用位？那就在主线程上调 EntityManager.SetComponentEnabled<T>(entity, false)，或者把逻辑拆成两个 Job、用 state.Dependency 串起来。

6. 何时该用可启用 Component，而非增删

场景	选择
高频切换（每帧、每事件）	可启用 Component
添加一种并非始终存在的状态（带可变字段的逐 Entity 修饰符）	可启用 Component
低频的 Archetype 变更（比如赋予一项永久能力）	照常增删 Component
让多个 System 用同一套规则过滤	可启用 Component（Query 会自动识别）

一条通则：如果某个 Component 的“有没有”在 Entity 一生中会多次反复，就把它做成可启用的。

7. 与其他特性的交互

特性	行为
WithChangeFilter<T>	切换启用位也会顺带更新 Component 的变更版本。
ICleanupComponentData	清理 Component 不能同时是可启用的。

特性	行为
Netcode for Entities	可启用 Component 能参与 Ghost 同步，但切换必须由权威端来做。
序列化 / World 导出	启用位会被保留。

8. 故障排查

症状	原因 / 解决方法
Query 把本该"关闭"的 Entity 也算进来了	Component 没实现 IEnableableComponent，或者设了 WithOptions(EntityQueryOptions.IgnoreComponentEnabledState)。
SetComponentEnabled<T> 抛出"类型 T 不支持启用"	这个 struct 只实现了 IComponentData，补上 IEnableableComponent 即可。
源码生成器拒绝 EnabledRefRW<T> 参数	Job 或 System 漏了 partial，或者这个 Component 不是可启用的。
源码生成器报错"同一 Component 不能同时有 RefRW 和 EnabledRefRW"	把逻辑拆开——每个 Job 里对同一个 Component，要么用 ref T，要么用 EnabledRefRW<T>，不能两者并用。参见 §5 的两种写法。
变更过滤器莫名其妙触发	对 WithChangeFilter 而言，切换 Enabled 也算一次变更，这是预期行为——改用 EnabledRefRO 读取，别反复切换。
ECB 的 SetComponentEnabled 不起作用	在创建该延迟 Entity 的那个 ECB 刷新之前，就对它调用了这个方法。这两步操作必须落在同一个 ECB 周期内。

Chapter 10. Singleton Component

1. Singleton 是什么（以及不是什么）

Entities 里的 **Singleton**，其实就是一个预期在整个 World 中**恰好只有一个 Entity** 持有的 Component。它没有专门的 ISingleton 接口——任何 IComponentData 按约定都能当 Singleton 用，SystemAPI 则提供了一组顺手的访问器，并会校验"恰好一个"这条规则。

典型用途：游戏状态、全局配置、时间/Tick 源、网络状态、中心化的 ECB 注册。

```
public struct GameState : IComponentData
{
    public int    Score;
    public float  TimeScale;
    public bool   Paused;
}
```

2. 访问器

在任意 ISystem / SystemBase 里：

```
// Reads
GameState gs      = SystemAPI.GetSingleton<GameState>();
RefRW<GameState> gsRW = SystemAPI.GetSingletonRW<GameState>();
bool has         = SystemAPI.HasSingleton<GameState>();
bool got         = SystemAPI.TryGetSingleton<GameState>(out var gs2);

// Writes
SystemAPI.SetSingleton(new GameState { Score = 10 });

// Entity handle, if you need it
Entity e         = SystemAPI.GetSingletonEntity<GameState>();

// Buffers work too
DynamicBuffer<Waypoint> buf = SystemAPI.GetSingletonBuffer<Waypoint>();
```

当 Singleton 数量为零或多于一个时，Get* 系列会抛异常，TryGet* 则返回 false。RW 系列会把 Component 标记为已变更，供变更跟踪使用。

3. 创建 Singleton

两种常见模式：

3.1 从 SubScene Bake

```
public class GameStateAuthoring : MonoBehaviour
{
    public float InitialTimeScale = 1f;

    class Baker : Baker<GameStateAuthoring>
    {
        public override void Bake(GameStateAuthoring authoring)
        {
            var e = GetEntity(TransformUsageFlags.None);
            AddComponent(e, new GameState
            {
                Score      = 0,
                TimeScale = authoring.InitialTimeScale,
                Paused     = false
            });
        }
    }
}
```

把 GameStateAuthoring 挂到 SubScene 里某一个 GameObject 上，Baker 就会生成恰好一个带 Singleton 的 Entity。

3.2 在 System 启动时创建

```
public void OnCreate(ref SystemState state)
{
    var e = state.EntityManager.CreateEntity(typeof(GameState));
    state.EntityManager.SetComponentData(e, new GameState { TimeScale = 1f });
}
```

适合那些不该放进 Authoring 的数据，比如服务器下发的配置、程序化生成的数据等。

4. 用 Singleton 来设 System 的运行门槛

如果一个 System 只有在 Singleton 存在之后才有意义：

```
public void OnCreate(ref SystemState state)
{
    state.RequireForUpdate<GameState>();
}
```

RequireForUpdate<T>() 会让 System 在该 Component 不存在时**整个跳过 OnUpdate**，省得在 OnUpdate 里写一堆防御性的空值判断。

5. 只读与读写性能

- GetSingleton<T>()——读出一份副本，不做变更跟踪。
- GetSingletonRW<T>()——返回一个指向 Chunk 内部的 RefRW<T>。给 .ValueRW 赋值会把 Component 标记为已变更，正合其他 System 用 WithChangeFilter<T> 的需要。
- SetSingleton——整体写入，每次都更新版本号。

以读为主的 System 优先用 GetSingleton；只有真的要改值时，才换成 GetSingletonRW。

6. 多个候选——"Singleton 超过一个"的陷阱

一旦有两个 Entity 同时带上了同一个 Singleton Component，GetSingleton<T> 就会抛出 InvalidOperationException: expected exactly one entity with component T, found N。常见原因有：

- SubScene 里有两个 GameObject 都挂了 Authoring 脚本。
- 测试夹具在运行时建了一个 Singleton，**同时** SubScene 又把它 Bake 了一份。
- System 在 OnCreate 里创建了 Singleton，却没先用 HasSingleton 判断一下。

稳妥的写法：

```
public void OnCreate(ref SystemState state)
{
    if (!SystemAPI.HasSingleton<GameState>())
    {
        var e = state.EntityManager.CreateEntity(typeof(GameState));
        state.EntityManager.SetComponentData(e, new GameState { TimeScale = 1f });
    }
}
```

7. Buffer Singleton

DynamicBuffer<T> 也可以是 Singleton：

```
public struct QueuedEvent : IBufferElementData
{
    public int Type;
    public int Payload;
}

// Access:
DynamicBuffer<QueuedEvent> events = SystemAPI.GetSingletonBuffer<QueuedEvent>();
events.Add(new QueuedEvent { Type = 1 });
```

适合做全局事件队列和命令流。

8. 故障排查

症状	原因 / 解决方法
InvalidOperationException: expected exactly one entity with component T, found 0	Singleton Entity 压根没创建。加个 Baker，或者在 OnCreate 里先用 HasSingleton 判断再创建。
... found 2	有两个 Entity 都带着这个 Component。在 Authoring 阶段去重，或在启动时清理掉多余的。
GetSingletonRW 好像没把 Component 标记成已变更	写的是 .ValueRO，或者干脆没用那个 RefRW。只有 .ValueRW 才会触发变更跟踪。
System 在 Singleton 创建之前就跑了	用 RequireForUpdate<T>，或用 [UpdateBefore] 调整顺序，让创建它的 System 先跑。
两个没有先后关系的 System 都在改同一个 Singleton	加上 [UpdateAfter]/[UpdateBefore]，或者把逻辑合并。Singleton 上的竞争条件尤其难查。
Netcode: 客户端和服务器的 Singleton 值对不上	对 Singleton 做 Ghost 同步，或者从已同步的输入出发，确定性地重建它。

Chapter 11. System — ISystem 与 SystemBase

1. 概述

Entities 有两种声明 System 的方式：**ISystem**（非托管 struct）和 **SystemBase**（托管 class）。除非确实要用到 SystemBase 才有的能力，否则一律选 ISystem。

	ISystem	SystemBase
声明方式	partial struct : ISystem	partial class : SystemBase
内存	非托管，可在栈上分配	托管，由 GC 管理
可 Burst 编译	是（添加 [BurstCompile]）	否
托管字段	否	是
托管 API 访问	通过 EntityManager + ManagedAPI	直接访问
默认选择	☒	不得已时才用

2. ISystem——默认选择

```
using Unity.Burst;
using Unity.Entities;

[BurstCompile]
public partial struct EnemyAISystem : ISystem
{
    [BurstCompile]
    public void OnCreate(ref SystemState state)
    {
        // Called once when the system is created.
        state.RequireForUpdate<EnemySpawnConfig>();
    }

    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        var config = SystemAPI.GetSingleton<EnemySpawnConfig>();
        // ... logic
    }

    [BurstCompile]
    public void OnDestroy(ref SystemState state)
    {
        // Called once when the system is destroyed.
    }
}
```

```
}  
}
```

必要要素：

- Struct 必须是 partial，源码生成器要靠它添加隐藏的包装层。
- OnCreate、OnUpdate、OnDestroy 三个方法都是可选的，用不到的就不写。
- struct 上和每个想被 Burst 编译的方法上都要加 [BurstCompile]。

3. SystemBase——非用托管 System 不可时

```
using Unity.Entities;  
  
public partial class PlayerInputSystem : SystemBase  
{  
    private InputActions _input;    // managed field – the reason SystemBase exists  
  
    protected override void OnCreate()  
    {  
        _input = new InputActions();  
        _input.Enable();  
    }  
  
    protected override void OnUpdate()  
    {  
        var move = _input.Gameplay.Move.ReadValue<Vector2>();  
        // ... dispatch to components  
    }  
  
    protected override void OnDestroy() => _input.Dispose();  
}
```

遇到下面这些情况，才动用 SystemBase：

- 要持有一个必须和 System 一同存在的托管字段（Input System 对象、UI Toolkit 句柄、托管服务等）。
- 要在 OnUpdate 里直接调用托管 API，而不想经由 EntityManager 中转。

SystemBase 里的一切都跑在主线程上，不走 Burst。

4. System 生命周期

两种形式用的是同一套回调名：

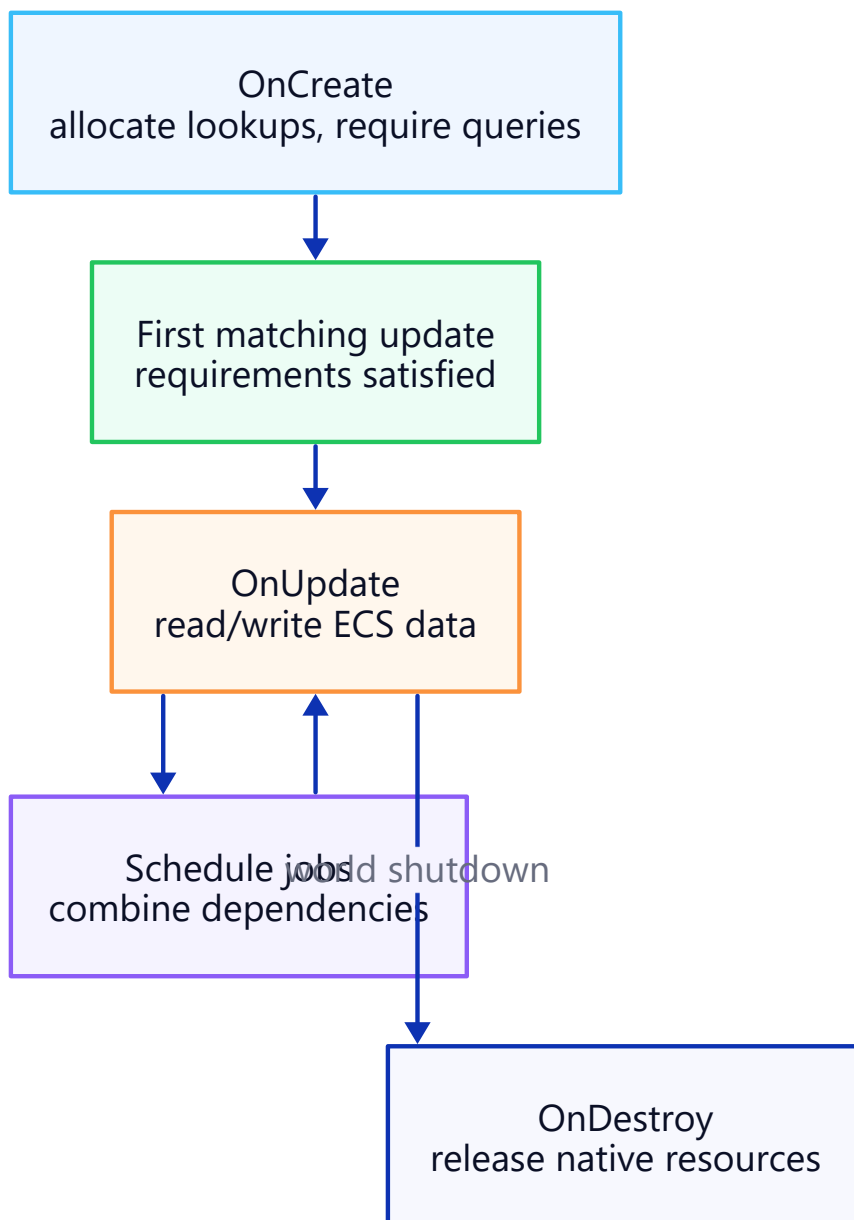


Figure: System Lifecycle.

回调	触发时机	典型用途
OnCreate	System 创建时，只调一次	缓存 Query、调 RequireForUpdate<T>、分配原生容器
OnStartRunning	Query 从空变为非空	重置每个会话的状态
OnUpdate	在父 SystemGroup 中每帧调一次	主要工作
OnStopRunning	Query 从非空变为空	刷新缓冲状态
OnDestroy	World 拆除时，只调一次	释放原生/托管资源

RequireForUpdate<T>()

当所需的 Component 或 Singleton 不存在时，直接跳过 OnUpdate：

```
public void OnCreate(ref SystemState state)
{
    state.RequireForUpdate<EnemySpawnConfig>();
    state.RequireForUpdate<GameRunning>();
}
```

要让 System 只在"配置已就绪"或"游戏已开始"时才运行，这就是标准做法。

5. 在两者之间做选择

经验法则：

1. **先用 ISystem。** 它对 Burst 友好，也没有 GC。
2. **只有当托管字段逼得你没办法时，才换 SystemBase。** 即便如此，也不妨拆开：托管的活儿放进一个轻量的 SystemBase，把结果写进 Component，再让一个读取这些 Component 的 ISystem 去跑繁重逻辑。
3. **别混着来——**往 ISystem struct 里塞托管字段是编译不过的。上面那种拆分就是惯用的变通办法。

拆分示例：

```
// Managed – bridges Input System into components.
public partial class InputBridgeSystem : SystemBase
{
    private InputActions _input;
    // ... fills a PlayerInput singleton each frame
}

// Unmanaged + Burst – reads the PlayerInput singleton and drives gameplay.
[BurstCompile]
public partial struct PlayerMoveSystem : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state) { /* ... */ }
}
```

6. SystemState 快速参考

在 ISystem 里，ref SystemState state 是你通往 World 的窗口。

成员	用途
<code>state.EntityManager</code>	在主线程上做结构性变更。
<code>state.World</code>	所属 World。
<code>state.Dependency</code>	当前的 Job 依赖句柄——和你调度的 Job 合并使用。
<code>state.GetEntityQuery(...)</code>	构建一个 Query，或取回已缓存的 Query。
<code>state.RequireForUpdate<T>()</code>	以某个 Component 是否存在作为 System 的运行门槛。
<code>state.Enabled</code>	设为 false 即可跳过 System，又不销毁它。

在 `SystemBase` 里，这些都是类自身的成员（比如 `this.EntityManager`）。

7. 故障排查

症状	原因 / 解决方法
<code>error CS0246: The type or namespace name 'Entities' could not be found</code>	漏了 <code>using Unity.Entities;</code> 。
<code>OnUpdate</code> 从未触发	<code>RequireForUpdate<T></code> 把门槛设在了一个还不存在的 Component 上——确认这个 Component 在别处确实被创建了。
编译错误: "partial struct 必须声明为 partial"	<code>ISystem</code> 必须是 <code>partial</code> ，源码生成器才能扩展这个 struct。
Burst 警告: "method is not burst compiled"	这个方法漏了 <code>[BurstCompile]</code> 。只在 struct 上加这个特性还不够。
System 跑在了错误的 World 上 (Netcode 下的客户端与服务端)	加上 <code>[WorldSystemFilter(...)]</code> ，挑选 <code>ServerSimulation / ClientSimulation / LocalSimulation</code> 。
不能向 <code>ISystem</code> 添加托管字段	这是正常的——按上文做法，把托管部分拆到一个 <code>SystemBase</code> 里。
System 未出现在 Window → Entities → Systems 中	漏了 <code>partial</code> ，或不在默认 World 内，又或者被 <code>[DisableAutoCreation]</code> 排除了。

Chapter 12. System Group 与更新顺序

1. 概述

每个 System 都隶属于某个 **SystemGroup**。Group 决定子项每帧何时运行，排序特性则决定 Group 内部的先后次序。分组分对了，才能避开慢一帧的写入、竞争条件，以及"这 Entity 怎么在抖"这类 Bug。

2. 内置 Group

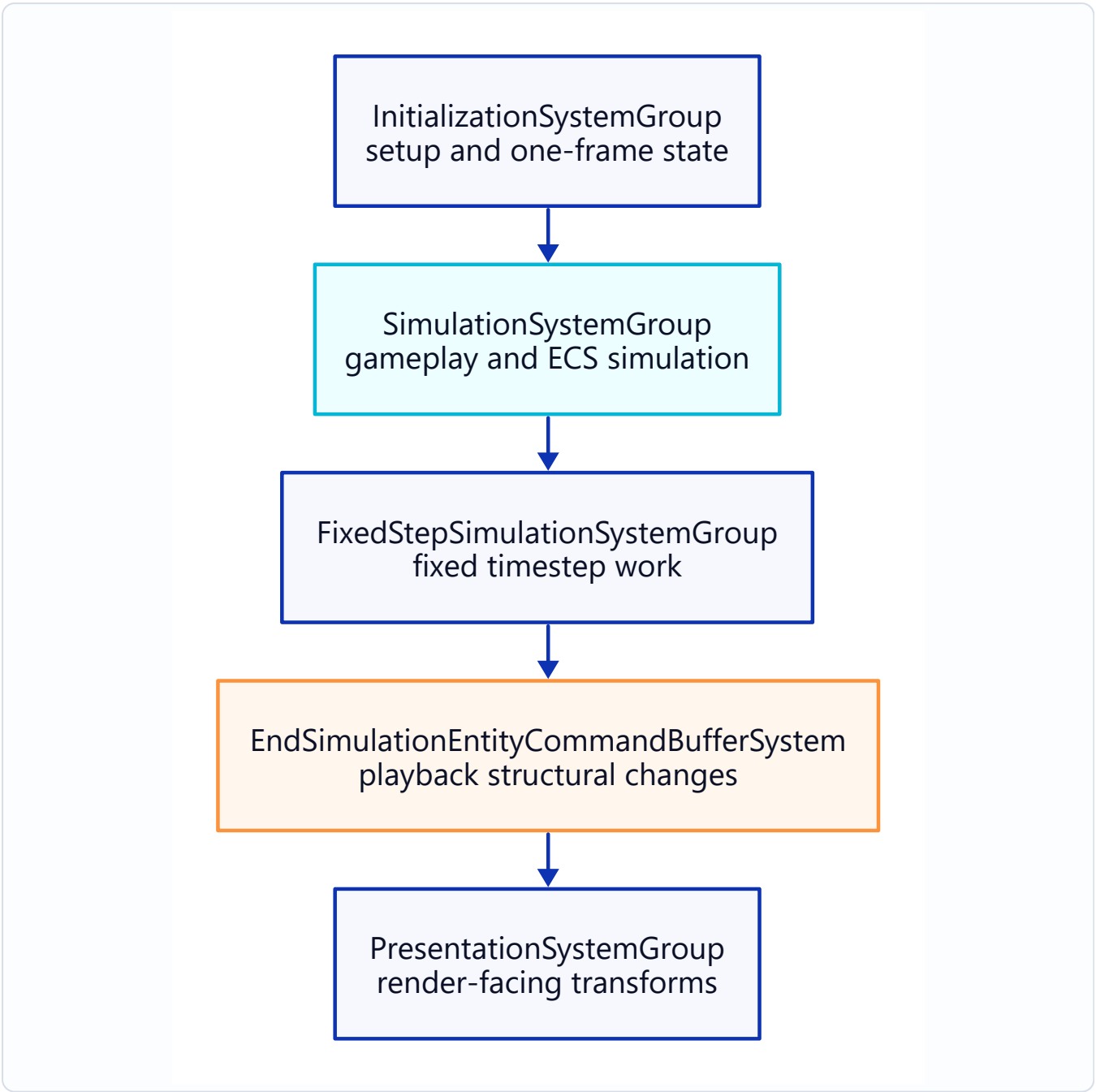


Figure: System Group and Update Order.

Group	运行时机	典型内容
InitializationSystemGroup	每帧一次，在模拟之前	一次性的初始化器、Bootstrap 任务
SimulationSystemGroup	每帧一次	大多数游戏逻辑
FixedStepSimulationSystemGroup	固定时间步长（默认 60 Hz）； 每帧可能运行 0、1 或 N 次	物理、需确定性的步进逻辑
PresentationSystemGroup	每帧一次，在模拟之后	渲染准备、插值

Group	运行时机	典型内容
BeginSimulationEntityCommandBufferSystem	Simulation 一开始	有序 ECB 回放（早期）
EndSimulationEntityCommandBufferSystem	Simulation 最末尾	有序 ECB 回放（晚期）

Netcode for Entities 还加了更多 Group（PredictedSimulationSystemGroup、GhostSendSystem 等），它们都嵌套在上述 Group 内部。详见 16_Netcode Client-Server World & Bootstrap.md 和 19_Netcode Prediction & Rollback.md。

3. 将 System 放入 Group

```
using Unity.Entities;

[UpdateInGroup(typeof(SimulationSystemGroup))]
public partial struct EnemyAISystem : ISystem { /* ... */ }
```

不写 [UpdateInGroup] 的话，System 默认归入 SimulationSystemGroup。

嵌套

Group 可以嵌套进别的 Group：

```
[UpdateInGroup(typeof(SimulationSystemGroup), OrderFirst = true)]
public partial class GameplaySystemGroup : ComponentSystemGroup { }

[UpdateInGroup(typeof(GameplaySystemGroup))]
public partial struct MovementSystem : ISystem { /* ... */ }
```

ComponentSystemGroup 是自定义 Group 的基类。Group 既是 System，又是容器。

4. Group 内部排序

```
[UpdateInGroup(typeof(SimulationSystemGroup))]
[UpdateAfter(typeof(EnemyAISystem))]
[UpdateBefore(typeof(MovementSystem))]
public partial struct EnemyPathfindingSystem : ISystem { /* ... */ }
```

特性	含义
[UpdateBefore(typeof(X))]	在 X 之前运行（两者必须同属一个 Group）。

特性	含义
[UpdateAfter(typeof(X))]	在 X 之后运行。
OrderFirst = true (在 [UpdateInGroup] 上)	在 Group 中尽量靠前运行。
OrderLast = true	在 Group 中尽量靠后运行。

提示：优先用 [UpdateAfter] 串成链，少用 OrderFirst/Last——代码审查时更好读，重构之后也不容易出错。

5. [RequireMatchingQueriesForUpdate]

这是个较新的特性（Entities 1.x+），它要求 System **只有当它的每个 Query 都至少匹配到一个 Entity 时，才执行 OnUpdate**：

```
[RequireMatchingQueriesForUpdate]
public partial struct EnemyAISystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {
        // Only runs when there is at least one entity matching every query the system uses.
    }
}
```

对比一下 state.RequireForUpdate<T>()：后者要求某个**特定的** Component/Singleton 存在。
[RequireMatchingQueriesForUpdate] 则更隐式——它依据的是 System 最终用到的所有 Query。

6. 自定义 Group

什么时候值得自建一个 Group：

- 一个内聚的游戏阶段（比如“先结算伤害，再播动画”）。
- 需要不同的 Tick 频率（比如自定义一个 30 Hz 的固定步）。
- 作为功能开关——整组一起启用或禁用。

```
using Unity.Entities;

[UpdateInGroup(typeof(SimulationSystemGroup))]
public partial class DamagePipelineGroup : ComponentSystemGroup
{
    // You can override OnUpdate to customise how the group ticks.
```

```

}

[UpdateInGroup(typeof(DamagePipelineGroup), OrderFirst = true)]
public partial struct ApplyDamageSystem : ISystem { /* ... */ }

[UpdateInGroup(typeof(DamagePipelineGroup))]
[UpdateAfter(typeof(ApplyDamageSystem))]
public partial struct DeathSystem : ISystem { /* ... */ }

```

自定义 Tick 频率

```

public partial class SlowTickGroup : ComponentSystemGroup
{
    private float _accum;
    protected override void OnUpdate()
    {
        _accum += (float)SystemAPI.Time.DeltaTime;
        if (_accum < 0.1f) return; // 10 Hz
        _accum = 0f;
        base.OnUpdate(); // tick children
    }
}

```

7. 固定步长与可变步长

如果确定性至关重要（物理、网络预测、确定性回放），就把 System 放进 FixedStepSimulationSystemGroup：

```

[UpdateInGroup(typeof(FixedStepSimulationSystemGroup))]
public partial struct PhysicsStepSystem : ISystem { /* ... */ }

```

注意事项：

- 为了追上真实时间，FixedStepSimulationSystemGroup 每帧可能运行**多次**，也可能一次都不运行。
- 用 SystemAPI.Time.DeltaTime——在这个 Group 内部，它就是固定步长的 dt。
- 凡是读输入或产出表现层结果的逻辑，都应留在 SimulationSystemGroup / PresentationSystemGroup 里。

8. 运行时检查顺序

Window → Entities → Systems 显示：

- 每个 Group 及其子项，按实际运行顺序列出。
- 每个 System 的耗时数据。
- 启用/禁用状态（针对那些运行时会切换的 System）。

如果特性里声明的顺序和你实际看到的不一致，通常是存在循环依赖，或者遗漏了 [UpdateInGroup]。

9. 故障排查

症状	原因 / 解决方法
System 跑在了错误的帧阶段	[UpdateInGroup] 漏写或写错了。默认是 SimulationSystemGroup。
[UpdateAfter] 被忽略	目标 System 在另一个 Group 里。两者必须同属一个 Group。
启动时出现循环依赖错误	一串 [UpdateBefore] / [UpdateAfter] 绕成了环。Systems 窗口会把这个环打印出来。
System 每帧莫名其妙运行了好几次	你把它放进了 FixedStepSimulationSystemGroup，这个 Group 会补帧追赶。如果不该这样，挪到 SimulationSystemGroup。
自定义 Group 停不下来	重写的 OnUpdate 里一直没调用 base.OnUpdate()。
System 跑是跑了，却什么也没做	RequireForUpdate<T> 的条件没满足，或者所有 Query 都是空的。查看 Window → Entities → Query 。

Chapter 13. Job System 与 Burst

1. 概述

DOTS 的性能，建立在两项引擎特性之上——它们早于 Entities 就已存在，至今仍是 Entities 的基石：

- **Job System**——借助依赖图，把工作调度到各工作线程上。
- **Burst Compiler**——把 C# Job 编译成紧凑的原生代码（用 SIMD，没有 GC，没有虚分发）。

这两者都不是 Entities 专属的概念。IJobEntity 和 IJobChunk 不过是 IJob 加 Burst 之上的一层薄封装。把底层弄明白了，再看上层那些模式就都清晰了。

2. Job System——核心类型

接口	形式	适用场景
IJob	void Execute()	一次性的单线程任务。
IJobParallelFor	void Execute(int index)	把 0 到 N 的循环并行跑。
IJobParallelForBatch	void Execute(int start, int count)	按批次并行循环 (比逐索引对缓存更友好)。
IJobChunk	void Execute(ArchetypeChunk chunk, int chunkIndex, bool useEnabledMask, in v128 enabledMask)	并行遍历 ECS Chunk。
IJobEntity	void Execute([component params])	遍历 ECS Entity—— 经源码生成后变成一个 Chunk Job。

这些接口全是 **struct**，也全都支持 Burst。

IJobParallelFor 最简示例

```
using Unity.Burst;
using Unity.Collections;
using Unity.Jobs;

[BurstCompile]
struct ScaleJob : IJobParallelFor
{
    public NativeArray<float> Data;
    public float Factor;
```

```
public void Execute(int index) => Data[index] *= Factor;
}

// Scheduling:
var handle = new ScaleJob { Data = arr, Factor = 2f }.Schedule(arr.Length, 64);
handle.Complete();
```

注意事项：

- 字段必须是 blittable 的。
- 多个 Job 要写同一个 NativeArray，就得用依赖链串起来（见下文）。

3. 调度与依赖

```
JobHandle a = jobA.Schedule();
JobHandle b = jobB.Schedule(a);           // b runs after a
JobHandle c = jobC.Schedule(JobHandle.CombineDependencies(a, b));
c.Complete();
```

规则：

- 每次 Schedule() 都会返回一个 JobHandle。新 Job 若要读写共享数据，就把上一个句柄作为依赖传进去。
- 主线程读取结果之前，**一定要对最终的句柄调用 .Complete()**（安全系统会强制这一点）。
- 在 ISystem 里，state.Dependency 是系统级的句柄；用 .Schedule(state.Dependency) 调度，或借助源码生成的辅助方法（如 IJobEntity.ScheduleParallel(...)），Job 都会自动和它合并。

在 ISystem 内部

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
{
    var job = new MyJob { Dt = SystemAPI.Time.DeltaTime };
    state.Dependency = job.ScheduleParallel(state.Dependency);
}
```

不带显式依赖参数的 IJobEntity.ScheduleParallel()，会隐式与 state.Dependency 合并。但要链式调度多个 Job 时，还是显式传入依赖为好。

4. Burst——是什么、为什么

Burst 借助 LLVM，把 C# 的一个子集编译成高度优化的原生代码——这个子集指标了 [BurstCompile] 的方法，不能有 GC 分配，数据要 blittable。

Burst 擅长编译什么

- 标了 [BurstCompile] 的 struct IJob* / ISystem 方法。
- 针对 Unity.Mathematics 类型 (float3、quaternion、int4 等) 的数学运算。
- 遍历 NativeArray<T>、NativeList<T>、DynamicBuffer<T>、ArchetypeChunk 列数组的紧凑循环。

Burst 编译不了什么

- 托管引用 (class、string、List<T>，凡是非托管的东西)。
- 异常 (能 catch，但一般还是别用)。
- 经由接口的动态分发 (显式的泛型实例化没问题)。
- System.IO、Reflection，以及基础类库里除数学和基本类型之外的大部分内容。

启用 Burst

```
using Unity.Burst;

[BurstCompile]
public partial struct MyJob : IJobEntity
{
    // attribute on the struct marks the type for Burst
    // methods need [BurstCompile] too if you want each compiled
    [BurstCompile]
    void Execute(ref Velocity v) { /* ... */ }
}
```

System 则是这样：

```
[BurstCompile]
public partial struct MySystem : ISystem
{
    [BurstCompile] public void OnCreate(ref SystemState state) { }
    [BurstCompile] public void OnUpdate(ref SystemState state) { }
}
```

光在类型上加特性**还不够**——每个想被编译的方法都得各自加上 [BurstCompile]。

Burst Inspector

Window → Analysis → Burst Inspector 会逐方法展示 LLVM IR、汇编代码和向量化情况。当你想搞清"这段代码怎么不快"时，就靠它来排查。

5. 原生容器

Burst 和安全系统希望你用**原生容器**，而不是托管集合：

托管类型	原生等效类型
T[]	NativeArray<T>
List<T>	NativeList<T>
Dictionary<K,V>	NativeHashMap<K,V>、NativeParallelHashMap<K,V>
HashSet<T>	NativeHashSet<T>、NativeParallelHashSet<T>
Queue<T>	NativeQueue<T>
string	FixedString32Bytes / FixedString64Bytes / 等

分配：

```
var arr = new NativeArray<int>(1024, Allocator.TempJob);
// ...
arr.Dispose();
```

生命周期至关重要：Burst 加安全系统会检查容器是否已释放，以及是否在 Job 生命周期结束后还被访问。

6. 分配器

分配器	生命周期	典型用途
Allocator.Temp	当前帧内，只限主线程	OnUpdate 里的临时容器
Allocator.TempJob	横跨单个 Job，不超过 4 帧	传递给 Job 的容器
Allocator.Persistent	需手动 Dispose()	由 System 持有、跨帧存活的容器
state.WorldUpdateAllocator	System Group 更新结束时清空	System 内每帧临时数据的首选

System 内部优先用 `state.WorldUpdateAllocator`（或它的 `Rewindable` 分配器）——省去手动 `Dispose()` 的那份心。

7. 安全系统

在 Editor 里打开"Jobs > Leak Detection"和"Jobs > Safety Checks"后，Unity 会检查：

- 没有依赖边的两个 Job 不会同时写同一个原生容器。
- 主线程的读取不会和正在进行的写入撞车。
- 原生容器不会在 `Dispose()` 之后、或超出分配器生命周期之后仍被访问。

安全错误会以异常形式抛出，并附上一句清楚的提示，比如"Job A 和 Job B 都访问了 X"。**开发阶段别关掉安全检查**——省下来的性能微乎其微，调试代价却高得多。

8. 故障排查

症状	原因 / 解决方法
Burst 警告: "method not burst compiled"	方法漏了 <code>[BurstCompile]</code> ；或者 Burst 碰到了不支持的类型（在 Burst Inspector 里查出报错的那一行）。
InvalidOperationException: ... writes to ... without declaring dependency	少了一条依赖边——把上一个 JobHandle 传进去，或者用 <code>state.Dependency</code> 串起来。
NativeArray 上抛出 ObjectDisposedException	Job 还没跑完就被释放了。改用 <code>arr.Dispose(handle)</code> ，或者先等句柄完成。
调度出去的 Job 一直完不成	依赖链里漏了 <code>.Complete()</code> ，或者存在循环依赖。
加了 <code>[BurstCompile]</code> 反而变慢	方法在每次域重载时都重新编译；检查 Burst 缓存。又或者混进了 Burst 不兼容的类型——方法悄悄退回到了 IL 执行。
枚举 NativeHashMap 时抛异常	边写边枚举本来就不稳——改成枚举一份 <code>NativeArray<K></code> 副本。
主线程卡在等 Job 上	<code>.Complete()</code> 调得太早了。尽量把完成点推到帧内尽可能靠后的位置。

Chapter 14. IJobEntity · SystemAPI.Query

1. 概述

在 Entities 6.5 里，两套 API 就覆盖了 95% 的 Component 遍历场景：

API	运行位置	适用场景
SystemAPI.Query<...>()	主线程，在 OnUpdate 里	工作量很小（几百个 Entity），或者需要用主线程 API。
IJobEntity	工作线程，通过 Schedule / ScheduleParallel	工作量较重，或者能从并行中受益。

它们是老式 Entities.ForEach 的推荐替代方案。Entities.ForEach 在 Entities 1.4 中已标为过时；在 6.5 上还能编译，但会报废弃警告；Unity 更新日志称将在未来某个大版本中移除它。

2. SystemAPI.Query<T>

在主线程上遍历匹配到的 Entity。Component 通过 RefRO<T> / RefRW<T> 访问。

```
[BurstCompile]
public partial struct HealthRegenSystem : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        float dt = SystemAPI.Time.DeltaTime;

        foreach (var (health, regen) in
            SystemAPI.Query<RefRW<Health>, RefRO<Regen>>())
        {
            health.ValueRW.Value = math.min(
                health.ValueRO.Value + regen.ValueRO.Rate * dt,
                health.ValueRW.Max);
        }
    }
}
```

规则：

- System struct 必须是 partial。
- 只读用 RefRO<T>，读写用 RefRW<T>，别拿 RefRO 来写。

- 需要 Entity ID 的话，把 Entity 也放进元组：SystemAPI.Query<RefRW<Health>>().WithEntityAccess()。

过滤器

```
foreach (var health in SystemAPI.Query<RefRW<Health>>()
    .WithAll<Enemy>()
    .WithNone<Dead>()
    .WithAny<Burning, Poisoned>())
{
    // only entities that are Enemy AND not Dead AND (Burning OR Poisoned)
}
```

- WithAll<T>()——必须带齐列出的所有 Component。
- WithNone<T>()——一个都不能带。
- WithAny<T...>()——至少要带其中一个。
- WithChangeFilter<T>()——只处理 Component T 自上次运行以来有改动的 Chunk。
- WithEntityAccess()——连 Entity ID 一起返回。

3. IJobEntity

一种源码生成的并行遍历。Execute 的签名声明了你要访问哪些 Component。

```
using Unity.Burst;
using Unity.Entities;
using Unity.Mathematics;

[BurstCompile]
public partial struct HealthRegenJob : IJobEntity
{
    public float DeltaTime;

    void Execute(ref Health health, in Regen regen)
    {
        health.Value = math.min(health.Value + regen.Rate * DeltaTime, health.Max);
    }
}

[BurstCompile]
public partial struct HealthRegenSystem : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        new HealthRegenJob { DeltaTime = SystemAPI.Time.DeltaTime }
            .ScheduleParallel();
    }
}
```


签名规则：

- `void Execute(...)`——参数和 `Component` ——对应。
- `ref Component` 是读写, `in Component` 是只读（不许写）。
- 可以把 `Entity entity` 作为参数——源码生成器会自动填好。
- `Job struct` 必须是 `partial`。

调度变体

调用	线程模式
<code>.Run()</code>	主线程执行，返回前就跑完。适合调试或小工作量。
<code>.Schedule()</code>	单个工作线程。
<code>.ScheduleParallel()</code>	把 <code>Chunk</code> 分摊到多个工作线程。

这三种在 `ISystem.OnUpdate` 内部都会自动更新 `state.Dependency`。

Job 中的过滤器

```
new HealthRegenJob { DeltaTime = dt }  
    .ScheduleParallel(SystemAPI.QueryBuilder()  
        .WithAll<Enemy, Regen>()  
        .WithNone<Dead>()  
        .Build());
```

也可以直接在 `Job struct` 上用 `[WithAll]`/`[WithNone]`/`[WithAny]` 特性来指定。

Job 中的 Entity 访问

```
void Execute(Entity entity, ref Health health) { /* ... */ }
```

Chunk 索引 / `[ChunkIndexInQuery]`

配合 `ECB` 的 `ParallelWriter` 做确定性回放时很有用——把 `Chunk` 索引作为 `sortKey` 传进去。详见 `15_ParallelWriter · Deterministic Playback.md`。

```
void Execute([ChunkIndexInQuery] int chunkIndex, in Health health, Entity entity)  
{  
    if (health.Value <= 0)  
        ecb.DestroyEntity(chunkIndex, entity);  
}
```

4. 如何选择

场景	选择
几十到几百个 Entity，运算很轻	SystemAPI.Query
上千个乃至更多 Entity，且各 Entity 的工作互不依赖	IJobEntity.ScheduleParallel
以 Chunk 为单位的工作（边界、求和、空间分区）	IJobChunk——参见 12_IJobEntity vs IJobChunk.md
循环里要用到托管 API	在 SystemBase 里用 SystemAPI.Query
需要做延迟的结构性变更	两种形式都行，但要通过 EntityCommandBuffer.ParallelWriter 写入——详见 14_EntityCommandBuffer · Deferred Entity.md

5. 注意事项

- 别在 Job 里捕获 this 或托管字段。你一这么写，源码生成器就会报错。
- RefRW<T>.ValueRW 会触发变更跟踪。哪怕只用 ValueRW 读、从不写，也照样会把 Chunk 标记为脏。
- WithChangeFilter<T> 是 Chunk 级别的。只要 Chunk 里有任意一个 Entity 写了 T，整个 Chunk 就算变更过。
- Aspect 已过时（但还没移除）。IAspect 这个分组接口在 1.4 中被废弃，在 6.5 的 API 参考里仍被列为合法的 Execute 参数类型。新代码应改用普通的 Component 参数；现有的 Aspect 在 6.5 上还能编译。

6. 故障排查

症状	原因 / 解决方法
编译错误: IJobEntity must be partial	给 Job struct 加上 partial。
ScheduleParallel 抛出竞争条件错误	两个 Job 没有排序边却都在写同一个 Component。确保 state.Dependency 合并正确，或在 System 之间加上 [UpdateAfter]。
Job 没走 Burst	Job struct 漏了 [BurstCompile]，或者捕获了 Burst 不兼容的类型（托管引用，或没换成 FixedString 的 string）。
SystemAPI.Query 返回空	过滤器组合下来没有匹配项。到 Window → Entities → Query 看看 Query 实际匹配到了什么。

症状	原因 / 解决方法
用 RefRO 做了写入 (隐蔽的 Bug)	如今编译器会对 refRO.ValueRW = ... 报错。万一旧版 source generator 没拦住, 改用 RefRW。
SystemAPI.Query 和你手搭的 EntityQuery, Entity 数量对不上	两者的过滤条件不一样。SystemAPI.Query 默认带上了可启用 Component 的过滤; EntityQueryBuilder 则要你显式配置。

Chapter 15. IJobEntity 与 IJobChunk

1. 简短结论

- **IJobEntity**——用于"对每个 Entity 做同一件事"。
- **IJobChunk**——用于"对每个 Chunk 做一些并非简单逐 Entity 映射的事"。

IJobEntity 是在 IJobChunk 之上生成出来的——两者跑的是同一套遍历机制。IJobChunk 只不过是把完整的 Chunk 直接交给你的代码。

2. IJobEntity——以 Entity 为单位，写起来顺手

```
[BurstCompile]
public partial struct DamageTickJob : IJobEntity
{
    public float DeltaTime;

    void Execute(ref Health health, in Burn burn)
    {
        health.Value -= burn.DamagePerSecond * DeltaTime;
    }
}
```

优势：

- 一个方法，一个 Entity 视角。好写，也好读。
- 样板代码由源码生成器代劳：Entity、[ChunkIndexInQuery]、[EntityIndexInQuery] 这些参数它会自动填好。
- 过滤条件可用 struct 上的 [WithAll] / [WithNone] / [WithAny] 特性，也可用传给 .Schedule* 的 QueryBuilder 来指定。

弱点：

- Execute 内部看不到其他 Entity。想"对本 Chunk 求和"，就得自己额外维护状态。
 - Execute 内部访问不到 Chunk 级的元数据（变更版本、共享 Component 值）。
-

3. IJobChunk——掌控整个 Chunk

```
[BurstCompile]
public partial struct HealthAggregateJob : IJobChunk
{
    [ReadOnly] public ComponentTypeHandle<Health> HealthHandle;
    public NativeArray<float> PerChunkTotal; // size = chunk count

    public void Execute(in ArchetypeChunk chunk,
                        int unfilteredChunkIndex,
                        bool useEnabledMask,
                        in v128 chunkEnabledMask)
    {
        var healths = chunk.GetNativeArray(ref HealthHandle);
        float total = 0f;
        for (int i = 0; i < chunk.Count; i++)
            total += healths[i].Value;

        PerChunkTotal[unfilteredChunkIndex] = total;
    }
}
```

优势：

- 直接拿到 Chunk 的数组：`chunk.GetNativeArray(ref handle)`、`chunk.GetBufferAccessor(ref bufferHandle)`。
- 能用 `chunk.Has<T>()`、`chunk.GetSharedComponent<T>(...)` 来做条件判断。
- 能看到每个 Chunk 的启用掩码和变更版本。
- 归约、空间分区、批量记录 ECB 都很合适。

代价：

- 样板代码更多：要在 System 里声明 `ComponentTypeHandle<T>`，还得每帧用 `handle.Update(ref state)` 更新它们。
- 内层那个 for 循环得你自己管——出错的地方也就更多。

在 System 中设置句柄

```
[BurstCompile]
public partial struct HealthAggregateSystem : ISystem
{
    private EntityQuery _query;
    private ComponentTypeHandle<Health> _healthHandle;

    public void OnCreate(ref SystemState state)
    {
        _query = state.GetEntityQuery(ComponentType.ReadOnly<Health>());
        _healthHandle = state.GetComponentTypeHandle<Health>(true);
    }
}
```

```
public void OnUpdate(ref SystemState state)
{
    _healthHandle.Update(ref state);

    var totals = CollectionHelper.CreateNativeArray<float>(
        _query.CalculateChunkCount(), state.WorldUpdateAllocator);

    state.Dependency = new HealthAggregateJob
    {
        HealthHandle = _healthHandle,
        PerChunkTotal = totals
    }.ScheduleParallel(_query, state.Dependency);
}
```

handle.Update(ref state) 每帧都会刷新缓存的类型句柄——漏了这一步，Burst 就会取到过期的索引。

4. 启用掩码 (useEnabledMask)

如果 Query 包含 IEnableableComponent，Execute 就会收到 useEnabledMask = true，以及一个 v128 的启用位掩码。你必须照这个掩码来，否则会误处理已禁用的 Entity。

辅助工具：

```
var enumerator = new ChunkEntityEnumerator(useEnabledMask, chunkEnabledMask, chunk.Count);
while (enumerator.NextEntityIndex(out int i))
{
    // i is the index of the next enabled entity in the chunk
}
```

IJobEntity 会自动替你应用掩码——这又是一条理由：除非你需要 Chunk 级的视野，否则优先选它。

5. 决策指南

场景	选择
更新生命值、平移位置、处理输入——纯粹逐 Entity 的数学运算	IJobEntity
对 Chunk 内所有 Entity 的值求和	IJobChunk
每个 Chunk 记一条 ECB 命令，而非每个 Entity 记一条	IJobChunk
循环里要读共享 Component 的值	IJobChunk

场景	选择
需要 Chunk 的变更版本	IJobChunk
遍历每个 Entity 上的 DynamicBuffer<T>	两者都行；IJobEntity 更简洁
逐 Entity 的逻辑，但要用到 Entity 的 Chunk 索引	IJobEntity 配合 [ChunkIndexInQuery] int chunkIndex
调度开销很关键，而工作量又极小	跑基准测试——Chunk 少比 Entity 少更划算。Chunk 数以千计时，IJobChunk 通常更优。

6. 互操作——两者配合使用

一种常见模式：先用 IJobChunk 收集每个 Chunk 的数据，再用 IJobEntity 逐 Entity 写入，两者用依赖串起来。

```
var gather = new HealthAggregateJob { ... }
    .ScheduleParallel(query, state.Dependency);

var act = new HealJob { PerChunkTotal = totals }
    .ScheduleParallel(gather);

state.Dependency = act;
```

7. 故障排查

症状	原因 / 解决方法
IJobChunk.Execute 读出来是垃圾值	调度前忘了调 handle.Update(ref state)，或者用错了 Query。
找不到 ComponentTypeHandle<T>	漏了 using Unity.Entities;; 或者句柄声明的访问模式不对。只读就用 GetComponentTypeHandle<T>(true)。
已启用的 Entity 反被跳过	没按掩码处理——用上文那个 ChunkEntityEnumerator。IJobEntity 不会有这个问题。
两个 Chunk 的 Job 都在写同一个 NativeArray	给每个 Chunk 留一个独立槽位（如 HealthAggregateJob 那样），或者用以 Chunk 索引为键的 NativeParallelHashMap。
调度 IJobChunk 时没传 query 参数	ScheduleParallel(query, ...) 必须显式传入 Query。 IJobEntity.ScheduleParallel() 则靠特性和源码生成自动取得 Query。
找不到 v128 类型	漏了 using Unity.Burst.Intrinsics;;。

Chapter 16. 结构性变更与安全性

1. 什么算作结构性变更

结构性变更，指的是任何可能在 Chunk 之间挪动 Entity、或改变 Chunk 构成的操作。下面这些操作都属于结构性变更：

操作	为何属于结构性变更
CreateEntity / DestroyEntity	会分配或释放 Chunk 槽位。
AddComponent<T> / RemoveComponent<T>	Entity 的 Archetype 变了，于是被实际搬到另一个 Chunk。
SetSharedComponent<T>	共享 Component 值不同 → 落到不同的 Chunk（按值分组）。
AddChunkComponent<T> / RemoveChunkComponent<T>	会改动 Chunk 自身的元数据。
SetName(entity, ...)	会触及 EntityManager 里的 Entity 元数据。
MoveEntitiesFrom	在 World 之间搬运 Entity。
Instantiate(prefab)	会分配新 Entity（可能预先填充一个 Archetype）。

而通过 SetComponentData / RefRW<T>.ValueRW 改写 Component 的**值**，**不算结构性变更**——Component 本就在 Chunk 里，你只是在改它的字节内容。

2. 为何结构性变更是危险的

一旦 Archetype 变了，任何按位置指向 Chunk 内部的引用都会失效：

- 在途的 Query 迭代器 (SystemAPI.Query<...>) 可能漏掉或重复访问某些 Entity。
- 受影响 Entity 的 ComponentLookup<T> 条目失效。
- Job 里的 ArchetypeChunk 句柄指向的还是**旧** Chunk。
- 并发运行的其他 System 看到的是不一致的状态。

在 Editor 里，安全系统大多会以 InvalidOperationException 把这类问题拦下来。可到了构建版本里，它们就成了崩溃或 Entity 损坏。

3. 基本原则

别在同一次 System 更新里，一边遍历 Query 一边做结构性变更。

两种合规的做法，按推荐程度排序：

- 1. 用 EntityCommandBuffer。遍历时先把操作记下来，过后再回放。详见 14_EntityCommandBuffer · Deferred Entity.md。
- 2. 先把要处理的 Entity 收进 NativeList<Entity>，循环结束后再动手。规模小的时候更简单，也没有 ECB 的开销。

第二种做法的示例：

```
var toKill = new NativeList<Entity>(state.WorldUpdateAllocator);

foreach (var (health, e) in SystemAPI
    .Query<RefRO<Health>>()
    .WithEntityAccess())
{
    if (health.ValueRO.Value <= 0f)
        toKill.Add(e);
}

state.EntityManager.DestroyEntity(toKill.AsArray());
```

4. 可启用 Component——逃生舱口

要是你发现自己每帧都在反复增删同一个 Component，那它几乎肯定该改成可启用 Component——切换启用位不算结构性变更。详见 06_Enableable Component.md。

5. Job 与结构性变更

结构性变更是主线程上的操作，不能在 Job 内部做。

要从 Job 里发起变更，惯用的几种办法是：

方式	适用场景
在 Job 里录入 EntityCommandBuffer.ParallelWriter，回到主线程回放	从并行代码发起、逐 Entity 的结构性变更。

方式	适用场景
在 IJob 或 IJobChunk 里录入单线程 EntityCommandBuffer, 回到主线程回放	批量的、或以 Chunk 为范围的变更。
在 Job 里用 EnabledRefRW<T> 切换可启用 Component	逐 Entity 的"激活/停用", 不动结构。

ECB 把操作录成一条条命令；回放则发生在主线程上一个确定的时间点，通常是某个 EntityCommandBufferSystem 的边界。

6. 结合结构性变更与读取

如果两个 System 前后脚运行，而 System A 做了一个 System B 依赖的结构性变更，那就把先后顺序写明确：

```
[UpdateInGroup(typeof(SimulationSystemGroup))]  
[UpdateBefore(typeof(MovementSystem))]  
public partial struct SpawnSystem : ISystem { /* uses ECB */ }
```

只要 ECB System 的回放夹在 A 和 B 之间（比如配上 [UpdateAfter] 链的 EndSimulationECBS），结构性变更就会赶在 B 读取之前落地。

7. 开销模型

单次变更的粗略排名：

SetComponentData (value only)	~ 1x
SetComponentEnabled (enableable bit)	~ 1x (per-chunk bitmask write)
AddComponent (structural)	~ 10x (entity moves chunk)
SetSharedComponent (structural)	~ 10x
DestroyEntity (structural)	~ 10x
Per-frame add + remove pattern	pathological

这只是数量级上的估算；精确数字要看 Archetype 大小、Chunk 占用率和分配器压力。

8. 诊断结构性变更抖动

症状：

- Window → Entities → Systems 里，某个 System 的耗时和它的逻辑量严重不相称。
- Profiler 标记 EntityManager.SetArchetype 或 Move Entity 吃掉了大量帧时间。
- Archetype 数量随时间不断增长。

应对策略：

- 排查一遍 AddComponent / RemoveComponent 调用——有没有每帧都在执行的？换成可启用 Component。
- 看看共享 Component 的取值集合——有没有不小心做成了逐 Entity 的键？
- 到 Archetypes 窗口里找那些只装一个 Entity 的 Chunk；那是 Component 组合切得太碎造成的碎片化。

9. 故障排查

症状	原因 / 解决方法
InvalidOperationException: EntityCommandBuffer has already been played back	ECB 回放之后还往里录命令。每个 ECB 都是一次性的——每帧从 ECBS 重新取一个。
foreach 里报 InvalidOperationException: ... invalidated ...	循环内部做了结构性变更。改用 ECB，或者先攒起来、循环后再应用。
Entity 莫名其妙就没了	DestroyEntity 执行过了，可其他 System 还攥着旧的 Entity 句柄。用 EntityManager.Exists 验一下句柄。
反复增删的写法吃掉大量帧时间	改用可启用 Component。
RemoveComponent 触发"存在清理 Component"错误	这个 Entity 带着清理 Component。先把清理工作做完（参见 05_Component Types.md）。
共享 Component 的取值集合激增 (出现几百个不同的值)	值切得太细了。做个分桶（比如取整到最近的 N），或者把会变的那部分单独存成普通 Component。

Chapter 17. EntityCommandBuffer · 延迟 Entity

1. ECB 存在的原因

EntityCommandBuffer (ECB) 把结构性变更——创建、销毁、增、删、设值——先记录下来，过后在主线程的安全时间点统一回放。借助它，你可以从这些地方变更 World：

- 在遍历 Query 的 foreach 里（这里直接调 EntityManager 会让迭代器失效）。
- 在 Job 里（这里不允许做主线程的结构性变更）。
- 在横跨多个 Chunk 的并行 Job 里（借助 ParallelWriter）。

"为什么这很危险"的来龙去脉，见 13_Structural Change & Safety.md。

2. ECB 生命周期



Figure: EntityCommandBuffer Lifecycle.

ECB 是**一次性的**：回放完就销毁。所以每帧都得向对应的 EntityCommandBufferSystem 要一个新的。

3. 内置 ECB System

System	回放时机	典型用途
BeginInitializationEntityCommandBufferSystem	Initialization Group 开始	处理上一帧遗留的早期 World 设置
EndInitializationEntityCommandBufferSystem	Initialization Group 结束	—
BeginSimulationEntityCommandBufferSystem	Simulation 开始	在游戏逻辑运行前，先把待处理的变更应用上

System	回放时机	典型用途
EndSimulationEntityCommandBufferSystem	Simulation 结束	由游戏逻辑驱动的生成/销毁， 默认就用它
BeginPresentationEntityCommandBufferSystem	Presentation 开始	少见；只用于纯视觉的设置

选对 System，就决定了**其他 System 什么时候能看到你的变更**。假设你在 SimulationSystemGroup 里生成一颗子弹，又希望它**当帧**就能动，那就录到下一帧的 BeginSimulation——要是下一帧才动也无妨，录到 EndSimulation 即可。

4. 获取 ECB

在任意 ISystem.OnUpdate 里：

```
var ecb = SystemAPI
    .GetSingleton<EndSimulationEntityCommandBufferSystem.Singleton>()
    .CreateCommandBuffer(state.WorldUnmanaged);
```

.Singleton 这个嵌套类型，是源码生成器为每个内置 ECB System 提供的、与该 System 对应的令牌。CreateCommandBuffer 会返回一个新的 ECB，它会在那个 System 执行时回放。

并行 Job 则是这样：

```
var ecb = SystemAPI
    .GetSingleton<EndSimulationEntityCommandBufferSystem.Singleton>()
    .CreateCommandBuffer(state.WorldUnmanaged)
    .AsParallelWriter();
```

并行回放的约定见 15_ParallelWriter · Deterministic Playback.md。

5. 录命令——创建 Entity

```
Entity e = ecb.CreateEntity();
ecb.AddComponent(e, new Health { Value = 100 });
ecb.AddComponent(e, LocalTransform.FromPosition(0, 0, 0));
```

或者从一个预制体 Entity 实例化：

```
Entity inst = ecb.Instantiate(prefab);
ecb.SetComponent(inst, LocalTransform.FromPosition(spawnPos));
```

CreateEntity 和 Instantiate 返回的都是一个**延迟 Entity**（见 §6）。

6. 延迟 Entity

ecb.CreateEntity() 或 ecb.Instantiate() 返回的 Entity **此刻还不存在**，它带的是一个负数的占位索引。你可以：

- 把它传给**同一个 ECB** 的后续调用——AddComponent、SetComponent、AppendToBuffer。
- 用 ecb.SetComponent(other, new ReferenceTo { Entity = inst }) 把它存进另一个 Component。

但你**不能**：

- 在 ECB 回放之前，通过 EntityManager 读写它。
- 在不同 ECB 之间传递它（占位符只在所属的那个 ECB 内才能解析）。

回放时，ECB System 会把所有用到占位符的地方都换成真实的 Entity 句柄。

7. 录命令——结构性变更

```
ecb.AddComponent(entity, new Stunned { Duration = 2f });
ecb.RemoveComponent<Stunned>(entity);
ecb.SetComponent(entity, new Health { Value = 50 });

ecb.SetComponentEnabled<Stunned>(entity, false);

var buf = ecb.AddBuffer<Waypoint>(entity);
buf.Add(new Waypoint { Position = new float3(1, 0, 0) });
ecb.AppendToBuffer(entity, new Waypoint { Position = new float3(2, 0, 0) });

ecb.DestroyEntity(entity);
```

上面这些都只是排进队列的命令，回放之前一概不生效。

8. 在 IJobEntity 中使用 ECB

```
[BurstCompile]
public partial struct KillOnHealthZeroJob : IJobEntity
{
    public EntityCommandBuffer ECB;

    void Execute(Entity entity, in Health health)
    {
        if (health.Value <= 0)
            ECB.DestroyEntity(entity);
    }
}
```

```

    }
}

// In OnUpdate:
var ecb = SystemAPI
    .GetSingleton<EndSimulationEntityCommandBufferSystem.Singleton>()
    .CreateCommandBuffer(state.WorldUnmanaged);

new KillOnHealthZeroJob { ECB = ecb }.Schedule();

```

要用 `.ScheduleParallel()` 的话，就改用 `.AsParallelWriter()` 并贯穿传入一个 `sortKey`——下一节细讲。

9. 独立（手动）ECB

有时你想自己掌控回放的时机——比如就在一个 `OnUpdate` 内部：

```

var ecb = new EntityCommandBuffer(Allocator.Temp);
// ... record ...
ecb.Playback(state.EntityManager);
ecb.Dispose();

```

这适合自成一体的一次性变更。但凡要跨 `System` 边界、或者在 `Job` 中运行的情况，都用上面那些内置 ECBS。

10. 确定性

在单写入者的 ECB 上，命令按**录入顺序**回放。在 `ParallelWriter` 上，回放顺序则由你给的 `sortKey` 决定——参见 `15_ParallelWriter · Deterministic Playback.md`。

对 `Netcode` 和回放系统来说，确定性回放没有商量余地。务必传入稳定的排序键（`Chunk` 索引、`Query` 内 `Entity` 索引），也别在那些以非确定性顺序运行的主线程代码路径里录命令。

11. 故障排查

症状	原因 / 解决方法
<code>InvalidOperationException: ECB has already been played back</code>	跨帧复用了 ECB。每帧用 <code>CreateCommandBuffer</code> 取一个新的。

症状	原因 / 解决方法
回放后延迟 Entity 句柄变成了 Entity.Null	它被横跨两个不同的 ECB 引用了。创建和后续的设置必须在同一个 ECB 里完成。
生成的 Entity 当帧没出现	命令录到了 EndSimulation ECBS——它在帧末才回放。改录到 BeginSimulation，下一帧就能在大多数 System 运行前看到它。
ecb.SetComponent 抛出"Component 不存在"	目标 Entity 还没有这个 Component（先调一下 AddComponent），或者回放还没发生。
对由 ECBS 托管的 ECB 调用了 Dispose()	别对 CreateCommandBuffer 返回的 ECB 调 Dispose()——它的生命周期由 ECBS 负责。只有独立创建的（new EntityCommandBuffer(Allocator.Temp)）才需要你 Dispose。
并行 ECB 回放时命令顺序乱了	sortKey 不具确定性。改用 [ChunkIndexInQuery] 或 Query 提供的稳定索引。

Chapter 18. ParallelWriter · 确定性回放

1. 为何存在独立的并行写入器

EntityCommandBuffer 只支持单个写入者。两个线程往同一个 ECB 录命令，会在它的内部链上发生竞争。EntityCommandBuffer.ParallelWriter 的解决办法是：给每个调用方一条**线程本地链**，并要求每条命令都带上一个**排序键**。

回放时，ECB System 会：

1. 把所有线程本地链摊平。
2. 按 sortKey 给命令排序（稳定排序）。
3. 在主线程上逐条执行。

只要排序键是确定性的，结果就是确定性的。

2. 获取 ParallelWriter

```
var ecb = SystemAPI
    .GetSingleton<EndSimulationEntityCommandBufferSystem.Singleton>()
    .CreateCommandBuffer(state.WorldUnmanaged)
    .AsParallelWriter();
```

AsParallelWriter() 返回一个 struct EntityCommandBuffer.ParallelWriter，可以拷贝进 IJobEntity、IJobChunk 或 IJobParallelFor。

3. sortKey 契约

每个并行 ECB 调用，第一个参数都是 sortKey：

```
ecb.Instantiate(sortKey, prefab);
ecb.SetComponent(sortKey, entity, value);
ecb.DestroyEntity(sortKey, entity);
ecb.AddComponent(sortKey, entity, component);
```

好的排序键的规则：

- **确定性。** 不管调度器怎么分配工作，相同输入都得出相同的键。

- **足够区分。** 除非你有意让回放顺序留有余地，否则同一 Entity 的两条命令不该撞键。
- **开销小。** 用一个手头现成的 int，别去算哈希。

实践中，常用的选择是：

来源	获取方式	适用场景
IJobEntity 上的 [ChunkIndexInQuery]	源码生成器自动填充	99% 的情况
IJobChunk.Execute 中的 unfilteredChunkIndex	API 提供	IJobChunk Job
IJobEntity 上的 [EntityIndexInQuery]	源码生成器	需要让每个 Entity 的命令各自有序时
手动计算的确定性索引	自行计算	少见；自定义形态的 Job

4. 完整示例——并行 Spawn

```
using Unity.Burst;
using Unity.Entities;
using Unity.Mathematics;
using Unity.Transforms;

[BurstCompile]
public partial struct SpawnParallelJob : IJobEntity
{
    public float DeltaTime;
    public EntityCommandBuffer.ParallelWriter ECB;

    void Execute([ChunkIndexInQuery] int chunkIndex,
                Entity spawnerEntity,
                ref Spawner spawner)
    {
        if (spawner.RemainingCount <= 0)
        {
            ECB.DestroyEntity(chunkIndex, spawnerEntity);
            return;
        }

        spawner.TimeLeft -= DeltaTime;
        if (spawner.TimeLeft > 0f) return;

        spawner.TimeLeft = spawner.Interval;
        spawner.RemainingCount--;

        var inst = ECB.Instantiate(chunkIndex, spawner.Prefab);
        ECB.SetComponent(chunkIndex, inst, LocalTransform.FromPosition(spawner.SpawnPoint));
    }
}

[BurstCompile]
public partial struct SpawnerParallelSystem : ISystem
{

```

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
{
    var ecb = SystemAPI
        .GetSingleton<EndSimulationEntityCommandBufferSystem.Singleton>()
        .CreateCommandBuffer(state.WorldUnmanaged)
        .AsParallelWriter();

    new SpawnParallelJob
    {
        DeltaTime = SystemAPI.Time.DeltaTime,
        ECB       = ecb
    }.ScheduleParallel();
}
```

把 chunkIndex 同时传给 Instantiate 和随后的 SetComponent，能保证这两条命令排到同一位置，从而让 Instantiate 返回的延迟 Entity 引用解析到正确的目标。

5. 确定性——保证什么，不保证什么

保证的： ECB 内容相同时，不管哪条命令是哪个工作线程录的，回放都会得到相同的结果状态。

不会自动保证的：

- **Entity ID。** 如果在此之前的命令有差异，两次运行生成的 Entity 可能有不同的 Index 值。
- **跨 System 的 Chunk 顺序。** 如果 System A 并行生成 Entity，System B 又按 ChunkIndexInQuery 顺序读取它们，遍历顺序仍可能受 Archetype 的创建历史影响。

如果你要的是跨运行逐位一致的确定性（回放、Netcode 回滚），就把以下几样组合起来用：

1. 排序键确定的并行 ECB。
2. 放在 FixedStepSimulationSystemGroup 里的固定步长 System。
3. 不依赖 Entity.Index，改用你自己掌控的游戏 ID 作键。

6. 排序键模式

6.1 按 Chunk（默认）

用 [ChunkIndexInQuery] int chunkIndex。如果 Chunk 里每个 Entity 生成的命令都一样（比如整个 Chunk 一起被标记为眩晕），那键冲突没关系。

6.2 按 Entity

当你需要严格的逐 Entity 排序时（比如记录那些必须按 Entity 顺序回放的逐 Entity 伤害事件），用 [EntityIndexInQuery] int entityIndex。

6.3 自定义确定性索引

极个别情况下，你会自己从输入算出一个扁平索引——比如 `int sortKey = baseOffset + localIndex;`。只有在前面两种常用方式都不合用时才这么做；自定义索引是确定性 Bug 的常见源头。

7. 避免以下模式

- 拿 `JobsUtility.ThreadIndex` 当排序键。 它不具确定性。
- 用 `Random` 来生成键。 同样不具确定性。
- 混用多个 ECB。 命令录进两个不同的 ECB，就丢掉了整体的排序。
- 从多个 System 往同一个 System 的 ECB 里录。 一个 ECB 只对应一个录入方（或一个包装着同一 ECB 的并行写入者）。

8. 故障排查

症状	原因 / 解决方法
延迟 Entity 解析到了错误的目标	Instantiate 和随后的 SetComponent 用了不同的排序键。让它们保持一致。
每次运行命令顺序都不一样	排序键不具确定性。改用 [ChunkIndexInQuery]。
编译报错"JobEntity must have partial"	给 Job struct 加上 partial。
ECB.SetComponent 在并行 Job 中抛出"线程错误"	在并行 Job 里用了非并行的 EntityCommandBuffer。先调 .AsParallelWriter()，再用并行版的 API。
Netcode 回放出现去同步	某个并行 ECB 没用确定性排序键，或者某个 IJobEntity 捕获了非确定性的值。把整条调用链查一遍。
在已释放的 ECB 上调用 AsParallelWriter()	ECB 已经回放过了——本帧重新取一个。

Chapter 19. Netcode 客户端-服务器 World 与 Bootstrap

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

1. 概述

Netcode for Entities 采用**客户端-服务器模型**，把客户端逻辑和服务器逻辑拆到不同的 ECS World 里。

World	角色
Server World	权威模拟。
Client World	本地玩家模拟、预测、插值，以及表现层。
Thin Client World	供开发和压力测试用的虚拟客户端。

当客户端设备同时充当服务器主机时，项目就跑在**客户端托管服务器**这种配置下。

2. World 类型与 System 过滤

2.1 WorldSystemFilter

用 WorldSystemFilter 声明一个 System 能在哪个 Netcode World 里运行。

```
using Unity.Entities;

[WorldSystemFilter(WorldSystemFilterFlags.ServerSimulation)]
public partial struct ServerOnlySystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {
        // Runs only in the server world.
    }
}
```

标志	含义
LocalSimulation	不带 Netcode System 的本地 World。
ServerSimulation	服务器模拟 World。
ClientSimulation	客户端模拟 World。
ThinClientSimulation	Thin Client 模拟 World。

2.2 System Group 过滤

有些 Netcode Group 只存在于特定的 World，所以把 System 放进这种 Group，也就顺带筛选了 World。

```
using Unity.Entities;
using Unity.NetCode;

[UpdateInGroup(typeof(GhostInputSystemGroup))]
public partial struct MyInputSystem : ISystem
{
    // GhostInputSystemGroup exists in Client and Local worlds.
    // The system is excluded from Server worlds.
}
```

System Group	存在于哪些 World
GhostInputSystemGroup	Client、Local
PredictedSimulationSystemGroup	Client、Server
PresentationSystemGroup	仅 Client；Server 和 Thin Client 都没有

3. Bootstrap 流程

ClientServerBootstrap 会在运行时创建 Server World 和 Client World。

3.1 默认行为

默认的 Bootstrap 会在启动时把 Server World 和 Client World 一起建好。但它不会自动把两者连起来，何时监听、何时连接仍由游戏自己掌控。

3.2 自定义 Bootstrap

```
using Unity.NetCode;

public class GameBootstrap : ClientServerBootstrap
{
    public override bool Initialize(string defaultWorldName)
    {
        // Create only the default local world.
        CreateLocalWorld(defaultWorldName);
        return true;
    }
}
```

之后你就能从开始按钮、大厅流程、测试框架或命令行来创建 World。

```
using Unity.NetCode;

void OnPlayButtonClicked()
{
    var serverWorld = ClientServerBootstrap.CreateServerWorld("ServerWorld");
    var clientWorld = ClientServerBootstrap.CreateClientWorld("ClientWorld");

    AutomaticThinClientWorldsUtility.NumThinClientsRequested = 4;
    AutomaticThinClientWorldsUtility.BootstrapThinClientWorlds();
}
```

AutomaticThinClientWorldsUtility 是 1.5.0 版本线新增的，用来在运行时创建或清理 Thin Client World。

4. Tick 频率配置

服务器的模拟速度，通过 ClientServerTickRate 这个 Singleton 来配置。

属性	默认值	含义
SimulationTickRate	60	服务器每秒的模拟 Tick 数。
NetworkTickRate	与 SimulationTickRate 相同	快照的发送速率，以每秒 Tick 数计。
MaxSimulationStepsPerFrame	4	每帧最多走多少个模拟步。
MaxSimulationStepBatchSize	4	能用缩放后的 deltaTime 合并批处理的 Tick 数。
TargetFrameRateMode	BusyWait	BusyWait、Sleep 或 Auto。

```
using Unity.Entities;
using Unity.NetCode;
using UnityEngine;

public class TickRateAuthoring : MonoBehaviour
{
    public int SimulationTickRate = 60;
    public int NetworkTickRate = 30;

    class Baker : Baker<TickRateAuthoring>
    {
        public override void Bake(TickRateAuthoring authoring)
        {
            var entity = GetEntity(TransformUsageFlags.None);
            AddComponent(entity, new ClientServerTickRate
            {
                SimulationTickRate = authoring.SimulationTickRate,
                NetworkTickRate = authoring.NetworkTickRate,
            });
        }
    }
}
```

```
}  
}
```

把 NetworkTickRate 调到比 SimulationTickRate 低，能省带宽。这种配置下，GhostSendSystem 的负载会以轮询调度的方式摊到多个模拟 Tick 上。

5. 服务器与客户端更新流程

```
Server World  
└─ Fixed timestep, 60 Hz default  
   └─ SimulationSystemGroup  
      ├── GhostSendSystem  
      └─ CommandReceiveSystem  
  
Client World  
├─ Dynamic timestep  
│   └─ SimulationSystemGroup  
│       ├── GhostReceiveSystem  
│       └─ CommandSendSystem  
├─ Fixed timestep, same as the server  
│   └─ PredictedSimulationSystemGroup  
│       └─ Prediction code and rollback loop  
└─ PresentationSystemGroup  
    └─ Rendering and interpolation
```

服务器用固定时间步长来做确定性的权威模拟。客户端用动态时间步长跑普通模拟和表现层，而预测代码则在固定的预测循环里运行。

6. World 迁移

当 World 切换时必须保住连接状态，就用 DriverMigrationSystem。

```
using Unity.Entities;  
using Unity.NetCode;  
  
public World MigrateServerWorld(World sourceWorld)  
{  
    DriverMigrationSystem migrationSystem = null;  
    foreach (var world in World.All)  
    {  
        if ((migrationSystem = world.GetExistingSystemManaged<DriverMigrationSystem>()) != null)  
            break;  
    }  
  
    var ticket = migrationSystem.StoreWorld(sourceWorld);  
    sourceWorld.Dispose();  
  
    var newWorld = ClientServerBootstrap.CreateServerWorld("NewServerWorld");  
    var newMigrationSystem = newWorld.GetExistingSystemManaged<DriverMigrationSystem>();
```



```
newMigrationSystem.LoadWorld(ticket);  
return newWorld;  
}
```

7. 相关文档

- 03_ECS Core Concepts.md——World、EntityManager、Archetype 和 Chunk 的基础知识。
- 09_System Group & Update Order.md——Netcode 在其上扩展的那些基础 ECS System Group。
- 17_Netcode Network Connection & Approval.md——监听、连接、审批，以及进入游戏状态。

8. 故障排查

症状	原因 / 解决方法
某个 System 跑到了服务器上，本不该如此	加上 WorldSystemFilter，或把它挪进正确的 Netcode System Group。
Tick 频率设置不起作用	确认 ClientServerTickRate 已经 Bake 进了一个已加载的 SubScene。
Thin Client 没被创建出来	确认在设好 NumThinClientsRequested 之后调用了 AutomaticThinClientWorldsUtility.BootstrapThinClientWorlds()。
客户端和服务器自己就连上了	查一下 Bootstrap 里是不是设了 AutoConnectPort。
表现层代码在 Play 模式下报错	渲染 System 跑到了服务器 World 里。加上 ClientSimulation 过滤，或把它挪进 PresentationSystemGroup。

Chapter 20. Netcode 网络连接与审批

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

1. 概述

Netcode for Entities 把每条网络连接都表示成一个 **Entity**。这个连接 Entity 上带着 `NetworkStreamConnection`；连接一结束，该 Entity 就被销毁。

在双方都加上 `NetworkStreamInGame` 之前，连接不算"在游戏中"。没有这个 Component，服务器不发快照，客户端也不发命令。

2. 核心连接 Component

Component	含义
<code>NetworkStreamConnection</code>	保存传输层句柄。
<code>NetworkId</code>	服务器审批通过后分配的唯一 ID。
<code>NetworkStreamInGame</code>	交换快照和命令数据的前提。
<code>CommandTarget</code>	接收玩家命令的 Entity；除非用了 <code>AutoCommandTarget</code> ，否则由游戏代码维护。
<code>NetworkStreamRequestDisconnect</code>	断开连接的请求；别直接调驱动层去断开。
<code>NetworkStreamIsReconnected</code>	1.6.1 版本线新增的重连跟踪 Component。

3. 三种连接方式

3.1 用 `NetworkStreamDriver` 手动连接

```
using Unity.Entities;
using Unity.Networking.Transport;
using Unity.NetCode;

// Server.
var serverDriver = SystemAPI.GetSingletonRW<NetworkStreamDriver>();
serverDriver.ValueRW.Listen(NetworkEndpoint.AnyIpv4.WithPort(7979));
```

```
// Client.
var clientDriver = SystemAPI.GetSingletonRW<NetworkStreamDriver>();
clientDriver.ValueRW.Connect(
    state.EntityManager,
    NetworkEndpoint.Parse("127.0.0.1", 7979));
```

3.2 在 Bootstrap 中使用 AutoConnectPort

```
using Unity.NetCode;

public class AutoConnectBootstrap : ClientServerBootstrap
{
    public override bool Initialize(string defaultWorldName)
    {
        AutoConnectPort = 7979;
        CreateDefaultClientServerWorlds();
        return true;
    }
}
```

服务器监听通配符地址，客户端连到回环地址。

3.3 请求 Component

```
using Unity.Entities;
using Unity.NetCode;

var listenReq = serverWorld.EntityManager.CreateEntity(
    typeof(NetworkStreamRequestListen));
serverWorld.EntityManager.SetComponentData(listenReq,
    new NetworkStreamRequestListen { Endpoint = serverEndpoint });

var connectReq = clientWorld.EntityManager.CreateEntity(
    typeof(NetworkStreamRequestConnect));
clientWorld.EntityManager.SetComponentData(connectReq,
    new NetworkStreamRequestConnect { Endpoint = serverEndpoint });
```

4. 连接状态机

4.1 客户端状态

Connecting → Handshake → [Approval] → Connected → Disconnected

状态	含义
Connecting	调用 Connect 之后触发一次。
Handshake	传输层连接已建立，协议握手尚待完成。

状态	含义
Approval	仅在启用连接审批时存在。
Connected	服务器已分配 NetworkId。
Disconnected	已断开或超时； DisconnectReason 已被设置。

4.2 服务器状态

Handshake → [Approval] → Connected → Disconnected

状态	含义
Handshake	客户端已接纳；正在交换 NetworkProtocolVersion。
Approval	仅在启用连接审批时存在。
Connected	连接已审批通过，并分配了 NetworkId。
Disconnected	客户端已断开。

4.3 监听连接事件

```
using Unity.Burst;
using Unity.Entities;
using Unity.NetCode;
using UnityEngine;

[UpdateAfter(typeof(NetworkReceiveSystemGroup))]
[BurstCompile]
public partial struct ConnectionEventListener : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        var events = SystemAPI.GetSingleton<NetworkStreamDriver>()
            .ConnectionEventsForTick;

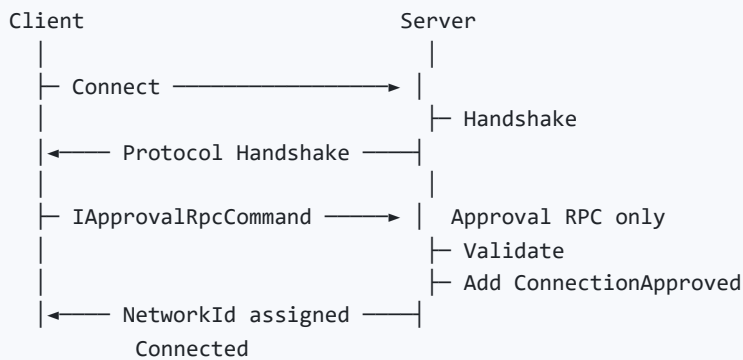
        foreach (var evt in events)
        {
            Debug.Log($"[{state.WorldUnmanaged.Name}] {evt.ToFixedString()}!");
        }
    }
}
```

ConnectionEventsForTick 只在 SimulationSystemGroup 内有效；在别处读取可能漏掉或重复事件。

5. 连接审批

连接审批让服务器在分配 `NetworkId` 之前，先校验白名单、密码、Token、匹配票据之类的准入条件。

5.1 审批流程



5.2 启用审批

```
using Unity.Entities;
using Unity.NetCode;

using var drvQuery = serverWorld.EntityManager
    .CreateEntityQuery(ComponentType.ReadWrite<NetworkStreamDriver>());

drvQuery.GetSingletonRW<NetworkStreamDriver>().ValueRW
    .RequireConnectionApproval = true;
```

客户端这边也要设上同样的审批要求。

5.3 审批 RPC

```
using Unity.Collections;
using Unity.Entities;
using Unity.NetCode;

public struct ApprovalRpc : IApprovalRpcCommand
{
    public FixedString512Bytes Token;
}

[WorldSystemFilter(WorldSystemFilterFlags.ClientSimulation |
    WorldSystemFilterFlags.ThinClientSimulation)]
public partial struct ClientApprovalSystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {
        var ecb = new EntityCommandBuffer(Allocator.Temp);

        foreach (var (connection, entity) in
            SystemAPI.Query<RefRW<NetworkStreamConnection>>())
```

```

        .WithNone<NetworkId>()
        .WithNone<ApprovalSent>()
        .WithEntityAccess())
    {
        var rpcEntity = ecb.CreateEntity();
        ecb.AddComponent(rpcEntity, new ApprovalRpc { Token = "MY_TOKEN" });
        ecb.AddComponent<SendRpcCommandRequest>(rpcEntity);
        ecb.AddComponent<ApprovalSent>(entity);
    }

    ecb.Playback(state.EntityManager);
}

[WorldSystemFilter(WorldSystemFilterFlags.ServerSimulation)]
public partial struct ServerApprovalSystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {
        var ecb = new EntityCommandBuffer(Allocator.Temp);

        foreach (var (req, rpc, entity) in
            SystemAPI.Query<RefRO<ReceiveRpcCommandRequest>, RefRO<ApprovalRpc>>()
                .WithEntityAccess())
        {
            var connEntity = req.ValueRO.SourceConnection;

            if (rpc.ValueRO.Token.Equals("MY_TOKEN"))
                ecb.AddComponent<ConnectionApproved>(connEntity);
            else
                ecb.AddComponent<NetworkStreamRequestDisconnect>(connEntity);

            ecb.DestroyEntity(entity);
        }

        ecb.Playback(state.EntityManager);
    }
}

```

审批超时由 `ClientServerTickRate.HandshakeApprovalTimeoutMS` 控制，默认 5000 毫秒。

6. 进入游戏状态

审批通过后，客户端和服务端双方都得加上 `NetworkStreamInGame`，游戏数据才会开始交换。

```

using Unity.Collections;
using Unity.Entities;
using Unity.NetCode;

[WorldSystemFilter(WorldSystemFilterFlags.ServerSimulation)]
public partial struct ServerGoInGameSystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {
        var ecb = new EntityCommandBuffer(Allocator.Temp);
    }
}

```

```

        foreach (var (id, entity) in
            SystemAPI.Query<RefRO<NetworkId>>()
                .WithNone<NetworkStreamInGame>()
                .WithEntityAccess())
        {
            ecb.AddComponent<NetworkStreamInGame>(entity);
            // Spawn the player entity and set CommandTarget as needed.
        }

        ecb.Playback(state.EntityManager);
    }
}

[WorldSystemFilter(WorldSystemFilterFlags.ClientSimulation |
    WorldSystemFilterFlags.ThinClientSimulation)]
public partial struct ClientGoInGameSystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {
        var ecb = new EntityCommandBuffer(Allocator.Temp);

        foreach (var (id, entity) in
            SystemAPI.Query<RefRO<NetworkId>>()
                .WithNone<NetworkStreamInGame>()
                .WithEntityAccess())
        {
            ecb.AddComponent<NetworkStreamInGame>(entity);
        }

        ecb.Playback(state.EntityManager);
    }
}

```

7. 数据流 Buffer

Buffer	方向	数据
IncomingRpcDataStreamBuffer	接收	RPC 数据。
IncomingCommandDataStreamBuffer	接收	命令数据。
IncomingSnapshotDataStreamBuffer	在客户端接收	快照数据
OutgoingRpcDataStreamBuffer	发送	RPC 数据
OutgoingCommandDataStreamBuffer	在客户端发送	命令数据

8. 主机迁移

主机迁移能在服务器更换时保住客户端的连接状态。

项目	含义
激活	1.7.0 版本线起默认启用；更早的版本需要 <code>ENABLE_HOST_MIGRATION</code> 。
重连跟踪	重连之后，两端的连接 Entity 上都会被加上 <code>NetworkStreamIsReconnected</code> 。
预生成 Ghost	预生成 Ghost 的 ID 在主机迁移过程中会被保留。

1.7.0 版本线大幅重构了主机迁移的 API 命名，旧版示例要先迁移才能用。

9. 相关文档

- 16_Netcode Client-Server World & Bootstrap.md——Netcode World 是怎么创建的。
- 18_Netcode Ghost Snapshot & Synchronization.md——为什么快照交换要以 `NetworkStreamInGame` 为前提。
- 21_Netcode RPC.md——普通 RPC 与审批 RPC。

10. 故障排查

症状	原因 / 解决方法
连接成功了，Ghost 却没出现	客户端和服务端上都要加 <code>NetworkStreamInGame</code> 。
审批通过了，数据还是不交换	确认 <code>ConnectionApproved</code> 已经加到了连接 Entity 上。
应用一失去焦点连接就断	设置 <code>Application.runInBackground = true</code> 。
协议版本不匹配	客户端和服务端要用完全一致的构建和 Ghost 配置。
审批超时	调大 <code>HandshakeApprovalTimeoutMS</code> ，或者排查审批 RPC 这条路径。

Chapter 21. Netcode Ghost 快照与同步

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

1. 概述

Ghost 是一种网络 Entity——它在服务器上模拟，再复制到各个客户端。服务器会定期把 Ghost 状态打包成快照发出去，每个客户端则通过插值或预测来应用这些快照。

机制	用途
Ghost	在不可靠数据包之上实现最终一致性，并内置了复制优化。
RPC	一次性可靠事件。

2. Ghost 与 RPC 的对比

使用 Ghost 的场景	使用 RPC 的场景
客户端附近、有空间属性的 Entity 状态。	偏宏观的游戏流程事件。
需要客户端预测的数据。	聊天、改设置这类一次性命令。
断线、重连之后仍需保留的状态。	不带持久复制状态的事件。

3. Ghost Authoring

给需要复制的 Prefab 加上 GhostAuthoringComponent。

属性	含义
Name	Ghost 标识符。
Importance	带宽吃紧时的快照优先级。
Supported Ghost Mode	All、Interpolated 或 Predicted。
Default Ghost Mode	Interpolated、Predicted 或 OwnerPredicted。
Optimization Mode	默认 Dynamic；极少变化的对象可用 Static。

属性	含义
MaxSendRate	最大复制频率（Hz），受 NetworkTickRate 限制。
UseSingleBaseline	1.5.0 版本线新增的单基线增量压缩开关。

3.1 Ghost 模式

模式	含义
Interpolated	客户端在多帧服务器快照之间做插值。画面流畅，但有延迟。
Predicted	客户端提前模拟，以降低输入延迟。CPU 开销更高。
OwnerPredicted	归属方客户端做预测，其余客户端做插值。

4. 序列化特性

4.1 GhostField

给那些需要序列化的 Component 字段标上 GhostField。

```
using Unity.Entities;
using Unity.Mathematics;
using Unity.NetCode;

public struct MyComponent : IComponentData
{
    [GhostField(Quantization = 1000, Smoothing = SmoothingAction.Interpolate)]
    public float3 Position;

    [GhostField(Quantization = 100)]
    public float Health;

    [GhostField]
    public int Score;

    [GhostField(SendData = false)]
    public float LocalOnly;
}
```

GhostField 属性	含义
Quantization	浮点转整数时的乘数；1000 即保留三位小数。
Smoothing	Clamp、Interpolate 或 InterpolateAndExtrapolate。
MaxSmoothingDistance	距离阈值；超过它就不做插值，以适应传送。
Composite	把整个 struct 当成一个变更位来处理。

GhostField 属性	含义
SendData	设为 false，该字段就不参与序列化。
SubType	指定一条专用的序列化规则。

4.2 GhostEnabledBit

用 GhostEnabledBit 来复制可启用 Component 的启用状态。

```
using Unity.Entities;
using Unity.NetCode;

[GhostEnabledBit]
public struct MyEnableableComp : IComponentData, IEnableableComponent
{
    [GhostField] public int Value;
}
```

4.3 GhostComponent

用 GhostComponent 来控制 Prefab 变体、子 Entity 的序列化，以及复制范围。

```
using Unity.Entities;
using Unity.NetCode;

[GhostComponent(
    PrefabType = GhostPrefabType.All,
    SendTypeOptimization = GhostSendType.AllClients,
    SendToOwner = SendToOwnerType.SendToNonOwner)]
public struct EnemyData : IComponentData
{
    [GhostField] public float Health;
}
```

属性	选项
PrefabType	InterpolatedClient、PredictedClient、Client、Server、AllPredicted、All。
SendTypeOptimization	None、Interpolated、Predicted、All。
SendToOwner	SendToOwner、SendToNonOwner、All。
SendDataForChildEntity	复制子 Entity 的 Component 数据；比复制根 Entity 数据慢。

5. Buffer 序列化

对 IBufferElementData 来说，每个公共字段都必须带 [GhostField]。

```
using Unity.Entities;
using Unity.NetCode;

[InternalBufferCapacity(8)]
public struct MyBuffer : IBufferElementData
{
    [GhostField] public int Value;
    [GhostField] public float Score;
}
```

Buffer 的要求比 Component 严：Buffer 元素的每个公共字段都得标注，而不是只标要序列化的那几个。

6. Ghost Component 变体

用变体可以定义一套替代的序列化方案，又不必改动原始的 Component 类型。

```
using Unity.Mathematics;
using Unity.NetCode;
using Unity.Transforms;

[GhostComponentVariation(typeof(LocalTransform), "Transform - 2D")]
[GhostComponent(PrefabType = GhostPrefabType.All)]
public struct Transform2dVariant
{
    [GhostField(Quantization = 1000, Smoothing = SmoothingAction.InterpolateAndExtrapolate)]
    public float3 Position;
}
```

变体	用途
ClientOnlyVariant	该 Component 只存在于客户端。
ServerOnlyVariant	该 Component 只存在于服务器。
DontSerializeVariant	完全禁用序列化。

7. 快照系统

7.1 部分快照

一旦快照会超过 MTU，Netcode 就先发重要性更高的 Ghost，剩下的留到后续 Tick 再发——于是 World 状态会以流式方式逐步送达。

7.2 NetworkTickRate

当 NetworkTickRate 低于 SimulationTickRate 时, GhostSendSystem 的工作量会摊到多个模拟 Tick 上。

```
SimulationTickRate = 60, NetworkTickRate = 30
Tick 1: GhostSendSystem sends a snapshot
Tick 2: GhostSendSystem skips
Tick 3: GhostSendSystem sends a snapshot
...
```

8. Ghost Prefab 注册 workflow

1. Create a Ghost prefab
 - └ Add GhostAuthoringComponent
 - └ Add the required authoring components
2. Place or reference it from a SubScene
 - └ Put the prefab inside the SubScene, or
 - └ Let GhostCollection register the prefab
3. Bake
 - └ Baker analyzes GhostField attributes
 - └ Source Generator emits serialization code
4. Runtime
 - └ Server instantiates the Ghost entity
 - └ GhostSendSystem sends snapshots
 - └ Client automatically spawns the Ghost

8.1 预生成 Ghost

摆在 SubScene 里的 Ghost 会成为**预生成 Ghost**:

- SubScene 加载时, 服务器和客户端会自动对应到同一个 Entity。
- 不需要单独的 Spawn 消息, 靠快照同步就够了。
- 在 1.6.1+ 版本线中, 主机迁移期间预生成 Ghost 的 ID 会被保留。

建筑、地形这类静态世界对象, 通常最适合做成开了静态优化模式的预生成 Ghost。

9. 检查清单

- ☐ 给 Ghost Prefab 加上 GhostAuthoringComponent。
 - ☐ 用 [GhostField] 标出需要复制的字段。
 - ☐ 选好 Ghost 模式：Predicted、Interpolated 或 OwnerPredicted。
 - ☐ 加上 NetworkStreamInGame，快照交换才能开始。
 - ☐ 按带宽和精度需求调好 Quantization。
 - ☐ 只在确实需要子 Entity 复制时，才开 SendDataForChildEntity。
-

10. 相关文档

- 22_Netcode Ghost Optimization · Importance · Relevancy.md——带宽、相关性与发送速率的控制。
 - 19_Netcode Prediction & Rollback.md——预测 Ghost 的模拟。
 - ../Changelog/Netcode for Entities 1.4 → 6.5 Key Changes.md——跨版本的 Ghost 序列化变更。
-

11. 故障排查

症状	原因 / 解决方法
Ghost 没出现在客户端	加上 NetworkStreamInGame，并确认 Ghost Prefab 已经注册。
浮点值抖动	Quantization 太低，或者没做平滑。
传送时出现插值瑕疵	设置 MaxSmoothingDistance，或用一条自定义的平滑路径。
协议版本不匹配	客户端和服务器的 Ghost Schema 对不上。
Buffer 没被序列化	Buffer 元素的每个公共字段都必须带 [GhostField]。

Chapter 22. Netcode 预测与回滚

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

1. 概述

客户端预测，就是在权威的服务器结果到来之前，先在客户端跑一遍同样的模拟代码。这能压低输入延迟，让玩家自己控制的移动显得即时跟手。

预测是有实打实的 CPU 开销的：权威快照一到，客户端可能要回滚到更早的 Tick，再一路向前重新模拟到当前的预测 Tick。

2. 核心 Component 与 System

Component / System	含义
PredictedGhost	客户端上加在预测 Ghost 上，服务器上则加在所有 Ghost 上。
Simulate	一个可启用 Tag；该在当前预测 Tick 参与模拟的 Entity 上才会设它。
PredictedSimulationSystemGroup	跑预测循环的那个固定时间步长 Group。
NetworkTime	保存服务器 Tick、预测 Tick，以及相关的计时数据。

3. 客户端预测流程

1. Receive a server snapshot
 - └ Apply the latest authoritative state to predicted entities
 2. Find the oldest applied tick
 - └ Use the oldest tick across affected predicted entities
 3. Roll back and re-simulate
 - └ PredictedSimulationSystemGroup repeats from oldest tick to target tick
 - └ TimeData and NetworkTime update for each tick
 4. Render
 - └ Present the predicted result

延迟一高，开销就成倍上涨。视 Tick 频率和预测设置而定，300 毫秒的延迟可能要重新模拟 20 多帧。

4. 编写预测 System

4.1 使用 Simulate Tag

预测循环一开始，Netcode 会先把预测 Ghost 上的 Simulate 全部禁用，再只为那些该在当前 Tick 运行的 Entity 把它打开。

```
using Unity.Burst;
using Unity.Entities;
using Unity.Mathematics;
using Unity.NetCode;
using Unity.Transforms;

[UpdateInGroup(typeof(PredictedSimulationSystemGroup))]
public partial struct MovementSystem : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        var speed = 5f;
        var dt = SystemAPI.Time.DeltaTime;

        foreach (var (transform, input) in
            SystemAPI.Query<RefRW<LocalTransform>, RefRO<PlayerInput>>()
                .WithAll<Simulate>())
        {
            var move = new float3(input.ValueRO.Horizontal, 0, input.ValueRO.Vertical);
            transform.ValueRW.Position += move * speed * dt;
        }
    }
}
```

预测 Query 里务必带上 `.WithAll<Simulate>()`。少了它，System 可能会去重新模拟那些已经结算完的 Tick。

4.2 服务器端行为

在服务器上，预测循环每个服务器 Tick 只跑一次：

- TimeData 里是常规的服务器 Tick 数据。
- Simulate 始终是启用的。
- 同一份 System 代码，客户端和服务器都能跑。

5. 远程玩家预测

只要另一个玩家的命令数据也参与了序列化，那个远程玩家同样可以被预测。


```

using Unity.Burst;
using Unity.Entities;
using Unity.Mathematics;
using Unity.NetCode;
using Unity.Transforms;

[UpdateInGroup(typeof(PredictedSimulationSystemGroup))]
public partial struct RemotePlayerMovement : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        foreach (var (transform, input) in
            SystemAPI.Query<RefRW<LocalTransform>, RefRO<PlayerInput>>()
                .WithAll<PredictedGhost, Simulate>())
        {
            transform.ValueRW.Position +=
                new float3(input.ValueRO.Horizontal, 0, input.ValueRO.Vertical)
                * SystemAPI.Time.DeltaTime;
        }
    }
}

```

用了 `IInputComponentData` 时，输入系统会自动为当前 Tick 备好输入。

6. 预测平滑

当服务器结果和预测结果对不上时，预测平滑能减少肉眼可见的纠正跳变。

```

using Unity.NetCode;
using Unity.Transforms;

GhostPredictionSmoothing.RegisterSmoothingAction<LocalTransform>(
    state.EntityManager,
    CustomSmoothing.Action);

```

```

using Unity.Burst;
using Unity.Collections.LowLevel.Unsafe;
using Unity.Mathematics;
using Unity.NetCode;
using Unity.Transforms;

[BurstCompile]
public static unsafe class CustomSmoothing
{
    public static readonly PortableFunctionPointer<
        GhostPredictionSmoothing.SmoothingActionDelegate> Action =
        new(SmoothingAction);

    [BurstCompile(DisableDirectCall = true)]
    private static void SmoothingAction(
        void* currentData, void* previousData, void* userData)
    {
        ref var current = ref UnsafeUtility.AsRef<LocalTransform>(currentData);
    }
}

```

```
ref var backup = ref UnsafeUtility.AsRef<LocalTransform>(previousData);

var dist = math.distance(current.Position, backup.Position);
if (dist > 0)
{
    current.Position = math.lerp(backup.Position, current.Position, 0.5f);
}
}
```

7. 预测切换

预测开销不小，所以 Ghost 可以在 Predicted 和 Interpolated 之间动态切换。

7.1 切换先决条件

- Supported Ghost Modes 必须是 All。
- 当前模式不能是 OwnerPredicted。
- 该 Ghost 不能正在切换中。

7.2 排队切换

```
using Unity.NetCode;

ref var switchQueues = ref SystemAPI
    .GetSingletonRW<GhostPredictionSwitchingQueues>()
    .ValueRW;

switchQueues.ConvertToPredictedQueue.Enqueue(new ConvertPredictionEntry
{
    TargetEntity = entity,
    TransitionDurationSeconds = 1f,
});

switchQueues.ConvertToInterpolatedQueue.Enqueue(new ConvertPredictionEntry
{
    TargetEntity = entity,
    TransitionDurationSeconds = 0f,
});
```

插值 Ghost 和预测 Ghost 跑在不同的时间线上，两者通常相差约两个 Ping。

SwitchPredictionSmoothing 会在你设定的时长内，对位置和旋转做插值，以免出现肉眼可见的传送跳变。

8. 预测性能控制

技术	效果
预测切换	把远处或重要性低的 Ghost 切到插值模式。
限制 GhostField 写访问	预测循环里不会改的字段，用只读访问。
物理批处理	用 MaxPredictionStepBatchSizeRepeatedTick 来批处理重复的物理重新模拟。
Tick 频率调整	调低 SimulationTickRate 能减少重新模拟的次数，代价是响应变迟钝。

9. 相关文档

- 20_Netcode Command Stream & Input.md——预测 System 所用的输入数据。
- 22_Netcode Ghost Optimization · Importance · Relevancy.md——把预测切换当作一种优化手段。
- 23_Netcode Physics Integration & Lag Compensation.md——预测下的物理行为。

10. 故障排查

症状	原因 / 解决方法
预测没运行	查一下 Ghost 模式是不是 Predicted，预测 Query 里有没有 Simulate。
抖动严重	注册一个预测平滑，并确认 Quantization 跟游戏需求相称。
服务器校正时 Entity 出现传送跳变	设置 MaxSmoothingDistance，或加一个自定义平滑。
预测的 CPU 开销过高	用预测切换、调低 Tick 频率，并在合适的地方批处理物理重新模拟。
Entity 每个 Tick 模拟了不止一次	预测 Query 漏了 .WithAll<Simulate>()。

Chapter 23. Netcode 命令流与输入

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

1. 概述

客户端一进入游戏状态，就会不停地向服务器发送**命令流**。命令流里既有输入数据，也有快照 ACK 数据；哪怕玩家没在按任何键，这条流也照样持续，好让连接保持同步。

2. ICommandData 与 IInputComponentData

	ICommandData	IInputComponentData
用途	偏底层的命令控制。	偏高层的输入工作流。
Buffer 管理	手动，靠 AddCommandData。	自动，由生成的代码管理。
Tick 映射	手动，靠 GetDataAtTick。	自动；会直接提供当前 Tick 的输入。
InputEvent	不支持。	支持，可处理一次性事件。
最大负载	1024 字节	1024 字节
最佳适用	自定义的命令管线。	常规的玩家输入。

3. IInputComponentData

3.1 定义输入

```
using Unity.NetCode;

public struct PlayerInput : IInputComponentData
{
    public int Horizontal;
    public int Vertical;
    public InputEvent Jump;
}
```

InputEvent 会把 GetKeyDown 这类一次性事件保留下来，这样即便冗余的命令包丢了一个，该事件在服务器上也恰好只触发一次。

3.2 在客户端收集输入

```
using Unity.Entities;
using Unity.NetCode;
using UnityEngine;

[UpdateInGroup(typeof(GhostInputSystemGroup))]
public partial struct GatherInputSystem : ISystem
{
    public void OnCreate(ref SystemState state)
    {
        state.RequireForUpdate<PlayerInput>();
    }

    public void OnUpdate(ref SystemState state)
    {
        bool jump = Input.GetKeyDown(KeyCode.Space);
        bool left = Input.GetKey(KeyCode.A);
        bool right = Input.GetKey(KeyCode.D);

        foreach (var input in
            SystemAPI.Query<RefRW<PlayerInput>>()
                .WithAll<GhostOwnerIsLocal>())
        {
            input.ValueRW = default;
            if (jump) input.ValueRW.Jump.Set();
            if (left) input.ValueRW.Horizontal -= 1;
            if (right) input.ValueRW.Horizontal += 1;
        }
    }
}
```

3.3 在服务器和预测循环中处理输入

```
using Unity.Burst;
using Unity.Entities;
using Unity.Mathematics;
using Unity.NetCode;
using Unity.Transforms;

[UpdateInGroup(typeof(PredictedSimulationSystemGroup))]
public partial struct ProcessInputSystem : ISystem
{
    public void OnCreate(ref SystemState state)
    {
        state.RequireForUpdate<PlayerInput>();
    }

    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        var speed = 3f;
        var dt = SystemAPI.Time.DeltaTime;

        foreach (var (input, transform) in
            SystemAPI.Query<RefRO<PlayerInput>, RefRW<LocalTransform>>()
                .WithAll<Simulate>())
        {
            // ...
        }
    }
}
```

```

        {
            if (input.ValueR0.Jump.IsSet)
            {
                // Jump logic; runs once on the input tick.
            }

            transform.ValueRW.Position +=
                new float3(input.ValueR0.Horizontal, 0, 0) * speed * dt;
        }
    }
}

```

预测循环在重新模拟时,可能把这个 System 跑很多遍。而 `InputEvent.IsSet` 只在对应的那个 Tick 上才为 true。

4. ICommandData

4.1 定义命令

```

using Unity.Mathematics;
using Unity.NetCode;

public struct MyCommand : ICommandData
{
    public NetworkTick Tick { get; set; }
    public int Action;
    public float2 Direction;
}

```

4.2 添加和读取命令数据

```

using Unity.Entities;
using Unity.NetCode;

DynamicBuffer<MyCommand> buffer = default;
NetworkTick networkTick = default;

buffer.AddCommandData(new MyCommand
{
    Tick = networkTick,
    Action = 1,
    Direction = new float2(1, 0),
});

if (buffer.GetDataAtTick(networkTick, out var cmd))
{
    // Use cmd.Action and cmd.Direction.
}

```

5. 命令发送机制

```
Command packet
├─ Tick, Command
├─ Delta(Tick - 1)
├─ Delta(Tick - 2)
└─ Delta(Tick - 3)
```

The last three inputs are redundantly sent with delta compression.
Maximum payload: 1024 bytes.

CommandSendPacketSystem 按 SimulationTickRate 的频率刷新；CommandReceiveSystem 则在服务器上接收命令数据，再转发给目标 Entity。

6. AutoCommandTarget

只要 Ghost 带了 AutoCommandTarget，命令就会自动发送。

要求	含义
Ghost 已启用所有者	在 GhostAuthoringComponent 上勾选。
已启用 Support Auto Command Target	在 GhostAuthoringComponent 上勾选。
客户端是所有者	服务器把 GhostOwner.NetworkId 设成该客户端的 NetworkId.Value。
Ghost 为 Predicted 或 OwnerPredicted	命令数据是为预测准备的。
AutoCommandTarget.Enabled 为 true	运行时可以禁用。

不用 AutoCommandTarget 时，就得手动在连接 Entity 上设置 CommandTarget。

```
using Unity.Entities;
using Unity.NetCode;

Entity connectionEntity = default;
Entity playerEntity = default;

state.EntityManager.SetComponentData(connectionEntity,
    new CommandTarget { targetEntity = playerEntity });
```

7. 识别本地归属的 Entity

7.1 GhostOwnerIsLocal

```
using Unity.Entities;
using Unity.NetCode;
using Unity.Transforms;

foreach (var (input, transform) in
    SystemAPI.Query<RefRW<PlayerInput>, RefRW<LocalTransform>>()
        .WithAll<GhostOwnerIsLocal>())
{
    // Local-owned entity only.
}
```

7.2 手动检查所有者

```
using Unity.Entities;
using Unity.NetCode;
using Unity.Transforms;

var localId = SystemAPI.GetSingleton<NetworkId>().Value;

foreach (var (owner, transform) in
    SystemAPI.Query<RefRO<GhostOwner>, RefRW<LocalTransform>>())
{
    if (owner.ValueRO.NetworkId == localId)
    {
        // Local-owned entity.
    }
}
```

8. 强制输入延迟

`ClientTickRate.ForcedInputLatencyTicks` 会人为加上一段输入延迟。

设置	含义
ForcedInputLatencyTicks	额外的输入延迟，以 Tick 计；默认 0。
用途	在格斗游戏这类品类里，让所有玩家承受相同的输入延迟。
效果	会自动计入 <code>MaxPredictAheadTimeMS</code> 的计算。

8.1 InputTargetTick 与 ServerTick

开启强制输入延迟后，输入 System 里应该用 `NetworkTime.InputTargetTick`，而不是 `NetworkTime.ServerTick`。

```
using Unity.NetCode;

var inputTick = SystemAPI.GetSingleton<NetworkTime>().InputTargetTick;
var serverTick = SystemAPI.GetSingleton<NetworkTime>().ServerTick;
```


ForcedInputLatencyTicks 一旦不为零，InputTargetTick 和 ServerTick 就会不一样。输入应当对准 InputTargetTick。

9. 相关文档

- 19_Netcode Prediction & Rollback.md——会消费输入的预测 System。
- 17_Netcode Network Connection & Approval.md——NetworkStreamInGame 与连接 Entity。
- 21_Netcode RPC.md——可靠的一次性消息，而非连续的输入。

10. 故障排查

症状	原因 / 解决方法
服务器收不到输入	加上 NetworkStreamInGame、设置 CommandTarget，或者满足 AutoCommandTarget 的那几项要求。
跳跃触发了不止一次	用 InputEvent.Set() / .IsSet，别用普通的 bool。
报负载大小错误	把命令 struct 控制在 1024 字节以内。
预测过程中输入丢失	优先用 IInputComponentData 的自动 Tick 映射，或者检查一下 GetDataAtTick。
服务器代码跑进了 GhostInputSystemGroup	这个 Group 只属于客户端；服务器逻辑该放进 PredictedSimulationSystemGroup。

Chapter 24. Netcode RPC

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

1. 概述

RPC 是客户端和服务端之间的一种**可靠的一次性消息**。它的序列化和反序列化代码会自动生成，RPC 的处理也照样融入 ECS / Job System 的帧循环。

RPC 替代不了 Ghost 复制或命令流。它该用于零散的游戏流程事件，而不是高频的状态同步。

2. RPC、Ghost 与命令的对比

	RPC	Ghost	命令
传输行为	可靠、有序。	不可靠，但带复制优化。	不可靠，但带冗余发送。
方向	双向	服务器 → 客户端	客户端 → 服务器
频率	一次性	每个网络 Tick 按需发送	每个模拟 Tick 都发
用途	游戏流程、大厅、聊天	需复制的 Entity 状态	玩家输入
在途限制	有，受可靠管线限制	无	无

正因为 RPC 既有序又可靠，频繁的 RPC 流量可能会卡在在途数据包后面排队。

3. 定义 RPC

3.1 空 RPC

```
using Unity.NetCode;

public struct PingRpc : IRpcCommand
{
}
```

3.2 带数据的 RPC

```

using Unity.Collections;
using Unity.NetCode;

public struct ChatMessageRpc : IRpcCommand
{
    public FixedString128Bytes Message;
    public int SenderId;
}

```

4. 发送 RPC

4.1 客户端向服务器

```

using Unity.Collections;
using Unity.Entities;
using Unity.NetCode;
using UnityEngine;

[WorldSystemFilter(WorldSystemFilterFlags.ClientSimulation)]
public partial struct SendChatSystem : ISystem
{
    public void OnCreate(ref SystemState state)
    {
        state.RequireForUpdate<NetworkId>();
    }

    public void OnUpdate(ref SystemState state)
    {
        if (!Input.GetKeyDown(KeyCode.Space))
            return;

        var ecb = new EntityCommandBuffer(Allocator.Temp);
        var entity = ecb.CreateEntity();

        ecb.AddComponent(entity, new ChatMessageRpc
        {
            Message = "Hello!",
            SenderId = SystemAPI.GetSingleton<NetworkId>().Value,
        });

        ecb.AddComponent<SendRpcCommandRequest>(entity);
        ecb.Playback(state.EntityManager);
    }
}

```

ISystem 是个 struct。别把跨帧的状态存在 System 的实例字段里；状态应放进 Component 或 Singleton。

4.2 服务器向单个客户端

```

using Unity.Entities;
using Unity.NetCode;

Entity targetConnectionEntity = default;

var entity = ecb.CreateEntity();
ecb.AddComponent(entity, new NotifyRpc());
ecb.AddComponent(entity, new SendRpcCommandRequest
{
    TargetConnection = targetConnectionEntity,
});

```

4.3 服务器广播

```

using Unity.Entities;
using Unity.NetCode;

var entity = ecb.CreateEntity();
ecb.AddComponent(entity, new GameStartRpc());
ecb.AddComponent(entity, new SendRpcCommandRequest
{
    TargetConnection = Entity.Null,
});

```

填 `Entity.Null` 即表示广播这条 RPC。

5. 接收 RPC

```

using Unity.Collections;
using Unity.Entities;
using Unity.NetCode;
using UnityEngine;

[WorldSystemFilter(WorldSystemFilterFlags.ServerSimulation)]
public partial struct ReceiveChatSystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {
        var ecb = new EntityCommandBuffer(Allocator.Temp);

        foreach (var (req, chat, entity) in
            SystemAPI.Query<RefRO<ReceiveRpcCommandRequest>, RefRO<ChatMessageRpc>>()
                .WithEntityAccess())
        {
            var sourceConnection = req.ValueRO.SourceConnection;
            Debug.Log($"Player {chat.ValueRO.SenderId}: {chat.ValueRO.Message}");

            ecb.DestroyEntity(entity);
        }

        ecb.Playback(state.EntityManager);
    }
}

```

```
}  
}
```

处理完之后，务必销毁收到的那个 RPC Entity。不然同一条 RPC 会每帧都被处理一遍。

6. RpcQueue

只有在需要手动调度 RPC 时，才用 RpcQueue。

```
using Unity.Entities;  
using Unity.NetCode;  
  
var rpcQueue = SystemAPI.GetSingleton<RpcCollection>()  
    .GetRpcQueue<MyRpc, MyRpc>();  
var bufferLookup = SystemAPI.GetBufferLookup<OutgoingRpcDataStreamBuffer>();  
  
if (bufferLookup.HasBuffer(connectionEntity))  
{  
    var buffer = bufferLookup[connectionEntity];  
    rpcQueue.Schedule(buffer, new MyRpc());  
}
```

7. 手动序列化

默认就是自动生成。一旦关掉它，序列化和反序列化就必须严格对称。

```
using Unity.Burst;  
using Unity.Entities;  
using Unity.NetCode;  
using Unity.Networking.Transport;  
  
[BurstCompile]  
public struct MyCustomRpc : IComponentData, IRpcCommandSerializer<MyCustomRpc>  
{  
    public int Data;  
  
    public void Serialize(ref DataStreamWriter writer, in RpcSerializerState state,  
        in MyCustomRpc data)  
    {  
        writer.WriteInt(data.Data);  
    }  
  
    public void Deserialize(ref DataStreamReader reader, in RpcDeserializerState state,  
        ref MyCustomRpc data)  
    {  
        data.Data = reader.ReadInt();  
    }  
}
```

```
// Execute implementation omitted.  
}
```

8. IApprovalRpcCommand

IApprovalRpcCommand 是一种特殊 RPC，只在连接审批阶段使用。在握手/审批状态下，普通的 IRpcCommand 消息会被拦下。

```
using Unity.Collections;  
using Unity.NetCode;  
  
public struct LoginApproval : IApprovalRpcCommand  
{  
    public FixedString512Bytes AuthToken;  
}
```

完整的审批流程见 17_Netcode Network Connection & Approval.md。

9. 常见模式

模式	示例 RPC
大厅 / 匹配	JoinLobbyRpc、ReadyRpc
游戏流程	GameStartRpc、RoundEndRpc
聊天	ChatMessageRpc
物品交易	TradeRequestRpc、TradeAcceptRpc
连接审批	各种 IApprovalRpcCommand 实现

10. 相关文档

- 18_Netcode Ghost Snapshot & Synchronization.md——用 Ghost 复制状态。
- 20_Netcode Command Stream & Input.md——用命令流传输入。
- 17_Netcode Network Connection & Approval.md——审批 RPC。

11. 故障排查

症状	原因 / 解决方法
RPC 没收到	确认 SendRpcCommandRequest 已加上，并且 System 跑在目标 World 里。
RPC 每帧都被处理一次	处理完之后，把收到的 RPC Entity 销毁掉。
RPC 似乎到得有延迟	可靠有序的传输可能卡在在途数据包后面排队。高频数据改用 Ghost 或命令。
RPC 在审批阶段被拦下	用 IApprovalRpcCommand，别用 IRpcCommand。
手动序列化出错	确保 Serialize 和 Deserialize 按同样的顺序读写字段。

Chapter 25. Netcode Ghost 优化 · 重要性 · 相关性

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

1. 概述

大型多人游戏世界，不可能每个 Tick 把所有 Ghost 发给每个客户端。为此 Netcode for Entities 用上了**优化模式**、**重要性缩放**、**相关性**，以及逐 Ghost 的发送速率控制，来压低 CPU 开销和带宽占用。

核心原则很简单：服务器要把快照预算花在那些此刻对某条连接真正要紧的 Ghost 上。

2. 优化模式

优化模式在 GhostAuthoringComponent 上设置。

模式	含义	适合场景
Dynamic	默认值。无论数据变没变，都会优化快照大小。	会移动的普通 Ghost。
Static	Ghost 没变化时，就不重发数据。	建筑、地形、静态道具。

1.11.0 版本线新增了一项静态 Ghost 优化，能大幅降低未变化的静态 Ghost 在 GhostUpdateSystem 上的耗时。

3. 快照大小限制

快照预算可以在好几个层级上加以限制。

设置	含义
NetworkStreamSnapshotTargetSize	每条连接的软性字节目标。
GhostSendSystemData.MaxSendEntities	每个快照最多容纳的 Entity 数。
GhostSendSystemData.MaxSendChunks	每个快照最多容纳的 Chunk 数。

预算吃紧时，重要性更高的 Ghost 会被优先发送。

4. 重要性缩放

重要性缩放会在服务器上算出每个 Ghost 的发送优先级。

4.1 输入

GhostImportance 综合了三类数据。

数据	含义
逐连接数据	与某条连接相关的数据，比如玩家位置。
Singleton 数据	全局设置，比如瓦片大小。
逐 Chunk 的共享数据	Chunk 级别的缩放参数。

4.2 基于距离的重要性

```
using Unity.Entities;
using Unity.Mathematics;
using Unity.NetCode;

var gridSingleton = state.EntityManager.CreateSingleton(
    new GhostDistanceData
    {
        TileSize = new int3(50, 50, 256),
        TileCenter = new int3(0, 0, 128),
        TileBorderWidth = new float3(1f, 1f, 1f),
    });

state.EntityManager.AddComponentData(gridSingleton, new GhostImportance
{
    ScaleImportanceFunction = GhostDistanceImportance.ScaleFunctionPointer,
    GhostConnectionComponentType = ComponentType.ReadOnly<GhostConnectionPosition>(),
    GhostImportanceDataType = ComponentType.ReadOnly<GhostDistanceData>(),
    GhostImportancePerChunkDataType = ComponentType.ReadOnly<GhostDistancePartitionShared>(),
});
```

1.8.0 版本线带来的一项破坏性变更：GhostDistanceImportance 的缩放函数不再把 baseImportance 乘以 1000。凡是按旧乘数写的自定义重要性代码，都得检查一遍。

4.3 自动 GhostDistancePartitionShared

```
using Unity.NetCode;

GhostDistancePartitioningSystem.AutomaticallyAddGhostDistancePartitionSharedComponent = true;
```

4.4 重要性可视化工具

从 1.8.0+ 版本线起，PlayMode Tool 内置了重要性可视化工具。用它来查看运行时的重要性分布，别再对着代码瞎猜。

5. Ghost 相关性

Ghost 相关性让服务器可以按连接来隐藏或放出特定的 Ghost。

模式	含义
Disabled	默认值。不做过滤，所有 Ghost 都是候选。
SetIsRelevant	只有列出的 Ghost 才复制给该连接。
SetIsIrrelevant	列出的 Ghost 不发给该连接。

5.1 使用场景

场景	实现思路
距离剔除	把远处的 Ghost 标记为不相关。
战争迷雾	把视野之外的 Entity 隐藏起来。
专属于某客户端的 Ghost	只对一个客户端把某 Ghost 标记为相关。
队伍过滤	把只该敌方知道的内部数据藏起来。

5.2 相关性示例

```
using Unity.Entities;
using Unity.Mathematics;
using Unity.NetCode;
using Unity.Transforms;

[WorldSystemFilter(WorldSystemFilterFlags.ServerSimulation)]
[UpdateInGroup(typeof(GhostSimulationSystemGroup))]
[UpdateBefore(typeof(GhostSendSystem))]
public partial struct RelevancySystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {
        var relevancy = SystemAPI.GetSingletonRW<GhostRelevancy>();

        relevancy.ValueRW.GhostRelevancyMode = GhostRelevancyMode.SetIsRelevant;
        relevancy.ValueRW.GhostRelevancySet.Clear();

        foreach (var (ghostInstance, transform) in
            SystemAPI.Query<RefRO<GhostInstance>, RefRO<LocalTransform>>())
        {
            foreach (var (connId, connPos) in
                SystemAPI.Query<RefRO<NetworkId>, RefRO<GhostConnectionPosition>>())
```

```

    {
        float dist = math.distance(
            transform.ValueRO.Position,
            connPos.ValueRO.Position);

        if (dist < 100f)
        {
            relevancy.ValueRW.GhostRelevancySet.TryAdd(
                new RelevantGhostForConnection(
                    connId.ValueRO.Value,
                    ghostInstance.ValueRO.ghostId),
                1);
        }
    }
}
}
}
}

```

这个简化示例的时间复杂度是 $O(\text{Ghost} \times \text{连接数})$ 。对大型世界，优先用基于距离的重要性分区，把相关性留给分区表达不了的规则，比如战争迷雾或队伍可见性。

5.3 DefaultRelevancyQuery

DefaultRelevancyQuery 用来指定那些对所有连接始终相关的 Ghost Chunk，比如全局游戏状态 Ghost，或与 UI 有关的网络状态。

5.4 相关性 + 重要性

从 1.6.1+ 版本线起，可以通过 `PrioChunks.isRelevant` 把 Ghost 相关性和重要性缩放结合起来，从而走上一条更快的相关性路径。

6. 预测 CPU 优化

6.1 预测切换

把远处或不重要的 Ghost 从预测切到插值。

```

using Unity.Entities;
using Unity.Mathematics;
using Unity.NetCode;

float distance = math.distance(playerPos, ghostPos);
if (distance > predictionRange)
{
    switchQueues.ConvertToInterpolatedQueue.Enqueue(new ConvertPredictionEntry
    {
        TargetEntity = ghostEntity,
        TransitionDurationSeconds = 0.5f,
    });
}

```

```
});  
}
```

6.2 最小化 GhostField 写访问

对 GhostField Component 拥有写权限的 Job，可能会触发序列化检查。预测循环里不会改的字段，就用只读访问。

6.3 物理重新模拟

遇到要重新模拟 20 多帧的高延迟情况：

- 当放回主线程汇总反而更省时，就通过 PhysicsStep Singleton 关掉多线程物理。
- 用 MaxPredictionStepBatchSizeRepeatedTick 来批处理重复的物理 Tick。

7. MaxIterateChunks

GhostSendSystemData.MaxIterateChunks 限定单个 Tick 内处理多少个 Chunk。

设置	含义
MaxIterateChunks	限制每个 Tick 的序列化 CPU 开销。
MaxSendChunks	通过给快照里的 Chunk 数量设上限来控制带宽。

这两个设置针对的是不同的问题。MaxIterateChunks 用来摊开 CPU 工作量；MaxSendChunks 用来约束快照大小。

8. MaxSendRate

GhostAuthoringComponent.MaxSendRate 以 Hz 为单位限制某个 Ghost 的发送频率。

```
NetworkTickRate = 60, MaxSendRate = 10  
→ This Ghost can be included once every 6 network ticks.
```

低频或低重要性的 Ghost 适合用它。

9. UseSingleBaseline

GhostAuthoringComponent.UseSingleBaseline = true 会用单个基线来做增量压缩。这能降低 CPU 开销，但带宽消耗可能上升。

10. 优化检查清单

- ☐ 极少变化的 Ghost 用 **Static** Optimization Mode。
- ☐ 设置 NetworkStreamSnapshotTargetSize，给每条连接的带宽设上限。
- ☐ 实现基于距离的 Importance Scaling。
- ☐ 只在分区规则表达不了的场合才用 Relevancy。
- ☐ 把远处的 Ghost 切到 Interpolated 模式。
- ☐ 低频 Ghost 用 MaxSendRate。
- ☐ 别去写访问那些没变化的 GhostField 数据。
- ☐ 用 Importance Visualizer 查看运行时优先级。

11. 相关文档

- 18_Netcode Ghost Snapshot & Synchronization.md——Ghost 序列化与快照流程。
- 19_Netcode Prediction & Rollback.md——预测切换的行为。
- 24_Netcode Profiler & Debugging.md——分析带宽和预测开销。

12. 故障排查

现象	原因 / 解决方案
Ghost 迟迟才出现	重要性太低，或者快照预算太紧。
启用 Importance Scaling 后 Ghost 不再发送	确认 GhostDistancePartitionShared 已经存在，或者打开自动添加的选项。
启用 Relevancy 后 Ghost 闪烁	检查边界条件，并确保每帧都用一致的方式重建 relevancy 集合。
带宽够用，Ghost 还是缺失	查一下 MaxSendEntities 和 MaxSendChunks 的限制。

Chapter 26. Netcode Physics Integration & Lag Compensation

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

1. 概述

Netcode for Entities 与 **Unity Physics** 集成，用于联网的物理模拟。物理表现如何，取决于 Ghost 是 Interpolated 还是 Predicted。

场景里至少得有一个 Predicted Ghost，预测物理才会运行。

2. Interpolated 与 Predicted 物理

2.1 客户端上的 Interpolated Ghosts

- 位置和旋转来自服务器快照。
- Simulate 被禁用，物理对象表现得像运动学物体。
- PhysicsVelocity 会被忽略。

2.2 客户端上的 Predicted Ghosts

- 物理在预测循环里运行，每渲染一帧可能执行多次。
- PhysicsSystemGroup 在 PredictedFixedStepSimulationSystemGroup 内部运行。
- 对预测物理而言，客户端应得出与服务器一致的模拟结果。

Netcode initialization

```
└ PhysicsSystemGroup moves under PredictedFixedStepSimulationSystemGroup
└ FixedStepSimulationSystemGroup moves under PredictedFixedStepSimulationSystemGroup
```

3. 预测物理性能

预测物理的开销不小，因为回滚可能让物理反复执行。

3.1 服务器批处理

```
using Unity.NetCode;

ClientServerTickRate tickRate = default;
tickRate.MaxSimulationStepBatchSize = 4;
tickRate.MaxSimulationStepsPerFrame = 4;
```

3.2 客户端批处理

```
using Unity.NetCode;

ClientTickRate clientRate = default;
clientRate.MaxPredictionStepBatchSizeFirstTimeTick = 2;
clientRate.MaxPredictionStepBatchSizeRepeatedTick = 4;
```

批处理会加大误预测的风险，所以把它当成一种性能上的取舍，别默认就开。

3.3 量化

物理 Ghost 的 Transform / Velocity，默认量化精度是 1000。

更高量化精度	更低量化精度
模拟更精确	带宽更省
带宽占用更高	精度损失更大

4. 延迟补偿

由于客户端的预测领先服务器，权威服务器有时得去查一个与客户端操作时刻对应的旧碰撞世界。

4.1 启用延迟补偿

给 SubScene 加上 NetCodePhysicsConfig，并设置 EnableLagCompensation = true。

4.2 查询旧碰撞世界

```
using Unity.NetCode;
using Unity.Physics;

var physicsHistory = SystemAPI.GetSingleton<PhysicsWorldHistorySingleton>();
```

```
physicsHistory.GetCollisionWorldFromTick(
    serverTick,
    interpolationDelay,
    ref physicsWorld,
    out var collisionWorld);

// Use collisionWorld for raycasts or other physics queries.
```

4.3 Collider 深度克隆

设置	默认值	含义
DeepCopyDynamicColliders	true	深度克隆动态 collider；这是 6.5.0 的默认行为。
DeepCopyStaticColliders	false	深度克隆静态 collider。

6.5.0 版本默认开启动态 Ghost collider 的深度克隆，以避免 blob asset 断言错误。

4.4 物理历史缓冲区

版本	历史大小
1.4.x 及以前	16 个 Tick
1.5.0+	32 个 Tick
自定义	用编译器宏指定 GhostSystemConstants.SnapshotHistorySize

5. 多物理世界

对于不参与联网模拟的客户端专属效果（比如 VFX、粒子），用一个独立的物理世界。

5.1 设置

1. 给 SubScene 加上 NetCodePhysicsConfig。
2. 把 **Client Non Ghost World** 设为一个非零值，比如 1。
3. 给那些只在客户端的物理对象加上 PhysicsWorldIndex，值设成同一个数。

5.2 自定义物理 System 组

```
using Unity.Entities;
using Unity.NetCode;
using Unity.Physics.Systems;

[WorldSystemFilter(WorldSystemFilterFlags.ClientSimulation)]
```



```
[UpdateInGroup(typeof(FixedStepSimulationSystemGroup))]  
public partial class VfxPhysicsGroup : CustomPhysicsSystemGroup  
{  
    public const int WorldIndex = 1;  
  
    public VfxPhysicsGroup() : base(WorldIndex, shareStaticColliders: true)  
    {  
    }  
}
```

shareStaticColliders: true 让自定义物理世界能与主物理世界共用静态 collider。

6. Physics Proxy

当 Predicted Ghost 需要和只存在于客户端的物理实体交互时，就用 Physics Proxy。

- CustomPhysicsProxyAuthoring 负责创建 proxy entity。
- CustomPhysicsProxyDriver 把 proxy 和 Predicted Ghost 关联起来。
- SyncCustomPhysicsProxySystem 负责同步位置和旋转。

7. PredictedFixedStepSimulationSystemGroup

只有所有必要条件都满足时，PredictedFixedStepSimulationSystemGroup 才会更新。

条件	含义
NetworkStreamInGame singleton 存在	连接已经进入游戏。
至少存在一个 Predicted Ghost	预测物理有东西可模拟。
固定 Tick 率正在运行	固定时间步长的模拟处于活动状态。

在网络连接进入游戏之前、或 Predicted Ghost 流入之前，预测物理不会运行。

7.1 连接前的物理

如果物理必须在网络连接建立之前就运行，那就关掉预测物理的相关设置，好让常规的 FixedStepSimulationSystemGroup 物理跑起来。

```
using Unity.Entities;  
using Unity.NetCode;  
  
[UpdateInGroup(typeof(InitializationSystemGroup))]  
[CreateAfter(typeof(PredictedPhysicsConfigSystem))]  
public partial struct DisablePredictedPhysics : ISystem
```

```
{
    public void OnCreate(ref SystemState state)
    {
        state.World.GetExistingSystemManaged<PredictedPhysicsConfigSystem>()
            .Enabled = false;
    }
}
```

8. 限制

限制	含义
不支持部分 tick	物理不走部分 Tick；改用 Physics Interpolation。
多世界调试受限	Unity Physics 的调试系统对多世界设置支持得不完整。
最少 Predicted Ghost 要求	预测物理要运行，至少得有一个 Predicted Ghost。

9. 相关文档

- 19_Netcode Prediction & Rollback.md——预测循环的行为。
- 22_Netcode Ghost Optimization · Importance · Relevancy.md——把 Ghost 切出预测模式。
- 24_Netcode Profiler & Debugging.md——分析预测物理的开销。

10. 故障排查

现象	原因 / 解决方案
物理不运行	查一下 Predicted Ghost 和 NetworkStreamInGame 在不在。
物理抖动	检查量化设置，以及预测平滑。
延迟补偿的射线检测失败	开启 EnableLagCompensation，并确认历史缓冲区的覆盖范围够。
Blob asset 断言错误	检查 collider 深度克隆设置；6.5.0 里动态克隆默认就开着。
客户端专属的物理不运行	把 PhysicsWorldIndex 设成跟 Client Non Ghost World 一样的值。
预测物理的 CPU 开销过高	开启批处理，并把没必要预测的 Ghost 切到 Interpolated。

Chapter 27. Netcode Profiler & Debugging

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

1. 概述

Netcode for Entities 提供了一个 **Netcode Profiler**，用来分析连接状态、快照、预测、插值和带宽。从 1.12.0 版本起，基于浏览器的 Network Debugger 已被弃用，改用内置的 Unity Profiler 模块。

2. Netcode Profiler

2.1 访问方式

在 Unity 6.0+ 上，Unity Profiler 带有两个 Netcode 模块。

模块	检查内容
Server Profiler	服务器端的 Ghost 发送、快照构成和带宽。
Client Profiler	客户端的接收、预测 Tick 和插值状态。

2.2 主要功能

功能	含义
Ghost Snapshot View	逐 Ghost 展示快照大小与构成的树形视图。
Tick 导航	在收发过快照的那些帧之间跳转。
搜索字段	搜索 Ghost Snapshot TreeView 和 Prediction Error List。
Client ticks	把 Prediction Tick 和 Interpolation Tick 可视化呈现。
右键菜单	在 6.5.0 版本里，可以从 TreeView 标签直接查看 Ghost Prefab 和 Component。
逐实体平均列	显示每个 entity 的平均数据大小。

2.3 NETCODE_PROFILER_ENABLED

1.12.0 版本不再需要 NETCODE_PROFILER_ENABLED 这个脚本宏。没有它也能用 Profiler。

3. Network Debugger

基于浏览器的 Network Debugger 在 1.12.0 版本中已弃用，请改用 Netcode Profiler。

旧的流程是点 **Multplayer** → **Open NetDbg**，它会打开一个基于浏览器的视图，用来查看快照构成、Ghost 类型分布和大小分析。

4. Multiplayer PlayMode 窗口

Multiplayer PlayMode 窗口是在 Play 模式下测试客户端/服务器配置的主要编辑器工作流。

功能	含义
Play Mode Type	选 Client + Server、Client Only 或 Server Only。
Thin Client 数量	配置虚拟客户端数量。
Simulator	模拟网络延迟与丢包。
Importance Visualizer	1.8.0+ 版本可把 Importance Scaling 可视化。

5. 调试检查清单

5.1 连接

- ☐ Application.runInBackground = true
- ☐ Protocol versions match between client and server
- ☐ NetworkStreamInGame is present
- ☐ Firewall and port are open
- ☐ Unity Transport version is compatible

5.2 Ghost 同步

- ☐ GhostAuthoringComponent is present
- ☐ GhostField attributes are correct
- ☐ Quantization is appropriate
- ☐ Smoothing is configured
- ☐ Ghost prefab is registered on both sides

5.3 预测

- ☐ Systems run in PredictedSimulationSystemGroup
- ☐ Queries include Simulate
- ☐ Simulation code is deterministic enough for prediction

- Client and server run the same simulation code
- Prediction smoothing is registered where needed

5.4 输入与命令

- Input is gathered in GhostInputSystemGroup
- AutoCommandTarget requirements are met, or CommandTarget is set manually
- Command payload is 1024 bytes or less
- One-shot input uses InputEvent

6. Prefab List 菜单

1.11.0+ 版本带有一个 Prefab List 菜单，能快速列出并查看项目或场景层级里已注册的 Ghost Prefab。

7. 常用调试组件

组件	用途
NetworkStreamConnection	查看连接状态。
NetworkId	查看客户端 ID。
GhostInstance	查看 Ghost 的类型和 ID。
PredictedGhost	查看预测状态和已应用的 Tick。
NetworkSnapshotAck	查看快照 ACK 状态和估算的 RTT。
GhostOwner	查看 Ghost 的所有者 ID。

7.1 Entity Inspector

在 Entities Hierarchy 里选中一个 Ghost entity，就能查看：

- 实时的 GhostField 值，
- 预测 Tick 数据，
- 网络连接状态。

8. 数据包转储

需要做底层网络数据包分析时，就用数据包转储。从 1.6.2 版本起，数据包的时间戳日志里多了 `UnityEngine.Time.frameCount`，按帧对照起来更方便。

9. 常见瓶颈

区域	排查位置
快照带宽	Profiler 的 Ghost Snapshot View；看逐 entity 的大小。
预测重新模拟	Profiler 的 Client Tick；看预测 Tick 的数量。
GhostSendSystem	Profiler 的 Timeline；看 System 的耗时。
序列化开销	检查 GhostField 的写访问和变化频率。
物理重新模拟	看 PredictedFixedStepSimulationSystemGroup 的耗时。

10. Editor 分析

1.12.0 版本会收集 Netcode Profiler 使用情况的编辑器分析数据。

11. 相关文档

- Optimizations and Debugging/03_Profiler · Bottleneck Analysis.md——通用的 DOTS 性能分析 workflow。
 - 22_Netcode Ghost Optimization · Importance · Relevancy.md——带宽控制。
 - 23_Netcode Physics Integration & Lag Compensation.md——预测物理的开销。
-

12. 故障排查

现象	原因 / 解决方案
Netcode Profiler 没数据	确认测试跑在客户端/服务器 Play Mode 下，并且用的是 Unity 6.0+。
Ghost Snapshot View 数值偏大，但游戏体验没问题	找一找那些字段过多、或发送率过高的低优先级 Ghost。

现象	原因 / 解决方案
预测误差激增	检查 Simulate 查询、确定性的代码路径，以及输入 Tick 的处理。
找不到 Network Debugger 这套流程了	它已经弃用，请改用 Unity Profiler 的 Netcode 模块。
数据包级别的日志难以对照	在 1.6.2+ 版本里，用带帧数时间戳的数据包转储。

Part III. Optimization and Debugging

Chapter 28. Chunk Layout & TypeManager

1. 16 KB 的 Chunk

每个 Archetype 都把自己的 Entity 存放在 **16 KB 的 Chunk** 里。一个 Chunk 包含：

- 1. 一个小小的头部（元数据——Archetype id、各种版本号、启用位掩码、变更版本）。
- 2. 一段连续的 **SoA** 区域：每种 Component 占一列，每列是一个值数组。
- 3. 补齐到 16 KB 边界的填充字节。

Chunk 容量——也就是能装多少个 Entity——可由下面这个式子算出：

```
capacity ≈ (16 KB - header) / sum(sizeof(component_i))
```

只带几个小 Component 的精简 Archetype，一个 Chunk 能装几百个 Entity；塞了一堆大 Component 的臃肿 Archetype，一个 Chunk 只装得下寥寥几个。占用率低的 Chunk，等于把本可以多装几个 Entity 的字节白白浪费了。

2. TypeManager——Component 注册表

TypeManager 会在 World 启动时把每个 Component、Buffer 和 System 都注册一遍，并为各自分配一个稳定的**类型索引**。运行时则从这张注册表里读取：

字段	用途
TypeIndex	在查询和 Chunk 元数据里用到的打包标识符。
SizeInChunk	在 Chunk 里，这个 Component 每个 Entity 占多少字节。
Alignment	Chunk 内列的对齐方式。
ElementSize	Buffer 元素的大小（用于 IBufferElementData）。
HasBlobReferences / HasEntityReferences	在序列化和移动操作时启用相应的验证。

你一般不会直接碰 TypeManager。真正要紧的是：**新增 Component 类型、重命名它们、或把它们在程序集之间挪动，都会触发类型注册表重建**——这正是大型项目启动慢的主要原因。

Entities 1.4 称，靠跳过不必要的程序集扫描，大型项目里 `TypeManager.Initialize` 提速了约一倍。这项优化在 6.5 中得以延续。

3. 检查 chunk 布局——Archetypes 窗口

Window → Entities → Archetypes 会列出 World 里的每个 Archetype。每一行显示：

- **Entities**——存活的 Entity 数。
- **Chunks**——正在使用的 Chunk 数。
- **Entities / Chunk**——平均占用率。
- **Capacity**——每个 Chunk 理论上的最大容量。

需要关注的情况：

读数	解读
一大堆 Archetype，每个 Chunk 只装着 1 个 Entity	Tag 粒度切得太细，导致 Archetype 数量激增。归并一下。
某个 Archetype 装了大量 Entity，占用率也高	这很健康——敌人、抛射物、子弹都是如此。
Entities / Chunk 远小于 Capacity	碎片化——Entity 不停往别的 Archetype 迁移（结构性抖动），把占用打散了。
Capacity = 1	Archetype 太宽了（Component 太大）。查一查有没有过大的托管/共享 Component。

4. 设计对 Archetype 友好的 Component

下面这些经验法则，大致按影响从大到小排列：

1. **非托管结构体就让它保持非托管。** 哪怕只有一个托管 `IComponentData`，也会逼着每个 Entity 走附属表存储，并且破坏 Burst。
2. **别让结构体出现填充。** 把 `float3` 放在 `float` 前面而不是后面——对齐造成的空洞会撑大 `SizeInChunk`。
3. **把冷字段拆出去。** 如果某个字段只有一个 System 会读，就把它挪到一个并列的 Component。用不到的列仍然要占每个 Entity 的字节。
4. **优先用可启用 Component，别每帧增删。** 参见 `13_Structural Change & Safety.md`——翻一个位，远比重塑 Archetype 便宜。
5. **限制共享 Component 的取值集合。** 每多一个值，就多一个 Chunk 桶。
6. **Tag Component 虽占零字节，却照样多出一个 Archetype。** 别给每个 Entity 都打上独一无二的标记。

5. Chunk 版本号

每个 Component 列都跟踪着一个**变更版本**——一个 32 位计数器，只要有 System 通过 `RefRW<T>.ValueRW`、`SetComponentData` 或 ECB 的 `SetComponent` 写入，它就加一。

System 可以跳过那些自上次运行以来版本号没变的 Chunk：

```
var query = state.GetEntityQuery(ComponentType.ReadWrite<Health>());
query.SetChangedVersionFilter(typeof(Health));
```

带变更过滤器迭代查询时，只有自 System 上次运行以来变过的 Chunk 才会被访问。"Health 一变就刷新 UI"这类模式，正是靠它才得以低成本实现。

注意事项：

- 过滤器是 **Chunk 级别**的，不是 Entity 级别。一个 Entity 的写入就会把整个 Chunk 弄"脏"。
- 用 `RefRW<T>.ValueRW` 读取（哪怕你根本没改值）也会更新版本号。
- 切换可启用 Component 同样会更新版本号。

6. BlobAsset 存储

BlobAssetReference<T> 把大块的只读数据（网格、LOD 表、导航场）存放在 Chunk **之外**，Chunk 里只留一个指针。这样一来，许多 Entity 共用同一份烘焙数据时，就不会把 Chunk 容量撑大。

一个实用小窍门：如果某个 Component 的体量主要来自一个 FixedList，或一个同类 Entity 之间完全相同的 DynamicBuffer，那就把这块大数据存进 blob asset，再从一个精简的 Component 去引用它。

7. 示例

7.1 一个该避免的臃肿 Archetype

```
public struct EnemyState : IComponentData
{
    public float3 Position;
    public quaternion Rotation;
    public float Health;
    public float HealthMax;
    public float Stamina;
    public float StaminaMax;
    public int AIState;
    public float3 LastKnownPlayerPos;
    public float TargetDistance;
    public Entity Target;
    public FixedList512Bytes<float3> PathBuffer;    // 512 bytes per entity!
}
```

光 PathBuffer 这一个字段，每个 Entity 就吃掉 512 字节，每个 16 KB Chunk 的容量因此跌到两位数的低段。把它拆开：

```
public struct EnemyStats : IComponentData
{
    public float Health, HealthMax, Stamina, StaminaMax;
    public int AIState;
    public Entity Target;
}

public struct EnemyPathPoint : IBufferElementData
{
    public float3 Value;
}
```

现在动态 Buffer 有了自己独立的存储，Chunk 密度也就回来了。

7.2 用变更过滤器做低成本的 UI 更新

```
var query = state.GetEntityQuery(ComponentType.ReadWrite<Health>(),
                                ComponentType.ReadWrite<HealthBarLink>());
query.SetChangedVersionFilter(typeof(Health));

state.Dependency = new UpdateHealthBarJob { /* ... */ }
    .ScheduleParallel(query, state.Dependency);
```

只有 Health 变过的 Chunk 才会被访问。

8. 故障排查

现象	原因 / 解决方案
大型项目启动缓慢	TypeManager.Initialize 扫了太多程序集。看一下 Profiler 的启动追踪；确认用的是较新的 Entities 版本。
某个 Archetype 的 Capacity 是 1	Component 太大了（Buffer 容量、内嵌的 blob）。把大块数据挪到 BlobAssetReference<T> 或 IBufferElementData。
明明"什么都没变"，变更过滤器却每帧都触发	有谁用 RefRW<T>.ValueRW 读了却没写。能改成 RefRO<T> 的地方就改。
Archetype 数量持续增长	高基数的逐 Entity 标签，或者以连续值为键的共享 Component。归并一下。
Entities / Chunk 远小于 Capacity	结构性抖动——增删操作把 Entity 打散到各个 Archetype。改用可启用 Component。
CPU Profile 被托管 Component 占据	把托管字段挪到 UnityObjectRef<T>，或者把 Component 拆开；非托管路径快得多。

Chapter 29. Systems · Entity Inspector · Query Window

1. 概述

你会长期泡在这三个 Entities 工具窗口里：

窗口	打开方式	解答的问题
Entities Hierarchy	Window → Entities → Hierarchy	"现在有哪些 Entity? 它们各带着哪些 Component? "
Systems	Window → Entities → Systems	"哪些 System 在运行, 顺序怎样, 各自耗时多少? "
Query	Window → Entities → Query	"给定一组 Component, 哪些 Entity 能匹配? 又有哪些 System 在查询它? "

还有第四个窗口 **Archetypes** (Window → Entities → Archetypes) , 在 01_Chunk Layout & TypeManager.md 里介绍。

2. Entities Hierarchy

它就像一个实时的 Hierarchy, 只不过面向的是 Entity, 而非 GameObject。

World 选择器 (窗口顶部)

在 DefaultGameObjectInjectionWorld (单机) 、ClientWorld/ServerWorld (Netcode) 或测试夹具 World 之间切换。一切换 World, 树形视图就会针对那个 World 的 Entity 重新生成。

树形视图

- 每一行是一个 Entity; 子行是通过 Parent / LinkedEntityGroup 关联起来的 Entity。
- 右侧两列计数: Component 数量、子项数量。
- 点一行 → 它的 Component 就显示在 **Inspector** 里。

搜索

- c:Health——筛出带 Health Component 的 Entity。
- n:Enemy——按名称筛选。

- w:Server——按 World 筛选。
- 可以组合：c:Health c:Enemy n:boss。

Inspector

一个 Entity 的 Inspector 会显示：

- **Components** 区——每个 IComponentData 及其当前值，在 Play 模式下可实时编辑。
- **Relationships**——与其他 Entity 的关联（Parent、Child、LinkedEntityGroup）。
- **Enabled state**——每个 IEnableableComponent 的开关状态。

实时编辑是**直写**的：改一个字段，就会立刻更新那个 Chunk 的变更版本。

3. Systems

Systems 窗口会列出每个已注册的 System，按各自的 SystemGroup 分组，并按调度器实际执行的次序排列。

优先阅读的列

列	含义
Name	System 的类型名。
Namespace	它所在的位置（往往能据此分辨是包的代码还是你自己的代码）。
Time (ms)	OnUpdate 占用的主线程时间，大约每帧更新一次。
Queries	这个 System 用到的查询数量。
Entity Count	主查询匹配到的 entity 数量。

需要关注的内容

读数	解读
某个 System 显示 0 ms，却是绿色	被 RequireForUpdate<T> 或 [RequireMatchingQueriesForUpdate] 跳过了。这是正常的。
某个 System 的 ms 一直很高	这是真实的工作量。到 Profiler 里深挖一下细节。
某个 System 的 entity 计数显示为 --	没有匹配的查询（或者查询还没构建出来）。
两个 System 的先后顺序和预期对不上	[UpdateBefore] / [UpdateAfter] 链配错了，或者它们根本不在同一个组里。

启用 / 禁用

每个 System 都有个复选框——Play 模式下取消勾选就能跳过它，不用重新编译。做 A/B 对比时很顺手。

System 详情面板

选中一个 System，会弹出一个侧边面板，里面有：

- **Queries**——每个查询的 component 组合（All/Any/None/Disabled/Present/Absent）。
- **Dependencies**——该 System 的 [UpdateBefore]/[UpdateAfter] 依赖，已解析成实际的调度顺序。
- **Type Info**——类上、以及每个方法上有没有 [BurstCompile]。

在 1.4+ / 6.x 中，**Queries 标签页现在会显示 Disabled / Present / Absent / None 这几种 component 状态**——涉及可启用 Component 的查询不再含糊不清。

4. Query 窗口

专门用来回答"谁匹配 X？"这类问题。

工作流程：

1. 用加号按钮添加 All component。
2. 按需再加 Any / None。
3. 窗口左侧面板列出**匹配到的 entity**，右侧面板列出**跑这个查询的 System**。

使用场景：

- "哪些 System 读了 Health？"→ 把 Health 加进 All；右侧列出的 System 全都会用到它。
- "这个 entity 为什么不匹配我 System 的查询？"→ 把 System 的查询贴进来，对照 Hierarchy 窗口里这个 entity 到底带了哪些 component。
- "有没有 Prefab 混进了这个查询？"→ 1.4+ 起，结果列表里 Prefab 会用图标区分出来。

Query 窗口还能把查询存下来，在本次会话里反复使用。

5. Journaling——万不得已时的诊断手段

Window → Analysis → Entities Journaling（在 Project Settings → Entities → Journaling 中启用）会把每一次结构性变更、component 写入和 System 更新都记进一个环形缓冲区。它的回放视图有：

- **Operations**——逐条列出记录下来的变更，附带时间戳、来源 System 和受影响的 entity。
- **Entities**——每个 entity 的时间轴，记录谁在什么时候动过它。

Journaling 是有运行时开销的——只在查 bug 时开，平时开发就关掉。

6. 实战流程——排查"更新没生效"

有个常见 bug："System 明明处理了这个 entity，可 component 没变。"排查步骤如下：

1. **Entities Hierarchy** → 找到这个 entity，确认它确实带着那个 component。
2. **Inspector** → Play 之前先记下值，按下 Play 后盯着它看。
3. 值如果不变：到 **Systems 窗口** → 找到相关 System，看 **Time (ms)**。是 0，说明这个 System 被跳过了。
4. 时间大于 0 但值还是不变：到 **Query 窗口** → 把 System 的查询和这个 entity 的 component 都贴进去，确认 entity 到底匹不匹配。
5. 还是没头绪：开 **Journaling** 录 1 到 2 帧，看看是不是有别的 System 写了这个 component、把你的结果覆盖掉了。

7. 故障排查

现象	原因 / 解决方案
Play 模式下 Entities Hierarchy 是空的	World 选择器选错了 World（比如选了只有 Client、没有 Server 的那个）。
实时字段不刷新	域重载之后，你看的是个缓存下来的旧 entity。重新选一下它。
Systems 窗口显示 0 ms，可 System 明明在跑	如果实际工作发生在 ScheduleParallel 中，Profiler 可能会把它记录在 chunk job 的标记下。这种情况下，主线程 OnUpdate 确实为 0；应在 Profiler 中查看 job 耗时。
Query 窗口找到了 entity，我的 System 却没找到	System 加了 WithAll<T>() 或变更过滤器，而 Query 窗口里没填这些。把条件对齐到完全一致。
Journaling 窗口是空的	Project Settings 里没启用 Journaling，或者缓冲区已经被覆盖了。

现象	原因 / 解决方案
Entity 和 GameObject 看着对不上	SubScene 还开着（白色图标）——关掉它（图标变黄），让 Baker 运行起来，entity 就出现了。

Chapter 30. Profiler · Bottleneck Analysis

1. 概述

在 DOTS 里，要回答“这帧怎么这么慢”，Unity Profiler (Window → Analysis → Profiler) 是头号工具。Entities 打出的标记足够多，让你能把耗时归到具体的 System、Job 和 Chunk 上——前提是你知道往哪儿看。

本节会讲怎么读 Profile、有哪些典型的 DOTS 瓶颈模式，以及该如何应对。

2. 分析前的准备

1. **少用 Deep Profile。** 它对追查某条特定的托管路径有用，但会把耗时数据搞失真——别拿 Deep Profile 的性能数字当真。
 2. **勾上“Record in Editor & Standalone”，** 这样能把构建版本也录进来，而不只是 Editor 的帧。
 3. 测量正式发布的性能时，**把 Profiler 的目标设成 Player，而不是 Editor。** Editor 的开销不小。
 4. **先跑上几秒再读数据；** 头几帧里夹着域重载和 JIT 的活儿。
-

3. 读取 CPU Usage 模块

一个 DOTS 帧下的顶层标记：

```
PlayerLoop
├─ Initialization.*
├─ Update.ScriptRunBehaviourUpdate          (MonoBehaviour.Update)
├─ SimulationSystemGroup                    ← DOTS simulation lives here
│   ├── SystemA.OnUpdate
│   ├── SystemB.OnUpdate
│   └─ EndSimulationEntityCommandBufferSystem.Playback
├─ FixedStepSimulationSystemGroup
├─ PresentationSystemGroup
├─ Render
└─ WaitForTargetFPS
```

展开 SystemA.OnUpdate 标记，就能看到它的子调用——ScheduleParallel、等待 job 完成、EntityManager 调用。

4. 时间到底花在哪——几种常见情形

4.1 主线程 OnUpdate 占了大头

如果单个 `SystemX.OnUpdate` 就吃掉 10 多 ms，说明这些活儿是在主线程上跑的。检查：

- 是不是在紧凑循环里用了 `SystemAPI.Query`，而它本该是 `IJobEntity.ScheduleParallel`？
- 是不是有个 `SystemBase` 因为带了托管字段，被迫跑在主线程上？
- 是不是在逐个 entity 调用 `EntityManager.AddComponent/RemoveComponent`？

4.2 Job 计时

Job 会显示在**时间轴**视图里（层次视图里则在"Workers"下面）。重点关注：

- 一条 Job 泳道满负荷，其余泳道却闲着 → 工作没摊到各个 Chunk 上。
- 主线程在等 Job (`JobHandle.Complete()` 标记) → `.Complete()` 调得太早了，往帧的后面挪。
- 超长的 Burst 函数名 → 正常，那个符号是完整的泛型实例化名。

4.3 EntityManager 热点

留意这些名字的标记：

- `EntityManager.SetArchetype / Move Entity`——结构性变更抖动。详见 13_Structural Change & Safety.md。
- `EntityCommandBufferSystem.OnUpdate (Playback)`——昂贵的 ECB 回放。通常说明每帧录的命令太多了。
- `TypeManager.Initialize`——只该在启动时出现；要是每帧都冒出来，说明有东西在反复重载 World。

4.4 GC.Alloc 标记

DOTS 模拟中只要出现 `GC.Alloc`，就是退步。可能的来源有：

- 某个 `SystemBase` 在 `OnUpdate` 里分配了内存。
- `System` 内部 `foreach` 遍历了一个托管集合。
- 装箱——比如对一个结构体调用 `object.ToString()`。

在 Profiler 的 Memory 模块里，Burst System 应该是零分配。

5. 值得常驻的 Profiler 模块

- **CPU Usage**——标记的时间轴视图和层次视图。
- **Memory**——已分配的堆；盯着 GC.Used Memory 看它有没有往上爬。
- **Entities**（如果有）——DOTS 专属模块，展示各 System 随时间变化的耗时。

Entities Profiler 模块是 1.8.0 / 6.x 引入的，它在时间轴上和 Systems 窗口对齐。要是你看到跨多帧的尖峰，用它来归因最快。

联网项目则要配合 ../DOTS Workflows/24_Netcode Profiler & Debugging.md 一起用，去查看 Ghost 快照、预测 Tick 和 Netcode 专属的带宽。

6. 常见瓶颈与修复方案

模式	Profiler 里的表现	修复办法
逐 entity 的 Entities.ForEach (遗留)	OnUpdate 重度占用主线程，且有大量细碎样本	移植到 IJobEntity.ScheduleParallel。
每帧都 AddComponent/RemoveComponent	Move Entity 标记占了大头	把这个状态改成可启用 component。
ECB 负担过重	...ECBS.Playback 占用了好几 ms	把命令批量化（每个 chunk 一条，而非每个 entity 一条），或者调大批处理大小。
共享 component 的取值爆炸式增长	SetSharedComponent 占了大头	给值做分桶（量化成更少的不同键）。
在帧中途就调了 .Complete()	主线程上 JobHandle.Complete 等了很久	把完成点推到帧末，或者重构一下，让主线程不必那么早就要结果。
Burst 回退了	热点方法显示的是没经过修饰的 C# 名字	捕获了非 blittable 类型，导致回退到 IL。查看 Burst Inspector。
Archetype 碎片化	有许多小 chunk； EntityManager.*Query 标记偏高	Archetype 切得太碎——归并标签，或改用可启用 component。
Bake 阶段出现退步	Play 期间冒出 Bake 标记	域重载重新触发了 baking。一般只在 Editor 里发生；到构建版本里确认一下。

7. 帧预算示例

假设帧预算是 16.6 ms (60 Hz) 。排除掉 Render 之后：

- SimulationSystemGroup: 6 ms
 - EnemyAISystem.OnUpdate: 2.1 ms (Burst, 4 个工作线程上的并行 Job)
 - MovementSystem.OnUpdate: 1.4 ms (Burst, 并行)
 - EndSimulationECBS.Playback: 1.8 ms ← 可疑
 - 其余所有: 0.7 ms

这个场景本该只有寥寥几次结构性变更，ECB 回放却花了 1.8 ms——这通常就是最大的优化抓手。查一下命令数量：

1. 对一帧开启 Entities Journaling → 统计每个 System 的 Instantiate/AddComponent/DestroyEntity 各有多少条。
2. 如果某个 System 每帧录上数百条命令，就做批处理（每个 chunk 一条命令，或者用一个以 chunk 索引为排序键的并行记录器）。

8. 对独立版本做性能分析

针对真实的构建版本：

1. **File → Build Profiles → Development Build + Autoconnect Profiler。**
2. Player 跑起来，Profiler 从远程进程采集数据。
3. 这样的结果更贴近玩家的真实体验。Editor 的测量值可能慢上 2 到 5 倍，而原因往往和你的代码无关。

9. 故障排查

现象	原因 / 解决方案
Profile 里看不到 DOTS 标记	Profiler 模块把它们过滤掉了——在模块设置里把"Scripts"和"Jobs"打开。
帧很慢，却找不到热点标记	可能是在等渲染线程。打开 GPU 模块和时间轴视图。
Deep Profile 显示出不一样的热点	Deep Profile 会让分配和调用计数失真。相对耗时还是以非深度 Profile 为准。
Burst 方法的调用栈里出现了托管调用	这是回退——Burst 编译不了它。看看 Burst Inspector 的输出。

现象	原因 / 解决方案
GC.Collect 标记激增	某个 System 里有托管分配。用 Memory 模块 → Snapshot 来缩小排查范围。
System 耗时在各帧之间剧烈波动	工作量随 entity 数量增长，而且每帧都有 chunk 进出查询。检查结构性抖动。

Chapter 31. Managed Object Reference Audit

1. 概述

托管的 Unity 对象引用，是把 ECS System 不慎拽出快速路径的最常见诱因之一。这套审计能帮你揪出那些以妨碍 Burst、Job 或干净 SubScene Baking 的方式持有 `UnityEngine.Object` 引用的地方——无论它藏在 component 数据、System，还是渲染胶水代码里。

核心 ECS 迁移完成后，或者每当 Profiler 追踪在你预期是数据导向代码的地方暴露出托管开销时，就做一遍这套审计。

2. 搜索什么

模式	为何要在意
<code>class .*: IComponentData</code>	托管 component 数据；合法，但一般不适合热路径。
<code>UnityEngine.Object</code>	用来宽泛地标出托管 Unity 对象引用。
<code>GameObject</code> 、 <code>Transform</code> 、 <code>Component</code>	对运行时场景对象的引用，通常不该出现在模拟数据里。
<code>Mesh</code> 、 <code>Material</code> 、 <code>Texture</code> 、 <code>AudioClip</code>	通常该改用 <code>UnityObjectRef<T></code> ，或表现层的缓存数据。
<code>Resources.Load</code>	运行时的资源查找；通常更该交给 baking 或内容系统来处理。
<code>ScriptableObject</code>	是个好用的 Authoring 数据源，但运行时 Job 该读的是烘焙值或 blob。

把审计范围限定在面向 ECS 的代码上。纯 `MonoBehaviour` 的 UI 或编辑器工具，没必要硬塞进 DOTS 模式。

3. 分类规则

查到的情况	大致该怎么做
热点模拟查询里出现托管 component	改成非托管 <code>IComponentData</code> 、 <code>BlobAssetReference<T></code> 或 <code>UnityObjectRef<T></code> 。
只被表现层用到的托管 component	只要它和 Burst/Job 的模拟路径隔离开，就保留。
每帧都在读 <code>ScriptableObject</code>	把需要的值烘焙进 component 或 blob asset。

查到的情况	大致该怎么做
ECS 数据里有 Mesh/Material 引用	改用 UnityObjectRef<Mesh> / UnityObjectRef<Material>, 并在表现层代码里解引用。
component 数据里存了 GameObject/Transform	换成 ECS 数据, 或者把引用挪到一个混合式的表现层桥接里。

4. 审计示例

```
// Suspicious: managed component data used as per-entity visual state.
public class EnemyVisual : IComponentData
{
    public Material Material;
}
```

推荐做法:

```
using Unity.Entities;
using UnityEngine;

public struct EnemyVisual : IComponentData
{
    public UnityObjectRef<Material> Material;
}
```

然后在主线程的表现 System 里解引用, 或者把解析出来的 material 缓存进一张渲染器注册表。

5. Profiler 信号

Profiler 里的表现	该查什么
模拟 System 每帧都在分配内存	OnUpdate 里有托管对象查找, 或 LINQ/集合带来的分配。
本该纯数据的 System, Burst 却被禁用了	查询路径里掺了托管 component, 或访问了托管 API。
主线程上有大量细碎的渲染查找	把 UnityObjectRef<T>.Value 的结果缓存到逐 entity 循环之外。
SubScene 重新 Bake 的频率高于预期	Authoring 脚本或某些对象引用造成了不必要的依赖。

6. 验证

- 1. 把第 2 节的搜索模式再跑一遍。
- 2. 确认热点模拟 System 只查询非托管 component。
- 3. 对那些预期能被 Burst 编译的 System/Job，查一下 Burst Inspector。
- 4. 在一个有代表性的场景里做性能分析，确认热路径上的托管分配已经清干净。
- 5. 从干净启动的 Editor 进入 Play 模式，验证 SubScene Baking 依然能解析这些引用。

7. 故障排查

现象	原因 / 解决方案
转换之后，component 还是没法在 Burst 里查询	还有某个字段是托管类型。嵌套的结构体也要查。
Play 模式下 UnityObjectRef<T>.Value 返回 null	被引用的资源没被烘焙或加载。检查 Baker，以及 SubScene/内容工作流。
转换之后，视觉 System 还是很慢	你在每帧逐 entity 解引用。改成按共享资源缓存，或烘焙一个紧凑的查找键。
改了 ScriptableObject 却不影响运行时数据	那些值已经被烘焙固化了。重新导入/重新烘焙 SubScene，或者补上正确的 Baking 依赖。
只在编辑器里有效的引用，到 Player 构建里就失效	把编辑器 API 包进 #if UNITY_EDITOR，并烘焙出运行时安全的数据。

Part IV. Migration

Chapter 32. Entities 1.x → 6.5 Overview

1. 你即将面对的工作

从 Entities 1.x 升级到 6.5，主要是在**一次包分发模式转变**之上完成**三项 DOTS 迁移**：

1. **Editor/包分发升级。** 迁移到 Unity 6000.4+ (`com.unity.entities 6.4`) 或 6000.5+ (Entities 6.5)，其中 Entities 是嵌入 Editor 的 Core Package。
2. **ECS 数据清理。** 审查面向 ECS 的数据中的托管 Unity 对象引用，并将热路径 component 数据尽量迁移为非托管 component、烘焙值、`BlobAssetReference<T>` 或 `UnityObjectRef<T>`。
3. **ECS API 迁移。** 在 Entities 1.4 中标记为过时的旧版 API——`Entities.ForEach`、`IAAspect`——在 6.5 中仍然过时。另外：`ComponentLookup.GetRefRWOptional` → `TryGetRefRW`。

每项迁移都在本文件夹的独立子文档中完成。本页是**地图**——何时做哪项、按什么顺序。

命名注意： Entities **包**版本 (1.4 / 6.4 / 6.5) 与 Unity Editor 版本 (6000.3 / 6000.4 / 6000.5) 不同。从 6.4 起按惯例对齐，但在独立的发布说明中跟踪。本文档将它们视为两个轴；Changelog 页面 (`../Changelog/Entities 1.4 → 6.5 Key Changes.md`) 同样如此。

2. 推荐顺序

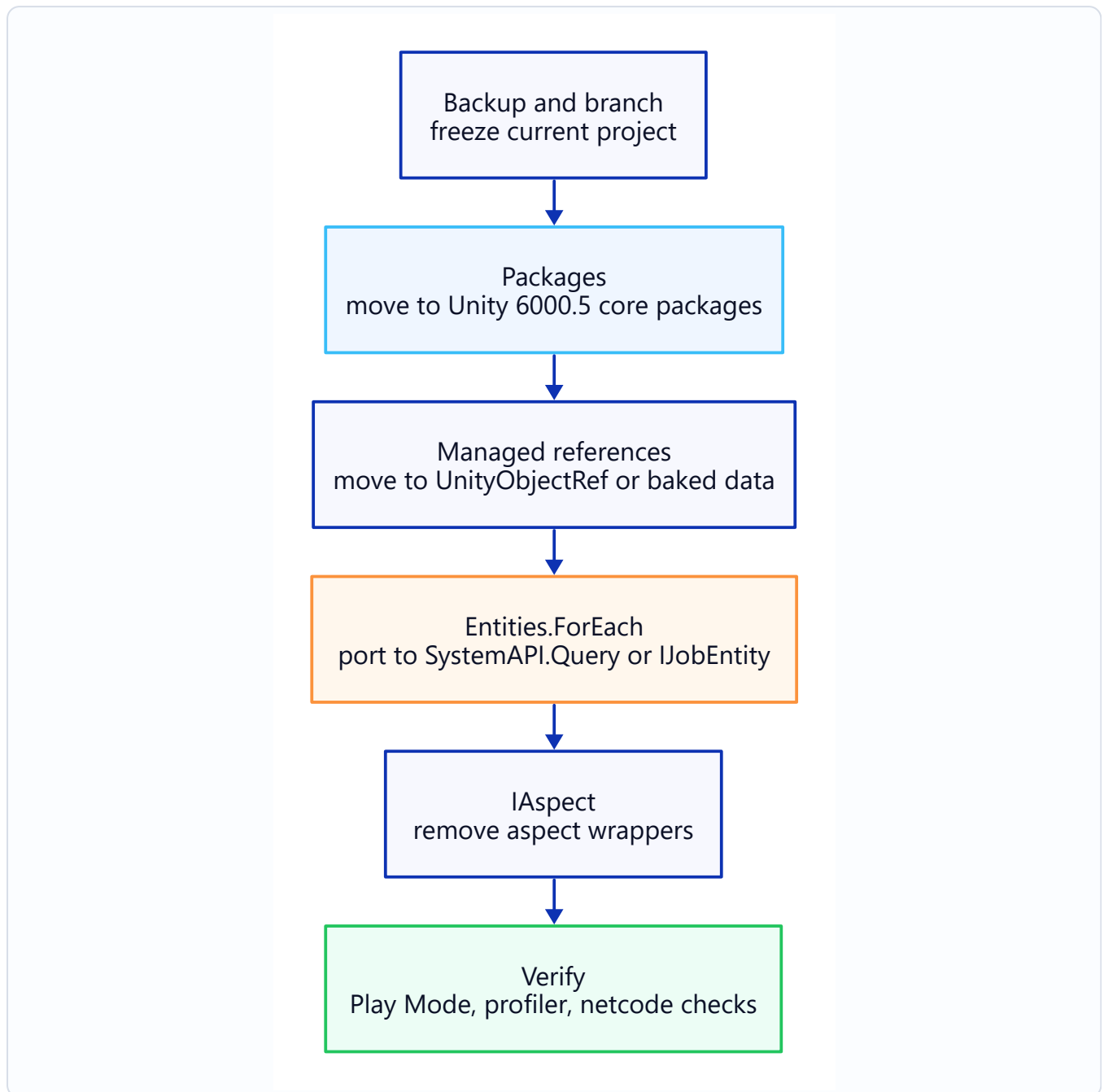


图: Entities 1.x 到 6.5 迁移顺序。

为何如此排序：

- **先升级 Editor**，因为在 Editor 识别 6.x API 之前，其他一切都无法编译。
- **接着升级 Package Manager → Core Package**——不这样做会导致包解析重复/冲突。
- **在迭代重构之前进行托管引用清理**——在围绕数据模型重写 System 和 Job 之前，先使数据模型清晰。
- **foreach / IAspect 最后处理**——这些是 ECS 内部的；不会阻碍代码库其余部分。

3. 什么保持不变

令人惊讶地多——这让迁移不像看起来那么可怕。

不变项	说明
IComponentData、IBufferElementData、ISharedComponentData	相同接口，相同语义。
ISystem / SystemBase	相同生命周期钩子，相同特性。
EntityManager API 接口	CreateEntity、AddComponent 等——完全相同。
SystemAPI.Query	在 1.x 中引入，仍是核心迭代 API。
IJobEntity、IJobChunk	形式相同；IJobEntity 是推荐的默认选项。
EntityCommandBuffer + ECBS	相同。
Burst、Jobs、Collections、Mathematics	相同。
Netcode for Entities 概念 (Ghost、RPC、Prediction)	相同架构；包的分发方式不同。

如果你的 1.x 代码已经使用 IJobEntity 和 SystemAPI.Query（而非遗留 Entities.ForEach），迁移的第 4 和第 5 步对你来说基本消失了。

4. 变化内容一览

区域	1.x	6.5
分发方式	com.unity.entities UPM 包	Core Package（无 manifest 条目）
版本号	1.4.x	6.5.x（与 Unity 6000.5 对齐）
ECS 数据中的托管 Unity 对象引用	通常存储在托管 component 中	适当时优先使用烘焙数据、blob 或 UnityObjectRef<T>
Component 迭代（遗留）	Entities.ForEach 在 1.4 中标记为过时	仍然过时，仍可带警告编译；计划在未来主版本中删除
Aspect	IAspect 在 1.4 中标记为过时	仍然过时，仍可编译；计划在未来主版本中删除
可选引用	GetRefRWOptional / GetRefROOptional 已废弃	TryGetRefRW / TryGetRefRO

该时期的完整 changelog 见 ../Changelog/Entities 1.4 → 6.5 Key Changes.md。

5. 备份策略

开始前不可妥协的步骤：

1. **在 git 中创建分支。** 使用描述性名称 (migrate/entities-6.5) 。不要在 main 上迁移。
 2. **提交迁移前状态的 manifest.json 和 packages-lock.json。** 如果解析失败，它们是回滚的唯一真实来源。
 3. **标记迁移前的提交。** git tag pre-entities-6.5 使回滚变得简单。
 4. **在准备好之前，保持项目在旧 Editor 版本中打开。** 在 6000.5 中打开一次会更改 ProjectVersion.txt 并强制进入迁移路径。
-

6. 各步骤检查清单

6.1 Editor 升级

- 通过 Hub 安装 Unity 6000.5.x。
- **尚不要**打开项目。先提交所有待处理工作。
- 在新 Editor 中打开项目——它将提示升级 ProjectVersion.txt。接受。
- 等待初始导入完成；预期会有关于过时 API 和缺失包的警告。这正是后续步骤要修复的。

6.2 包清理 → 02_Package Manager → Core Package.md

- 从 manifest.json 中删除 com.unity.entities、com.unity.collections、com.unity.mathematics、com.unity.entities.graphics。
- 删除 packages-lock.json 并让 Editor 重新生成。
- 如果项目使用 Netcode for Entities，也一并删除（6.5 版本是 Core Package）。

6.3 托管对象引用 → 03_Managed Object References → UnityObjectRef.md

- 审计面向 ECS 的代码中的托管 Unity 对象字段。
- 尽可能将热路径 component 数据转换为非托管 ECS 数据。
- 对 ECS 侧指向 Unity 资源/对象的引用使用 UnityObjectRef<T>。
- 对大型不可变游戏数据使用 BlobAssetReference<T>。

6.4 遗留 ForEach → 04_foreach → IJobEntity.md

- 对仍使用 `Entities.ForEach / Job.WithCode` 的 System——迁移到 `IJobEntity + SystemAPI.Query`。

6.5 Aspect 移除 → 05_IAspect Removal.md

- 使用 `RefRW<T>` / `RefRO<T>` 参数将 Aspect 方法内联到调用点。
 - 删除 `IAspect` 结构体。
-

7. 迁移后的测试

在宣布迁移完成前需要运行的冒烟测试：

1. **在代表性 SubScene 上进行 Play 模式测试。** Entity 存在，System 运行，启动时无异常。
 2. **构建独立 Player。** 仅在构建时才会出现的问题（通常是序列化问题）会在此处暴露。
 3. **运行现有的游戏玩法回归测试。** 如果性能重要，在迁移前捕获基线 Profiler 追踪并进行对比。
 4. **检查存档/读取。** 原始 Entity 句柄是 World 本地的且不稳定。请存储游戏自有 ID、内容键或 Authoring 数据。
 5. **在目标平台上运行。** 某些平台特定的后端（IL2CPP 问题、AOT）会在此处暴露。
-

8. 何时回滚

干净回滚的触发条件：

- Editor 根本无法打开项目（罕见；通常是 `ProjectVersion.txt` 不匹配）。
- `packages-lock.json` 解析产生无效图，删除后仍未修复。
- 你依赖的某个关键包（第三方）尚不兼容 6.5。

计划： `git checkout pre-entities-6.5`，重新安装旧版 Unity，验证游戏在旧版本上仍可运行。

已部分完成但卡住的迁移应干净地放弃并从标记重新开始——不要交付半迁移的代码。

9. 故障排查

现象	原因 / 解决方案
Editor 无法打开 ——"项目保存在更新的版本中"	之前有人在 6000.5 中打开过；将 ProjectSettings/ProjectVersion.txt 回滚到旧版本字符串。
清理 manifest 后， 编译错误"Unity.Entities 程序集未找到"	Editor 版本低于 6000.4——Core Package 不可用。升级 Editor 或重新添加 UPM 包。
数百个废弃警告	预期中的。将它们视为迁移清单——每个都指向需要修复的内容。
构建成功，游戏启动时崩溃	通常是 ECB 或序列化问题，因删除了某个 component 而引入。对比烘焙后的 EntityScene 资源大小；大幅缩减表明某个 component 被静默删除。
迁移后性能回归	通常是引入了不该引入的 SystemBase（托管字段），或 IJobEntity 从 Burst 回退。检查 Profiler 和 Burst Inspector。

Chapter 33. Package Manager → Core Package

1. 变化内容

在 Unity 6000.4+ 上，Entities（及其配套 DOTS 包）成为 **Core Package**：它们从 Editor 本身安装，而非从远程注册表下载，其版本由 Editor 固定。你**仍要通过 Package Manager 添加它们**，它们也仍会作为依赖列在你的项目中——成为 Core Package 改变的是包从何而来、由谁决定其版本，而不是你的项目是否需要声明它。迁移 1.x 项目时会发生以下变化：

- **Packages/manifest.json**——`com.unity.entities*` 条目**保留**，但其固定的 **1.x** 版本被替换为 Editor 的 Core 版本（从 Package Manager 重新添加它们）。
- **Packages/packages-lock.json**——受影响的包现在以 `"source": "builtin"` 解析。
- **Package Manager 窗口**——包仍显示在 "In Project" 下；你不能再选择它们的版本（版本跟随 Editor）。

你的 C# 代码**不会**改变。程序集名称（`Unity.Entities`、`Unity.Collections` 等）相同；`.asmdef` 引用不变。

2. 哪些包迁移

在 6000.4+ 中已转为 **Core Package**：

UPM id (之前)	之后
<code>com.unity.entities</code>	Core Package, 随 Editor 发布
<code>com.unity.collections</code>	Core Package
<code>com.unity.mathematics</code>	Core Package
<code>com.unity.entities.graphics</code>	Core Package
<code>com.unity.netcode</code> (Netcode for Entities)	6.5 版本中的 Core Package

仍通过 **Package Manager** 分发：

- `com.unity.physics` (Unity Physics)
- `com.unity.charactercontroller` (Character Controller)

内置于引擎（不变）：

- Jobs system
- Burst (com.unity.burst 在某些设置中仍作为 UPM 条目公开，但在 6000.4+ 上与 Editor 版本绑定；最安全的做法是保持 manifest 不变)。

3. 编辑 manifest.json

打开 Packages/manifest.json，内容类似：

```
{
  "dependencies": {
    "com.unity.entities": "1.4.2",
    "com.unity.collections": "2.4.5",
    "com.unity.mathematics": "1.3.2",
    "com.unity.entities.graphics": "1.4.2",
    "com.unity.netcode": "1.12.0",
    "com.unity.physics": "1.3.14",
    "com.unity.render-pipelines.universal": "17.0.4",
    ...
  }
}
```

在 6000.4+ 的 Editor 上，那些固定的 **1.x** 版本不再能解析——注册表不再为此 Editor 提供它们。**不要** 直接删除这些行（那会移除依赖，而 Core Package 不会被自动添加回来）。正确做法是让每个 Core Package 指向 Editor 的版本。有两种方式，两者都会把这些包保留为依赖：

推荐——通过 Package Manager 重新添加。对每个 Core Package，先移除其过时的 1.x 固定版本，然后通过 **Window → Package Manager → Unity Registry → Install**（或 **+ → Add package by name...**，例如 com.unity.entities）把它添加回来。Package Manager 会替你把 Editor 的 Core 版本写入 manifest.json，并自动引入依赖的 Core Package。

手动操作。把每个 Core Package 的 1.x 版本替换为 Editor 的 Core 版本（例如在 Unity 6000.5 上为 6.5.0）：

```
{
  "dependencies": {
    - "com.unity.entities": "1.4.2",
    + "com.unity.entities": "6.5.0",
    - "com.unity.entities.graphics": "1.4.2",
    + "com.unity.entities.graphics": "6.5.0",
    - "com.unity.netcode": "1.12.0",
    + "com.unity.netcode": "6.5.0",
    "com.unity.physics": "1.3.14",
    "com.unity.render-pipelines.universal": "17.0.4",
    ...
  }
}
```

```
}  
}
```

`com.unity.collections` 和 `com.unity.mathematics` 会作为 `Entities` 的依赖被引入，因此通常无需自己列出它们。保留 `com.unity.physics` 以及其他仍为注册表（UPM）包的内容不变。

保存文件。

4. 重建 `packages-lock.json`

两种选择：

4.1 让 Editor 重新生成

关闭 Editor，删除 `Packages/packages-lock.json`，重新打开。Unity 重新生成它。这是最干净的选择。

4.2 就地编辑

打开 `packages-lock.json` 并更新上述各包的过时条目（删除它们，让 Editor 重新解析）。Editor 下次打开时进行协调。如果你有复杂的锁定文件，速度稍快，但更容易出错。

两种方式最终的锁定文件都包含类似以下的条目：

```
"com.unity.entities": {  
  "version": "6.5.0",  
  "depth": 0,  
  "source": "builtin",  
  "dependencies": {}  
}
```

注意 `"source": "builtin"` 是包现在已成为 Core Package（由 Editor 提供）的标志，而 `"depth": 0` 表明它仍是 `manifest.json` 中声明的**直接依赖**——该条目并未消失。

5. 验证结果

- 1. **Package Manager 窗口** (Window → Package Manager → "In Project") ——Entities、Collections、Mathematics、Entities Graphics (如果使用, 还有 Netcode for Entities) 应列在其中, 其版本由 Editor 固定 (没有版本下拉菜单)。
- 2. **Editor 菜单**——Window → Entities → Hierarchy/Systems/Archetypes 存在并正常工作。
- 3. **编译**——按下 Play; 项目应在没有引用缺失程序集的编译错误的情况下进入 Play 模式。

6. 故障排查

现象	原因 / 解决方案
升级 Editor 后出现 Unable to find package 'com.unity.entities@1.x.y'	manifest 仍固定着 1.x 版本。把它更新为 Core 版本 (通过 Package Manager 重新添加), 然后删除 packages-lock.json 并让 Editor 重新生成。
The type or namespace Unity.Entities could not be found	项目中尚未添加 Entities (通过 Package Manager 添加), 或者你使用的 Editor 版本低于 6000.4 (升级它)。
Package Manager 显示过时或冲突的 Entities 版本	manifest.json 中仍残留着 1.x 固定版本。把它更新为 Core 版本 (通过 Package Manager 重新添加), 并重新生成 packages-lock.json。
第三方资源无法编译, 因为它固定了 com.unity.entities@1.x	该资源需要更新。等待、fork, 或让此项目保留在 1.x 版本线。
Unity Physics 因需要特定 Entities 版本而出错	将 Unity Physics 升级到与 Entities 6.5 兼容的版本。在 docs.unity3d.com 上查看包的 changelog。
packages-lock.json 重新生成失败	网络问题或固定的私有注册表。检查 Editor 日志中的具体解析失败信息。
Netcode for Entities 仍解析为旧的 1.x 版本	在 6000.5+ 上, Netcode 是 6.5 Core Package。把 com.unity.netcode 更新为 6.5 版本 (通过 Package Manager 重新添加) 并重新生成。

Chapter 34. Managed Object References → UnityObjectRef

1. 概述

迁移旧版 ECS 代码时，一个常见问题是托管对象引用泄漏到运行时 component 数据中。在 Entities 6.5 中，尽可能保持游戏 component 为非托管类型，并在 ECS 数据必须指向 Unity 对象（如网格、材质、纹理、音频片段或 Authoring ScriptableObject）时使用 `UnityObjectRef<T>`。

本页范围限于 DOTS component 设计。不涵盖引擎范围的 Unity 对象标识符 API。

2. 需要查找的内容

搜索直接存储托管 Unity 对象的 component：

模式	需要审查的原因
实现 <code>IComponentData</code> 的 class component	托管 component 是允许的，但速度较慢且受主线程约束。
component 数据内部的 <code>UnityEngine.Object</code> 字段	通常表明是托管 component 或无效的非托管 component 设计。
<code>GameObject</code> 、 <code>Transform</code> 、 <code>Material</code> 、 <code>Mesh</code> 、 <code>Texture</code> 、 <code>AudioClip</code> 字段	判断这是否属于 Authoring、表现层或 <code>UnityObjectRef<T></code> 。
运行时 System 逐 entity 调用 <code>Resources.Load</code>	将资源解析移至 Baking 或中央内容系统。

3. 迁移前后

迁移前——托管 component

```
using Unity.Entities;
using UnityEngine;

public class ProjectileVisual : IComponentData
{
    public Mesh Mesh;
    public Material Material;
}
```


这样写能跑，但会把 Component 变成托管类型。托管 Component 会削弱 Burst/Job 的灵活度，一般也不适合做逐 Entity 的玩法数据。

迁移后——使用 UnityObjectRef<T> 的非托管 component

```
using Unity.Entities;
using UnityEngine;

public struct ProjectileVisual : IComponentData
{
    public UnityObjectRef<Mesh> Mesh;
    public UnityObjectRef<Material> Material;
}
```

在 Baking 时把引用一次性烘焙好：

```
using Unity.Entities;
using UnityEngine;

public class ProjectileVisualAuthoring : MonoBehaviour
{
    public Mesh Mesh;
    public Material Material;
}

public class ProjectileVisualBaker : Baker<ProjectileVisualAuthoring>
{
    public override void Bake(ProjectileVisualAuthoring authoring)
    {
        var entity = GetEntity(TransformUsageFlags.Dynamic);
        AddComponent(entity, new ProjectileVisual
        {
            Mesh = new UnityObjectRef<Mesh>(authoring.Mesh),
            Material = new UnityObjectRef<Material>(authoring.Material)
        });
    }
}
```

4. 运行时访问模式

让模拟 Job 保持纯粹的数据导向。只在允许调用托管 Unity API 的地方,才解引用 Unity 对象。

```
using Unity.Entities;
using UnityEngine;

public partial class ProjectileVisualUploadSystem : SystemBase
{
    protected override void OnUpdate()
    {
        foreach (var visual in SystemAPI.Query<RefRO<ProjectileVisual>>())
        {
            Mesh mesh = visual.ValueRO.Mesh.Value;
        }
    }
}
```

```
Material material = visual.ValueRO.Material.Value;

if (mesh == null || material == null)
    continue;

// Upload to a rendering registry, material property block cache, etc.
}
}
}
```

渲染量大时，别每帧逐 entity 去解引用。把解析出来的对象缓存进一张表现层注册表，用一个紧凑的 ECS 值当键。

5. 何时不使用 UnityObjectRef<T>

场景	更好的选择
纯游戏玩法数据	把基础类型、数学类型或 blob 数据直接存进 IComponentData。
大型只读表	用 BlobAssetReference<T>。
已烘焙进同一 SubScene 的 Entity Prefab	用 Entity Prefab 引用。
经内容工作流流式加载的 Prefab	用 EntityPrefabReference。
逐 entity 的复杂托管行为	把它留在模拟路径之外，或隔离到一个托管的表现 System 里。

6. 迁移检查清单

- ☐ 找出那些只为存 Unity 对象引用而存在的托管 component 数据。
- ☐ 凡是适合用非托管 component 的地方，把这些字段换成 UnityObjectRef<T>。
- ☐ 把对象赋值挪进 Baker。
- ☐ 别把 .Value 解引用放进 Burst Job。
- ☐ 多个 entity 共用同一资源时，把解引用后的对象缓存起来。
- ☐ 大型不可变的玩法数据用 BlobAssetReference<T>，别在运行时去读 ScriptableObject。

7. 故障排查

现象	原因 / 解决方案
Component 没法在 Burst Job 里用	它还是托管类型。把 Unity 对象字段改成 UnityObjectRef<T>, 或者把表现数据拆出去。
.Value 是 null	被引用的资源没加载, 或已被销毁。检查 Authoring 引用和内容加载路径。
System 每帧都在分配内存	你在逐 entity 解析或加载资源。把引用烘焙好, 并缓存解析出的托管对象。
玩法 System 在运行时依赖 ScriptableObject	把需要的值烘焙进 component 数据或 blob asset。
构建时资源被剔除了	确保资源是经由 Baking/内容工作流被引用的, 而不只是靠运行时的字符串路径。

Chapter 35. foreach → IJobEntity Migration

1. 状态——已过时，尚未删除

老式的 component 遍历 API 在 Entities 1.4+ / 6.5 上已标为**过时**，但仍能编译：

遗留 API	6.5 中的状态	替代方案
Entities.ForEach	1.4 起就 过时 了；每用一次都报废弃警告；Unity 的 changelog 称会在未来某个大版本中移除	SystemAPI.Query<...>（主线程）或 IJobEntity（Job）
Job.WithCode	与 ForEach 一同 过时	用纯 IJob 结构体，或直接内联到 OnUpdate 里
IJobForEach (1.0 之前)	早在 1.4 之前就已移除	IJobEntity

你会读到这一章，要么是因为在 6000.5+ 里打开了 1.x 项目，编译器警告响个不停；要么是想趁未来某个大版本还没移除这个 API 之前，先把这些警告清掉。模式一旦看明白，这种重构就是照搬照改的事。

2. 两种替代 API

- **SystemAPI.Query<T, U, ...>**——在 OnUpdate 内做主线程遍历。简单，没有 Job 开销，适合小工作量或需要托管 API 的场合。
- **IJobEntity**——源码生成的并行 Job。准备好上多线程的话，它是直接的替代品——而多线程正是 DOTS 价值所在。

背景资料：../DOTS Workflows/11_IJobEntity · SystemAPI.Query.md。

3. 迁移——Entities.ForEach → SystemAPI.Query

3.1 迁移前

```
public partial class HealthRegenSystem : SystemBase
{
    protected override void OnUpdate()
    {
```

```

float dt = Time.DeltaTime;

Entities
    .WithAll<Alive>()
    .WithNone<Dead>()
    .ForEach((ref Health health, in Regen regen) =>
    {
        health.Value = math.min(health.Value + regen.Rate * dt, health.Max);
    })
    .ScheduleParallel();
}
}

```

3.2 迁移后——SystemAPI.Query (主线程)

```

[BurstCompile]
public partial struct HealthRegenSystem : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        float dt = SystemAPI.Time.DeltaTime;

        foreach (var (health, regen) in
            SystemAPI.Query<RefRW<Health>, RefRO<Regen>>()
                .WithAll<Alive>()
                .WithNone<Dead>())
        {
            health.ValueRW.Value = math.min(
                health.ValueRO.Value + regen.ValueRO.Rate * dt,
                health.ValueRO.Max);
        }
    }
}

```

形式变化说明：

- SystemBase → ISystem（更轻、能走 Burst）。要是旧代码捕获了托管字段，可能还得保留 SystemBase。
- ref Health → 元组元素 RefRW<Health>。用 .ValueRO 读，用 .ValueRW 写。
- in Regen → RefRO<Regen>。
- Time.DeltaTime → SystemAPI.Time.DeltaTime。
- 过滤条件 (WithAll/WithNone) 挪到 Query 构建器上。

4. 迁移——Entities.ForEach.ScheduleParallel → IJobEntity

当 ForEach 是并行调度的，迁成 Job 形式就是 1:1 的直接对应，并行性也保留了下来。

4.1 迁移后——IJobEntity

```
[BurstCompile]
public partial struct HealthRegenJob : IJobEntity
{
    public float DeltaTime;

    void Execute(ref Health health, in Regen regen)
    {
        health.Value = math.min(health.Value + regen.Rate * DeltaTime, health.Max);
    }
}

[BurstCompile]
public partial struct HealthRegenSystem : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        new HealthRegenJob { DeltaTime = SystemAPI.Time.DeltaTime }
            .ScheduleParallel();
    }
}
```

过滤条件以特性形式标在 Job 结构体上：

```
[WithAll(typeof(Alive))]
[WithNone(typeof(Dead))]
public partial struct HealthRegenJob : IJobEntity { ... }
```

或者在调度时把查询传进去：

```
var q = SystemAPI.QueryBuilder()
    .WithAll<Health, Regen, Alive>()
    .WithNone<Dead>()
    .Build();
new HealthRegenJob { DeltaTime = dt }.ScheduleParallel(q);
```

5. 捕获变量及其替代

老式的 Entities.ForEach 允许用 .WithReadOnly、.WithDisposeOnCompletion 之类捕获局部变量。到了 IJobEntity 里，被捕获的变量就成了 **Job 结构体上的字段**——Burst 清清楚楚地知道传进来的是什么。

```
// BEFORE
var buffer = new NativeArray<int>(count, Allocator.TempJob);
Entities.ForEach((ref Foo f) => { /* use buffer */ })
    .WithDisposeOnCompletion(buffer)
    .ScheduleParallel();

// AFTER
[BurstCompile]
partial struct FooJob : IJobEntity
{
    [ReadOnly] public NativeArray<int> Buffer;

    void Execute(ref Foo f) { /* use Buffer */ }
}

var buffer = new NativeArray<int>(count, Allocator.TempJob);
state.Dependency = new FooJob { Buffer = buffer }.ScheduleParallel(state.Dependency);
buffer.Dispose(state.Dependency); // chained dispose replaces WithDisposeOnCompletion
```

特性的对应替换：

- `.WithReadOnly(x)` → `[ReadOnly] public NativeArray<T> X;`
- `.WithDisposeOnCompletion(x)` → 调度之后调 `x.Dispose(handle)`。
- `.WithoutBurst()` → 给 Job 结构体加 `[WithoutBurst]`（但强烈建议把当初需要它的根因消除掉）。
- `.WithEntityAccess()` → 给 `Execute` 加一个 `Entity entity` 参数。

6. 何时仍需 `SystemBase`

如果原来的 `System` 确实带着托管字段（Input System、UI Toolkit、外部服务），就没法照搬着改成 `ISystem`。有两个办法：

1. **就让它保持 `SystemBase`**，在 `OnUpdate` 里用 `SystemAPI.Query`。单线程。
2. **拆开**：一个 `SystemBase` 负责衔接托管世界，另一个 `ISystem` 负责跑 `Burst` 的活儿——参见 [../DOTS Workflows/08_System - ISystem vs SystemBase.md](#)。

7. 验证

迁完一个 System 之后：

- 1. **对行为做单元测试。** 迁移不该改变逻辑；输出要是不一样，就说明重构带进了 bug。
- 2. **做性能分析。** 同等工作量下，并行 ForEach 和 IJobEntity.ScheduleParallel 的差距应在 5% 以内。退步明显，说明 Burst 没生效——查 Burst Inspector。
- 3. **把旧的 System 文件删掉。** 别留 #if LEGACY 这种回退——它只会慢慢烂掉。

8. 故障排查

现象	原因 / 解决方案
warning: Entities.ForEach is obsolete	这是 6.5 上的正常现象——代码照样编译。按上面的做法迁移，趁未来某个大版本移除该 API 之前把警告清掉。
error: 'SystemAPI' does not contain 'Query'	System 不是 partial，或者结构体没正确实现 ISystem。
IJobEntity 能编译，却不运行	结构体和 OnUpdate 上漏了 [BurstCompile]；源码生成器不会调度非 partial 的类型。
Job 读到的是过期的值	在调 .Schedule* 之前就捕获了局部变量。字段必须在调度之前赋值到 Job 结构体上。
迁移后性能退步	查查是不是 Burst 回退了（Burst Inspector），或者有个托管类型偷偷混进了字段。
迁移后只在编译期暴露问题	核对一下：被捕获的局部变量是否改用了特性来传递，而不是 lambda 闭包语法——后者只适用于 ForEach API。

Chapter 36. IAspect Deprecation — Migrating Off Aspects

1. 状态——已废弃，尚未删除

IAspect 在 Entities 1.4 中被标为**过时**，在 6.x 版本线上依旧过时。它还能编译、能运行，在 6.5 的 API 参考里也仍被列为合法的 IJobEntity.Execute 参数类型——但编译器会报废弃警告，而 Unity 已经声明这个类型会在**未来某个大版本中移除**。新代码别用它；现有代码可以见缝插针地迁移。

当初引入 IAspect，是想让 System 能为一组 component 声明一个"视图"，并在 Entities.ForEach 或 IJobEntity 里当成单个参数来用。可实践中，它多出了一层源码生成和一套平行的类型系统，收益抵不上代价：

- 写一个 Aspect，等于把 component 参数列表又抄了一遍。
- 调试时得在生成的包装代码里一步步走。
- IJobEntity 和 SystemAPI.Query 上的 RefRW<T> / RefRO<T>，不靠额外的类型就能达到同样"把 component 打包传递"的顺手效果。

由于这个接口仍然存在，升级 Editor 不会让代码失效——这次迁移主要是清理警告，并为 2.0 做准备。

2. 机械式重构

2.1 起始代码

```
using Unity.Entities;
using Unity.Mathematics;
using Unity.Transforms;

public readonly partial struct MovementAspect : IAspect
{
    public readonly RefRW<LocalTransform> Transform;
    public readonly RefRO<Velocity> Velocity;
    public readonly RefRO<Speed> Speed;

    public void Step(float dt)
    {
        Transform.ValueRW.Position += Velocity.ValueRO.Direction * Speed.ValueRO.Value * dt;
    }
}
```

```
public partial struct MovementSystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {
        float dt = SystemAPI.Time.DeltaTime;
        foreach (var m in SystemAPI.Query<MovementAspect>())
            m.Step(dt);
    }
}
```

2.2 迁移后——直接 component 参数

```
using Unity.Entities;
using Unity.Mathematics;
using Unity.Transforms;

public partial struct MovementSystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {
        float dt = SystemAPI.Time.DeltaTime;
        foreach (var (transform, velocity, speed) in
            SystemAPI.Query<RefRW<LocalTransform>, RefRO<Velocity>, RefRO<Speed>>())
        {
            transform.ValueRW.Position += velocity.ValueRO.Direction * speed.ValueRO.Value * dt;
        }
    }
}
```

两处变化：

1. Aspect 类型没了。它的字段变成了 SystemAPI.Query 的元组元素。
2. Aspect 上那个方法（Step）被内联进去。如果它太大、不适合内联，就抽出一个参数相同的**静态辅助函数**——这样仍对 Burst 友好：

```
static void Step(ref LocalTransform t, in Velocity v, in Speed s, float dt)
{
    t.Position += v.Direction * s.Value * dt;
}
```

调用方式：

```
Step(ref transform.ValueRW, velocity.ValueRO, speed.ValueRO, dt);
```

3. IJobEntity 中的 Aspect

3.1 迁移前

```
[BurstCompile]
public partial struct MoveJob : IJobEntity
{
    public float DeltaTime;
    void Execute(MovementAspect aspect) => aspect.Step(DeltaTime);
}
```

3.2 迁移后——component 作为 Execute 参数

```
[BurstCompile]
public partial struct MoveJob : IJobEntity
{
    public float DeltaTime;

    void Execute(ref LocalTransform transform, in Velocity velocity, in Speed speed)
    {
        transform.Position += velocity.Direction * speed.Value * DeltaTime;
    }
}
```

注意 `IJobEntity` 用的是普通的 `ref / in` 参数，而不是 `RefRW<T> / RefRO<T>`。（生成器会在底层把 component 包装成对 chunk 数组的访问。）

4. 包装 `DynamicBuffer` 的 `Aspect`

```
// BEFORE
public readonly partial struct PathAspect : IAspect
{
    public readonly DynamicBuffer<Waypoint> Path;
    public void Advance(int n) => Path.RemoveRange(0, n);
}

// AFTER – access buffer directly
foreach (var (path, e) in SystemAPI.Query<DynamicBuffer<Waypoint>>().WithEntityAccess())
{
    path.RemoveRange(0, 1);
}
```

在 Job 里，`DynamicBuffer<T>` 是合法的 `Execute` 参数——不用额外处理。

5. 跨 entity 的 `Aspect`

偶尔，`Aspect` 会包一个 `ComponentLookup<T>`，用来满足“我得查另一个 entity 的 component”这种需求。替代做法：

```
// BEFORE
public readonly partial struct AITargetAspect : IAspect
{
    public readonly RefRO<Target> Target;
    [ReadOnly] public readonly ComponentLookup<Health> HealthLookup;
    public float TargetHealth =>
        HealthLookup.TryGetComponent(Target.ValueRO.Entity, out var h) ? h.Value : 0f;
}

// AFTER – declare the lookup on the system / job directly
public partial struct AISystem : ISystem
{
    private ComponentLookup<Health> _healthLookup;

    public void OnCreate(ref SystemState state)
    {
        _healthLookup = state.GetComponentLookup<Health>(true);
    }

    public void OnUpdate(ref SystemState state)
    {
        _healthLookup.Update(ref state);

        foreach (var (target, e) in SystemAPI
            .Query<RefRO<Target>>()
            .WithEntityAccess())
        {
            float hp = _healthLookup.TryGetComponent(target.ValueRO.Entity, out var h)
                ? h.Value : 0f;
            // ...
        }
    }
}
```

这种写法也比 Aspect 版本更轻，因为这个查找表在所有 entity 之间共享。

6. Aspect 较复杂时

如果一个 Aspect 字段一大堆、封装了状态、或牵扯进不简单的继承关系——这就提示你该考虑一次真正的重构，而不只是平移。往往，把代码改成两个借 component 传数据的 System，会比一个揣着“神级 Aspect”的 System 更好读。

7. 故障排查

现象	原因 / 解决方案
编译警告 <code>IAspect is obsolete</code>	这是 6.5 上的正常现象。按上面的做法迁移即可清掉警告；在未来某个大版本移除它之前，代码照样能跑。

现象	原因 / 解决方案
type or namespace IAspect not found	你用的是已经移除 IAspect 的版本。把迁移做完——没有退路了。
Query 元组里类型太多，编译不过	SystemAPI.Query 支持很大的元组；但把一个超宽的查询拆成两个，或改用 IJobChunk，通常更好读。
Job 不再走 Burst 编译	某个 Aspect 方法可能捕获了 Burst 不支持的类型。直接把 Execute 参数重新检查一遍。
跨 entity 的 Aspect 简单平移后，运行很慢	ComponentLookup<T>.Update(ref state) 必须在 OnUpdate 里、使用前调用；忘了调，查找就会失效。
包装 DynamicBuffer 的 Aspect 不知怎么迁	DynamicBuffer<T> 本就能直接当 SystemAPI.Query 类型和 IJobEntity.Execute 参数用——根本不需要包装。
那些断言 Aspect 类型的测试失败了	照新签名把这些测试删掉或重写；它们没什么好留的。

Appendix. Changelogs

Appendix A. Entities 1.4 → 6.5 Key Changes

这是官方 `com.unity.entities` 包 changelog 的一份精选摘要，聚焦把 ECS 项目从 1.4 版本线迁到 6.5 Core Package 版本线时要紧的变化。本节只涉及 Entities 包本身的变化。

1. 版本轨道

Entities 6.x 与 Unity Editor 6000.x 对齐，以 Core Package 的形式发布。早先的 Entities 1.x 则是 Package Manager 包。

轨道	分发方式	示例
Entities 1.x	Package Manager (<code>com.unity.entities</code>)	官方 6.5 changelog 历史里的 1.4.2。
Entities 6.x	内嵌于 Editor 的 Core Package	6.4.0 / 6.5.0。

这层区别只关乎包的分发方式和版本管理。它不构成把无关的 UnityEngine 或 Editor 级别的迁移内容塞进 DOTS 编程文档的理由。

2. Entities 6.5.0

Entities 6.5.0 的官方包 changelog 条目是：

"Placeholder for 6.5.0"

把 6.5.0 当作与 Unity 6000.5 对齐的包版本即可。包 changelog 没有为这个版本列出新的 Entities API。

3. Entities 6.4.0

Entities 6.4.0 的官方包 changelog 条目是：

"Package is now a core package embedded in Unity."

实际影响：

区域	变化
安装	在 Unity 6000.4+ 上，你仍要从 Package Manager 添加 <code>com.unity.entities</code> ，但它从编辑器安装包中安装——没有版本可供选择。
版本选择	Entities 的版本由 Editor 安装决定。
锁定文件	<code>packages-lock.json</code> 会把它标成内置/核心包，而不是注册表依赖。
升级流程	替换掉 <code>com.unity.entities</code> 的任何固定 1.x 版本（从 Package Manager 重新添加），让 Editor 把它解析到 Core 版本。

参见 `../Migration/02_Package Manager → Core Package.md`。

4. Entities 1.4.2

随 `com.unity.entities@6.5` 一起附带的官方 changelog 历史，在 Core Package 迁移之前列有 1.4.2。

变更

变更	为何要在意
Burst 依赖更新到 1.8.24	给仍留在 1.4 版本线的项目做的一次依赖版本提升。
USS 类的尺寸单位从百分比改为像素	Editor UI 的样式细节。
字体大小字段从百分比改为像素	Editor UI 的样式细节。

修复

修复	为何要在意
<code>public partial SystemBase</code> 的源码生成错误	如果旧版 1.4 代码碰到过源码生成错误，这条就和你有关。
<code>Entities.ForEach</code> 加 <code>WithDeferredPlaybackSystem</code> 内部使用 <code>SystemAPI</code> 时报 <code>SGICE002</code>	从 <code>Entities.ForEach</code> 迁移时，这是一条有用的背景信息。
World 序列化内存泄漏	运行时/Editor 稳定性。
选中某个 Entity 时退出 Play Mode，Inspector 的初始化问题	Editor 工具的稳定性。

5. 值得迁移的 1.4.0 预发布条目

真正重要的 ECS API 走向，都体现在 1.4.0 的预览/预发布历史里。

区域	官方变更	迁移目标
迭代	Entities.ForEach 标为过时； 计划在未来某个大版本移除。	改用 IJobEntity 或 SystemAPI.Query。
Aspect	IAAspect 标为过时；计划在未来某个大版本移除。	改用直接的 component/查询 API。
可选 component 引用	ComponentLookup.GetRefRWOptional() / GetRefROOptional() 已废弃。	改用 TryGetRefRW<T>() / TryGetRefRO<T>()。
Chunk buffer	新增 ArchetypeChunk.GetBufferAccessorRO<T>() / GetBufferAccessorRW<T>()。	按你实际需要的访问模式来请求。
无类型 buffer	新增 GetUntypedBufferAccessorReinterpret<T>()。	只在源、目标的元素大小和 buffer 容量能安全别名时才用。
System 句柄	可以从 SystemHandle 取得 SystemTypeIndex。	更顺手的底层 System 访问。
查询工具	Disabled、Present、Absent、None 几列；Dependency 标签页；Query 窗口里对 Prefab 的区分。	查询的检查和调试更方便。
ECS 中的对象引用	新增了 UnityObjectRef 的手册文档。	非托管 ECS 数据里要引用托管 Unity 对象时，用 UnityObjectRef<T>。

6. 迁移检查清单

- ☐ 在 Unity 6000.4+ 上，从 Package Manager 重新添加 DOTS Core Package，替换掉 manifest.json 中固定的任何 1.x 版本。
- ☐ 把残留的 Entities.ForEach 换成 IJobEntity 或 SystemAPI.Query。
- ☐ 把 IAAspect 的用法换成直接的 component 访问。
- ☐ 把 GetRefRWOptional / GetRefROOptional 换成 TryGetRefRW<T>() / TryGetRefRO<T>()。
- ☐ 排查托管 Unity 对象引用，合适的地方改用 UnityObjectRef<T> 或烘焙数据。
- ☐ 迁移完成后，重新检查 Systems、Query 和 Entity Inspector 窗口。

7. 参考资料

- [Entities 6.5 Manual](#)
- [Entities 6.5 Changelog](#)

Appendix B. Netcode for Entities 1.4 → 6.5 Key Changes

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

这是一份面向迁移的阅读指南，梳理 Netcode for Entities 从 1.4 版本线到 6.5 Core Package 版本线一路累积下来的变化。Netcode 6.5.0 已作为内置包并入 Unity 6000.5 Editor，所以再往后的条目会转到 Editor 的发行说明里，而不再出现在这份包 changelog 中。

1. 版本亮点

6.5.0 (2026-02-05) ——内置 Editor 包

区域	变化
分发	从 Unity 6000.5 起，Netcode 变为内置的 Editor 包。
Entities 依赖	Changelog 条目里写的是 com.unity.entities 1.4.2 。注意把它和 DOTS 手册所针对的 Unity 6000.5 / Entities Core Package 区分开。
Profiler	Ghost Snapshot View 的 TreeView 标签新增了右键菜单，能快速查看 Prefab/Component。
源代码生成器	对 Rider 和 Visual Studio 关掉了 IDE 端的源码生成器执行，从而提升 IDE 性能。
破坏性变更	处理断连的 ECB 从 BeginSimulationECBS 改成了 NetworkGroupCommandBufferSystem。

1.12.0 (2026-02-01)

区域	变化
Profiler	基于 Tick 的导航、搜索字段，以及客户端 Prediction / Interpolation Tick 的显示。
Entities 依赖	从 1.4.2 升到 1.4.4 。
已废弃	基于浏览器的 Network Debugger 已废弃，转用 Netcode Profiler。
已删除	不再需要 NETCODE_PROFILER_ENABLED。
破坏性变更	GhostAuthoringComponentBaker 的 TransformUsageFlags 行为有变。

1.11.0 (2025-12-12)

区域	变化
Prefab List 菜单	快速 Ghost Prefab 检查。
GameObject Ghost	开始着手支持 GameObject 层的 Ghost。
Static Ghost 优化	对于未变化的数据，Static Ghost 的 Job 耗时大幅下降。
Ghost 迁移	IncludeInMigration 支持非 Ghost 数据迁移。
bool 序列化	打包/未打包 bool 的序列化开销有所改善。
API 破坏性变更	PrefabDebugName.Name 彻底废弃。

1.10.0 (2025-11-09)

区域	变化
零大小 component	保存 World 状态时支持零大小 component。
源代码生成器	关闭了 IDE 端的执行；这一点在 6.5.0 的行为里也有体现。
修复	修复了 PredictedGhostSpawnSystem 断言、Sleep 模式刷屏警告、AlwaysRun 预测循环异常等一系列问题。

1.9.x (2025-09 至 2025-10)

区域	变化
Ghost 优化文档	新增了专门的 Ghost 优化文档。
稳定化	1.8.0 的 Profiler 模块和 Importance Visualizer 转为稳定版。

1.8.0 (2025-08-17)

区域	变化
Importance Visualizer	加入了 PlayMode Tool。
强制输入延迟	新增 ClientTickRate.ForcedInputLatencyTicks。
Profiler 模块	面向 Unity 6+ 的实验性 Server / Client Profiler 模块。
破坏性变更	GhostDistanceImportance 的缩放函数不再把 baseImportance 乘以 1000。
Transport	更新至 com.unity.transport 2.5.3。

1.7.0 (2025-07-29)

区域	变化
Host Migration	移除了 <code>ENABLE_HOST_MIGRATION</code> 宏；Host Migration 默认开启。
Host Migration API	类名和方法名经过重构；确切的类型名请查当前的包文档或示例。
序列化	为配合 Host Migration，改进了 Ghost component 的序列化。
Transport	更新至 <code>com.unity.transport 2.1.0</code> 。

1.6.1 至 1.6.2 (2025-05 至 2025-07)

区域	变化
Predicted ECB	为 <code>PredictedSimulationSystemGroup</code> 新增了 <code>begin/end</code> 两个 ECB 系统。
Predicted 生成	在 <code>EndPredictedSimulationCommandBufferSystem</code> 之后新增了 <code>PredictedSpawningSystemGroup</code> 。
Host Migration	作为实验性功能，藏在 <code>ENABLE_HOST_MIGRATION</code> 宏后面。
重连追踪	新增了 <code>NetworkStreamIsReconnected</code> component。
快照历史	<code>GhostSystemConstants.SnapshotHistorySize</code> 现在可以用编译器宏来配置。
Relevancy + Importance	新增了一条快速路径，支持把 Ghost Relevancy 和 Importance Scaling 结合起来用。

1.5.0 (2025-04-22)

区域	变化
Thin Client	新增了 <code>AutomaticThinClientWorldsUtility</code> ，用于在运行时创建 Thin Client。
序列化	支持复制 <code>FixedList</code> 和 <code>unsafe</code> 固定缓冲区。
基线	新增了 <code>GhostAuthoringComponent.UseSingleBaseline</code> ，可用来省下 CPU 开销。
整数序列化	新增 <code>byte / short</code> 的专用模板。
Lag Compensation	物理历史缓冲区从 16 个 Tick 扩到 32 个。
Relay	Relay 的包路径转向了 <code>com.unity.services.multiplayer v1.1.0</code> 。

1.4.0 (2024-11-14)

区域	变化
发送速率	<code>GhostAuthoringComponent.MaxSendRate</code> 用来限制 Ghost 的发送频率。
Chunk 迭代	<code>GhostSendSystemData.MaxIterateChunks</code> 用来限制每个 Tick 处理的 Chunk 数量。
NetCodeConfig	新增了多个 Unity Transport 的 <code>NetworkConfigParameters</code> 。

区域	变化
快照 ACK	ACK 历史容量改为可配置。
修复	修复了超过 256 个 Tick 时的快照 ACK 问题、物理插值抖动等。

2. 累积破坏性变更

版本	破坏性变更
6.5.0	处理断连的 ECB 从 BeginSimulationECBS 改成了 NetworkGroupCommandBufferSystem。
1.12.0	GhostAuthoringComponentBaker 的 TransformUsageFlags 行为有变。
1.11.0	PrefabDebugName.Name 彻底废弃。
1.8.0	GhostDistanceImportance 的乘数行为有变。
1.7.0	Host Migration API 的类名和方法名经过重构。
1.5.0	命令校验收紧，握手超时的行为有变。
1.4.0	MaxSendChunks 的作用范围有变。

3. 值得记住的趋势

1. **调试以 Profiler 为先**——旧版基于浏览器的 Network Debugger 已废弃；请改用 Netcode Profiler 模块。
2. **Host Migration 成了默认行为**——1.6.1 时还是实验性功能，1.7.0+ 起默认开启。
3. **IDE 性能变好**——Netcode 的源码生成器不再在 IDE 分析过程中运行。
4. **序列化更高效**——bool 打包、byte/short 模板和单基线选项都减轻了 CPU 和带宽的压力。
5. **转入 Core Package**——Netcode 6.5.0 随 Unity 6000.5 一同发布；往后的变更会转到 Editor 发行说明里。

4. 相关文档

- `../DOTS Workflows/16_Netcode Client-Server World & Bootstrap.md`
 - `../DOTS Workflows/18_Netcode Ghost Snapshot & Synchronization.md`
 - `../DOTS Workflows/22_Netcode Ghost Optimization · Importance · Relevancy.md`
 - `../DOTS Workflows/24_Netcode Profiler & Debugging.md`
-

5. 参考资料

- [Netcode for Entities 6.5 Manual](#)
- [Netcode for Entities 6.5 Changelog](#)
- [Netcode for Entities 1.12 Changelog](#)

Reference. Glossary

Glossary

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

本书中每个不那么直白的术语，都附一句话定义。交叉链接指向该术语详细定义所在的文档。

1. 核心名词——ECS 的五大构成要素

术语	定义	参见
Entity	一个不透明的 64 位 ID (struct Entity, 位于 Unity.Entities 命名空间)。没有数据，也没有行为——只是用来查找 component 的一个键。	DOTS Workflows/03_ECS Core Concepts.md
Component	挂在 entity 上的数据。有好几种——见第 2 节。	DOTS Workflows/05_Component Types.md
Archetype	一个 entity 所拥有的那套 component 类型组合。archetype 相同的 entity 共用同一片存储。	DOTS Workflows/03_ECS Core Concepts.md
Chunk	装着同一 archetype 一批 entity 的 16 KB 数据块。chunk 内部，component 按列优先 (SoA) 存放。	Optimizations and Debugging/01_Chunk Layout & TypeManager.md
World	装 entity 和 System 的容器。运行时启动时会建一个“默认 World”。	DOTS Workflows/03_ECS Core Concepts.md

2. Component 种类

种类	含义
IComponentData	非托管的结构体 component——默认类型。
IBufferElementData	挂在 entity 上的 DynamicBuffer<T> 的元素类型。
ISharedComponentData	以值为键的 component——值相同的 entity 共用一个 chunk 桶。
ICleanupComponentData	能在 DestroyEntity 之后继续存留的 component，好让 System 在 entity 彻底消失前做收尾。
标签 component	空的 IComponentData 结构体——占零字节，纯粹拿来当查询过滤条件。
Chunk component	逐 chunk 的数据（每个 chunk 一份，而不是每个 entity 一份）。
IEnableableComponent	靠逐 chunk 位掩码来开关“是否存在”的 component—— 不涉及结构性变更 。

详情: DOTS Workflows/05_Component Types.md · DOTS Workflows/06_Enableable Component.md。

3. System 与 Job

术语	含义
ISystem	新版的、基于结构体的 System 接口。兼容 Burst。 写新 System 时首选它。
SystemBase	旧版的、基于类的 System 接口。只在 System 需要主线程托管访问时才用。
SystemGroup	有序排列 System 的容器。主要的几个组: InitializationSystemGroup、SimulationSystemGroup、PresentationSystemGroup。
IJobEntity	遍历查询所匹配 entity 的 Job。由源码生成，能被 Burst 编译。
IJobChunk	遍历 chunk（而非单个 entity）的 Job——用来做逐 chunk 的聚合。
SystemAPI	System 内部用的一组静态辅助 API: Query<>()、GetComponentLookup<T>()、Time、GetSingleton<T>() 等等。

详情: DOTS Workflows/08_System – ISystem vs SystemBase.md · DOTS Workflows/11_IJobEntity · SystemAPI.Query.md · DOTS Workflows/12_IJobEntity vs IJobChunk.md。

4. Authoring → 运行时

术语	含义
Baker	在 SubScene 保存或构建时，把 Authoring 用的 GameObject 加 MonoBehaviour 转换成 ECS component。
SubScene	Authoring 的容器——其中的 GameObject 会被烘焙成 entity，并序列化进一个 EntityScene 资源。
EntityScene	由 SubScene 烘焙生成的资源——运行时会被加载进 World。

详情: DOTS Workflows/01_Baker Pattern & SubScene.md。

5. 结构性变更安全

术语	含义
结构性变更	增删 component，或创建/销毁 entity。会让 entity 在 chunk 间搬动——开销大；遍历过程中做它不安全。
EntityCommandBuffer (ECB)	遍历期间把结构性变更记下来，到安全的边界处再回放。
EntityCommandBuffer.ParallelWriter	供并行 Job 用的线程安全 ECB 变体；每次调用都要传 sortKey，以保证回放的确定性。
延迟 entity	由 ecb.CreateEntity() / Instantiate() 创建、此刻还不存在的 entity——它的占位索引在 ECB 回放时才解析。
排序键	传给每条并行 ECB 命令的一个整数；默认用 [ChunkIndexInQuery]。

详情: DOTS Workflows/13_Structural Change & Safety.md · DOTS Workflows/14_EntityCommandBuffer · Deferred Entity.md · DOTS Workflows/15_ParallelWriter · Deterministic Playback.md。

6. Entity 引用类型

下面是 ECS 数据和 Baking 工作流里会用到的几种 DOTS 引用类型。

类型	含义
Entity	ECS 句柄 (Unity.Entities)。说白了就是 ECS World 里的一个 ID。
EntityPrefabReference	指向一份已烘焙 entity Prefab 资源的引用，用于内容/流式工作流 (Unity.Entities.Serialization)。
UnityObjectRef<T>	ECS 侧对 UnityEngine.Object 的引用——让 entity 能指向材质、网格这类托管资源，又不必装箱。

详情: DOTS Workflows/04_Entity References – Entity · EntityPrefabReference · UnityObjectRef.md。

7. Netcode for Entities

术语	含义	参见
Netcode for Entities	Unity 的 DOTS 多人联网层，面向带客户端预测的服务器权威游戏。	DOTS Workflows/16_Netcode Client-Server World & Bootstrap.md

术语	含义	参见
Server World	负责跑服务器模拟、发送 Ghost 快照的权威 ECS World。	DOTS Workflows/16_Netcode Client-Server World & Bootstrap.md
Client World	负责接收快照、发送命令、预测本地玩法并呈现插值状态的 ECS World。	DOTS Workflows/16_Netcode Client-Server World & Bootstrap.md
Thin Client World	一种轻量的虚拟客户端 World，用来测试负载和连接行为。	DOTS Workflows/16_Netcode Client-Server World & Bootstrap.md
NetworkStream Connection	连接 entity 上的 component，保存着 Transport 连接句柄。	DOTS Workflows/17_Netcode Network Connection & Approval.md
NetworkId	审批通过后由服务器分配的连接 ID。	DOTS Workflows/17_Netcode Network Connection & Approval.md
NetworkStream InGame	用来开启游戏数据交换的 component；缺了它，快照和命令都流不起来。	DOTS Workflows/17_Netcode Network Connection & Approval.md
Ghost	一种服务器权威的网络 entity，通过快照复制到客户端。	DOTS Workflows/18_Netcode Ghost Snapshot & Synchronization.md
Snapshot	由服务器发往客户端的序列化 Ghost 状态，通常走不可靠的 Transport 传输。	DOTS Workflows/18_Netcode Ghost Snapshot & Synchronization.md
GhostField	一个特性，用来标出参与 Ghost 序列化的 component 字段。	DOTS Workflows/18_Netcode Ghost Snapshot & Synchronization.md
PredictedGhost	用来标识参与客户端预测的 Ghost 的 component。	DOTS Workflows/19_Netcode Prediction & Rollback.md
Simulate	Netcode 用的一个可启用标签，标出哪些预测 entity 该在当前预测 Tick 上运行。	DOTS Workflows/19_Netcode Prediction & Rollback.md
IInputComponentData	偏高层的输入 component 类型，命令缓冲区的管理由生成的代码代劳。	DOTS Workflows/20_Netcode Command Stream & Input.md
ICommandData	偏底层的命令流类型，Tick 映射和缓冲区访问都得手动管理。	DOTS Workflows/20_Netcode Command Stream & Input.md
InputEvent	跳跃、射击这类事件用的一次性输入标记，必须在目标 Tick 上恰好注册一次。	DOTS Workflows/20_Netcode Command Stream & Input.md
IRpcCommand	可靠的一次性 RPC 消息的类型。	DOTS Workflows/21_Netcode RPC.md
IApprovalRpcCommand	一种特殊 RPC 类型，只在连接审批期间允许使用。	DOTS Workflows/21_Netcode RPC.md
Ghost Importance	服务器端的优先级分值，用于决定哪些 Ghost 优先纳入快照预算。	DOTS Workflows/22_Netcode Ghost Optimization · Importance · Relevancy.md

术语	含义	参见
Ghost Relevancy	一种逐连接的过滤，决定某个 Ghost 该不该复制给特定客户端。	DOTS Workflows/22_Netcode Ghost Optimization · Importance · Relevancy.md
Lag Compensation	服务器端去查询历史碰撞世界，从而按客户端感知到的时间来校验它的操作。	DOTS Workflows/23_Netcode Physics Integration & Lag Compensation.md

8. 遗留 / 过时（不应出现在新代码中）

术语	6.5 中的状态	迁移目标
<code>Entities.ForEach</code>	1.4 起过时； 仍能带警告编译； 计划在未来某个大版本中移除。	<code>SystemAPI.Query<>()</code> 或 <code>IJobEntity</code> ——见 Migration/04_foreach → IJobEntity.md
<code>IAAspect</code>	1.4 起过时； 仍能带警告编译； 计划在未来某个大版本中移除。	直接的 component 参数 (<code>RefRW<T></code> / <code>RefRO<T></code>) ——见 Migration/05_IAAspect Removal.md
<code>ComponentLookup.GetRefRWOptional</code>	已过时。	<code>TryGetRefRW</code> ——见 Migration/01_Entities 1.x → 6.5 Overview.md

9. 辅助概念

术语	含义
Burst	Unity 为 Job 和标了 <code>[BurstCompile]</code> 的结构体准备的原生编译器。它生成原生的 SIMD 代码。
变更版本	一个 32 位计数器，每当 System 写入 chunk 里某个 component 列时就加一。它让 <code>SetChangedVersionFilter</code> 得以跳过没变过的 chunk。
SoA（数组结构体）	一种存储布局：每个字段各自成为一个数组——对标量遍历很缓存友好。Chunk 用的就是 SoA。
<code>RefRW<T></code> / <code>RefRO<T></code>	在 <code>SystemAPI.Query</code> 里以读写/只读意图访问 component 的包装类型。
<code>EnabledRefRW<T></code> / <code>EnabledRefRO<T></code>	用来读写可启用 component 的开关位的包装器，且不会触发结构性变更。
TypeManager	一张运行时注册表，为每个 component 分配一个稳定的类型索引。它在启动时重建；大型项目能明显感觉到这一点。
<code>BlobAssetReference<T></code>	一个指针，指向存放在 chunk 之外的大块只读数据 blob，免得把 chunk 容量撑大。

